

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل یازدهم: Probabilistic Information Retrieval

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

11.1 Review of basic probability theory

11.2 The Probability Ranking Principle

11.2.1 The 1/0 loss case

11.2.2 The PRP with retrieval costs

11.3 The Binary Independence Model

11.3.1 Deriving a ranking function for query terms

11.3.2 Probability estimates in theory

11.3.3 Probability estimates in practice

11.3.4 Probabilistic approaches to relevance feedback

11.4 An appraisal and some extensions

11.4.1 An appraisal of probabilistic models

11.4.2 Tree-structured dependencies between terms

11.4.3 Okapi BM25: a non-binary model

11.4.4 Bayesian network approaches to IR

فصل ۱۱: بازیابی اطلاعات احتمالاتی

در فصل‌های قبل، مدل‌های Boolean و فضای برداری برای بازیابی اطلاعات معرفی شدند. این مدل‌ها مانند یک کلیداند که فقط در صورت تطابق کامل قفل را باز می‌کنند، اما در دنیای واقعی، جستجو اغلب مبهم و ناقص است. در این فصل، رویکردی متفاوت و اصولی بر پایهٔ **نظریهٔ احتمال** ارائه می‌شود که به سیستم اجازه می‌دهد تحت **عدم قطعیت** تصمیم‌گیری کند.

مثال واقعی: استفادهٔ آموزشی از IBM Watson در تشخیص پزشکی

در بیمارستان‌های آموزشی، دانشجویهای پزشکی هنگام کار با بیماران واقعی از Watson به عنوان یک ابزار «تصمیم‌یار» استفاده می‌کردند. هنگامی که دانشجوی علائمی مثل تنگی نفس، سرفه خشک یا کاهش اکسیژن خون را وارد می‌کرد، Watson فقط کلمات را مطابقت نمی‌داد، بلکه با توجه به داده‌های میلیون‌ها پروندهٔ قبلی و مدل‌های آماری خود، احتمال هر بیماری را محاسبه می‌کرد. برای مثال، اگر بیمار سرفه خشک، تنگی نفس و افت اکسیژن داشت، Watson احتمال حملهٔ آسم را بالاتر از آنفولانزا قرار می‌داد و گزینه‌هایی مثل COVID-19 یا ذات‌الریه را نیز با درصدی مشخص نمایش می‌داد. این تحلیل احتمالاتی به دانشجوی کمک می‌کرد بفهمد کدام تشخیص‌ها «محتمل‌تر» هستند و چه آزمایش‌های تکمیلی منطقی‌تر است.

در مدل احتمالاتی، اگر اسناد مرتبط و نامرتبتي در اختیار داشته باشیم، می‌توانیم احتمال ظاهر شدن یک واژه در سند مرتبط را تخمین بزنیم:

$$P(t \mid R = 1)$$

این مقدار می‌تواند پایه‌ای برای ساخت یک **طبقه‌بند** باشد که تصمیم می‌گیرد آیا سندی مرتبط است یا نه. در این فصل، این رویکرد به صورت سیستماتیک معرفی می‌شود.

مروری بر نظریه پایه‌ی احتمال

نظریه احتمال به ما کمک می‌کند تا در دنیای عدم قطعیت، تصمیمات منطقی بگیریم. متغیر تصادفی A نمایانگر یک رویداد است. احتمال وقوع آن بین ۰ تا ۱ است:

$$0 \leq P(A) \leq 1$$

مثال: فیلتر اسپم در جیمیل

جیمیل از نظریه احتمال برای فیلتر کردن ایمیل‌های اسپم استفاده می‌کند. برای هر ایمیل، سیستم احتمال اینکه آن اسپم باشد را محاسبه می‌کند. اگر ایمیل حاوی کلماتی مانند «ارائه ویژه» یا «برنده» باشد، احتمال اسپم بودن آن افزایش می‌یابد. این احتمال با در نظر گرفتن ویژگی‌های دیگر مانند فرستنده و تاریخچه ایمیل دقیق‌تر می‌شود.

برای دو رویداد A و B، احتمال مشترک آن‌ها با قانون زنجیره‌ای (Chain Rule) تعریف می‌شود:

$$P(A, B) = P(A \cap B) = P(A \mid B) \cdot P(B) = P(B \mid A) \cdot P(A)$$

قانون تقسیم (Partition Rule) بیان می‌کند که اگر B به زیرمجموعه‌های مجزا تقسیم شود، آنگاه:

$$P(B) = P(A, B) + P(\neg A, B)$$

اصل رتبه‌بندی احتمالاتی

اصل رتبه‌بندی احتمالاتی (Probability Ranking Principle) بیان می‌کند که اسناد باید بر اساس احتمال مرتبط بودن رتبه‌بندی شوند. این اصل، پایه بسیاری از مدل‌های بازیابی اطلاعات مدرن است.

مثال: الگوریتم PageRank گوگل

الگوریتم PageRank که گوگل در ابتدا از آن استفاده می‌کرد، بر اساس اصل رتبه‌بندی احتمالاتی عمل می‌کند. PageRank احتمال اینکه کاربر با کلیک روی یک لینک به صفحه‌ای خاص برسد را محاسبه می‌کند و صفحات را بر اساس این احتمال رتبه‌بندی می‌کند. صفحاتی که احتمال بازدید بیشتری دارند، در رتبه‌های بالاتر قرار می‌گیرند.

قانون بیز (Bayes' Rule): برای معکوس کردن احتمال شرطی استفاده می‌شود:

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)} = \frac{P(B | A) \cdot P(A)}{\sum_{X \in \{A, \neg A\}} P(B | X) \cdot P(X)}$$

مثال: سیستم توصیه نتفلیکس

نتفلیکس از قانون بیز برای توصیه فیلم استفاده می‌کند. اگر کاربری فیلم‌های اکشن را دوست داشته باشد (احتمال اولیه)، نتفلیکس با توجه به اینکه کاربر اخیراً به فیلم‌های ابرقهرمانی علاقه نشان داده (شواهد)، احتمال پس از مشاهده شواهد را به‌روز می‌کند. این رویکرد به نتفلیکس اجازه می‌دهد توصیه‌های دقیق‌تری به کاربر ارائه دهد.

در این فرمول:

- $P(A)$: احتمال اولیه (prior)
- $P(A|B)$: احتمال پس از مشاهده شواهد (posterior)
- $P(B|A)$: احتمال مشاهده شواهد در صورت صحت فرض

مدل استقلال دودویی

مدل استقلال دودویی (Binary Independence Model) یکی از اولین و تأثیرگذارترین مدل‌های بازیابی احتمالاتی است. این مدل فرض می‌کند که واژه‌ها در اسناد به صورت مستقل از هم ظاهر می‌شوند.

مثال: موتور جستجوی علمی PubMed

PubMed که برای جستجوی مقالات علمی پزشکی استفاده می‌شود، از مدل استقلال دودویی برای رتبه‌بندی نتایج استفاده می‌کند. این مدل فرض می‌کند که ظاهر شدن واژه‌ها در یک مقاله، مستقل از ظاهر شدن واژه‌های دیگر است. اگرچه این فرض ساده‌شده است، اما در عمل نتایج قابل قبولی ارائه می‌دهد و پایه بسیاری از سیستم‌های پیشرفته‌تر است.

در فصل‌های بعدی، مدل‌های پیشرفته‌تری مانند Okapi BM25 و مدل‌های شبکه بیزی معرفی خواهند شد که این فرض ساده را بهبود می‌بخشند.

۱۱.۲.۱ حالت Loss 1/0 در اصل رتبه‌بندی احتمالاتی (PRP)

در این بخش، فرض می‌کنیم که سیستم بازیابی اطلاعات در یک چارچوب رتبه‌دار عمل می‌کند (مطابق با فصل ۶.۳): کاربر یک پرسمان ارسال می‌کند و سیستم فهرستی مرتب‌شده از اسناد را باز می‌گرداند. همچنین فرض می‌کنیم که مرتبط بودن به صورت دودویی تعریف شده است (مطابق فصل ۸): هر سند یا مرتبط است یا نیست.

مثال واقعی: سیستم اولویت‌بندی ایمیل در Gmail

در سرویس Gmail، بخش «Important» بر پایه یک مدل احتمالاتی عمل می‌کند. سیستم برای هر ایمیل ورودی بررسی می‌کند که چقدر احتمال دارد این ایمیل برای کاربر مهم باشد: این احتمال از ترکیبی از داده‌هایی مثل فرستنده، سابقه پاسخ‌گویی کاربر، کلمات کلیدی، و رفتارهای گذشته محاسبه می‌شود. سپس ایمیل‌ها بر اساس همین احتمال اهمیت رتبه‌بندی می‌شوند. در این مدل، هر ایمیل عملاً برچسبی دودویی دارد (مهم یا غیرمهم) و Gmail فهرست را طوری مرتب می‌کند که ایمیل‌هایی با احتمال «مهم بودن» بالاتر، در بالاترین رتبه‌ها نمایش داده شوند.

برای هر پرسمان q و سند d ، متغیر تصادفی $R_{d,q}$ نشان می‌دهد که آیا سند d نسبت به پرسمان q مرتبط است یا نه:

$$R_{d,q} = \begin{cases} 1 & \text{if document } d \text{ is relevant to query } q \\ 0 & \text{otherwise} \end{cases}$$

در این مدل، طبیعی‌ترین روش برای رتبه‌بندی اسناد، مرتب‌سازی آن‌ها بر اساس احتمال مرتبط بودن است:

$$P(R = 1 \mid d, q)$$

این ایده، پایه اصل رتبه‌بندی احتمالاتی (Probability Ranking Principle) است که توسط van Rijsbergen (1979) معرفی شد:

اگر یک سیستم بازیابی اطلاعات، در پاسخ به هر پرسمان، اسناد را به ترتیب احتمال نزولی مرتبط بودن با نیاز اطلاعاتی کاربر مرتب کند – به طوری که این احتمال‌ها تا حد ممکن دقیق تخمین زده شده باشند – آنگاه کارایی کلی سیستم برای کاربر، در مقایسه با هر روش دیگری که فقط از همان داده‌ها استفاده کند، بیشینه خواهد بود.

در ساده‌ترین حالت، هیچ هزینه‌ای برای بازیابی یا سودمندی خاصی در نظر گرفته نمی‌شود. فقط دو نوع خطا وجود دارد:

- بازگرداندن سند نامرتب
- عدم بازگرداندن سند مرتبط

این وضعیت به عنوان **Loss 1/0** شناخته می‌شود: هر خطای واحد هزینه دارد. هدف، بازگرداندن بهترین اسناد ممکن در میان k سند اول است – برای هر مقداری از k که کاربر بررسی می‌کند.

مثال: سیستم اولویت‌بندی اخبار در اپلیکیشن‌های خبری

بسیاری از اپلیکیشن‌های خبری (مانند Google News یا Apple News) از مدل‌های احتمالاتی برای رتبه‌بندی اخبار استفاده می‌کنند. برای هر خبر، احتمال اینکه کاربر به آن علاقه‌مند باشد محاسبه می‌شود. این احتمال بر اساس سابقه مطالعه کاربر، موضوعات مورد علاقه، و اخبار مشابهی که قبلاً خوانده، تعیین می‌گردد. سپس اخبار با احتمال بالاتر در بالای فید کاربر نمایش داده می‌شوند. این سیستم بر اساس Loss 1/0 عمل می‌کند: نشان دادن یک خبر نامرتب در بالا (False Positive) ممکن است باعث از دست رفتن علاقه کاربر شود، اما عدم نمایش یک خبر بسیار مهم (False Negative) هزینه بیشتری دارد. هدف، کمینه کردن این خطاها و ارائه بهترین خبرها در ابتدای فهرست است.

در این حالت، PRP می‌گوید که باید اسناد را به ترتیب نزولی $P(R = 1 \mid d, q)$ مرتب کنیم.

اگر به جای رتبه‌بندی، بخواهیم مجموعه‌ای از اسناد را بازگردانیم، **قانون تصمیم بهینه بیزی (Bayes Optimal Decision Rule)** می‌گوید:

Doc d It is relevant if and only if $P(R = 1 | d, q) > P(R = 0 | d, q)$

این قانون تصمیم، **ریسک مورد انتظار** را در حالت Loss 1/0 کمینه می‌کند.

مثال: سیستم تشخیص تقلب در تراکنش‌های بانکی

بانک‌ها برای تشخیص تراکنش‌های جعلی از مدل‌های احتمالاتی پیچیده استفاده می‌کنند. سیستم برای هر تراکنش، احتمال اینکه آن تراکنش «تقلب‌آمیز» باشد را محاسبه می‌کند. اگر این احتمال از یک آستانه مشخص (که بر اساس تحلیل هزینه خطاها تعیین شده) بالاتر باشد، تراکنش مسدود شده یا برای تأیید بیشتر علامت‌گذاری می‌شود. این استفاده مستقیم از قانون تصمیم بیزی (Bayes Optimal Decision Rule) است؛ جایی که هزینه رد کردن یک تراکنش صحیح (False Positive) و هزینه پذیرش یک تراکنش جعلی (False Negative) به دقت محاسبه و در تصمیم‌گیری لحاظ می‌شود تا ریسک کلی به حداقل برسد.

قضیه ۱۱.۱: اصل رتبه‌بندی احتمالاتی (PRP) در حالت Loss 1/0 بهینه است، به این معنا که **ریسک مورد انتظار** را کمینه می‌کند.

اثبات این قضیه در Ripley (1996) آمده است، اما فرض می‌کند که همه احتمالات به درستی شناخته شده‌اند – که در عمل هرگز صادق نیست. با این حال، PRP همچنان پایه‌ای بسیار مفید برای طراحی مدل‌های بازیابی اطلاعات محسوب می‌شود.

۱۱.۲.۲ اصل رتبه‌بندی احتمالاتی با هزینه‌های بازیابی

در حالت ساده PRP، فرض می‌شود که همه خطاها هزینه یکسانی دارند (Loss 1/0). اما در بسیاری از کاربردهای واقعی، هزینه بازگرداندن سند نامرتب با هزینه عدم بازگرداندن سند مرتبط یکسان نیست. در این بخش، مدل رتبه‌بندی با در نظر گرفتن **هزینه‌های بازیابی** معرفی می‌شود.

مثال واقعی: سیستم پیشنهاد محصول در فروشگاه‌های آنلاین

در فروشگاه‌های آنلاین مانند دیجی‌کالا، نمایش یک محصول نامرتب به کاربر (False Positive) ممکن است باعث اتلاف وقت و کاهش احتمال خرید شود، اما عدم نمایش یک محصول بسیار مرتبط که کاربر به دنبال آن است (False Negative) می‌تواند منجر به از دست دادن یک فروش قطعی شود. بنابراین، سیستم‌های پیشنهاد محصول طوری طراحی می‌شوند که هزینه خطای False Negative (از دست دادن فروش) را بسیار بیشتر از False Positive در نظر بگیرند تا شانس موفقیت در فروش را به حداکثر برسانند.

فرض کنیم:

- C_1 : هزینه عدم بازیابی یک سند مرتبط (false negative)
- C_0 : هزینه بازیابی یک سند نامرتب (false positive)

در این مدل، PRP می‌گوید سندی را باید انتخاب کنیم که **ریسک مورد انتظار** آن کمینه باشد. برای سند d ، اگر برای همه سندهای d' که هنوز بازیابی نشده‌اند، رابطه زیر برقرار باشد، آنگاه d باید سند بعدی باشد که بازیابی می‌شود:

$$C_0 \cdot P(R = 0 | d) - C_1 \cdot P(R = 1 | d) \leq C_0 \cdot P(R = 0 | d') - C_1 \cdot P(R = 1 | d')$$

این مدل یک چارچوب رسمی برای مدل‌سازی هزینه‌های متفاوت خطاها فراهم می‌کند – برخلاف فصل ۸.۶ که فقط در مرحله ارزیابی به این موضوع پرداختیم، اینجا در **مرحله مدل‌سازی** لحاظ می‌شود.

🔗 مثال: سیستم اولویت‌بندی وظایف در نرم‌افزارهای مدیریت پروژه

در نرم‌افزارهای مدیریت پروژه مانند Asana یا Trello، گاهی لازم است بین وظایف مختلف اولویت‌بندی صورت گیرد. عدم پیگیری یک وظیفه حیاتی (False Negative) می‌تواند منجر به تأخیر در پروژه و هزینه‌های هنگفتی شود، در حالی که صرف زمان اندکی برای بررسی یک وظیفه کم‌اهمیت (False Positive) ممکن است هزینه کمتری داشته باشد. بنابراین، سیستم اولویت‌بندی، وظایف را بر اساس ترکیبی از فوریت و اهمیت، و با در نظر گرفتن هزینه بالاتر خطاهای False Negative، مرتب می‌کند تا اطمینان حاصل شود که مهم‌ترین کارها در اولویت اجرا قرار گیرند.

با این حال، در ادامه فصل، مدل‌های سود/زیان پیچیده‌تر بررسی نمی‌شوند. اما این بخش نشان می‌دهد که PRP می‌تواند به‌صورت تعمیم‌یافته برای شرایط واقعی‌تر نیز استفاده شود.

۱۱.۳ مدل دودویی استقلال (Binary Independence Model)

مدل دودویی استقلال (BIM) یکی از مدل‌های کلاسیک بازیابی اطلاعات است که با اصل رتبه‌بندی احتمالاتی (PRP) ترکیب می‌شود. این مدل با فرض‌های ساده‌ای طراحی شده که محاسبه احتمال مرتبط بودن سند را عملی و قابل پیاده‌سازی می‌کند.

🔍 مثال واقعی: سیستم پیشنهاد فیلم در سرویس‌های استریم

سرویس‌های استریم مانند نتفلیکس یا فیلیمو برای پیشنهاد فیلم و سریال به کاربران از مدل‌های مشابه دودویی استقلال استفاده می‌کنند. این سیستم‌ها با تحلیل حضور یا عدم حضور کلمات کلیدی در توضیحات فیلم، ژانرها، بازیگران و کارگردانان، احتمال مرتبط بودن یک فیلم با علایق کاربر را تخمین می‌زنند. به عنوان مثال، اگر کاربر به دنبال فیلم‌های “علمی-تخیلی” و “اکشن” باشد، سیستم فیلم‌هایی را که هر دو کلمه کلیدی را در توضیحات خود دارند، در اولویت نمایش قرار می‌دهد. این مدل، اگرچه فرض استقلال بین کلمات را دارد، اما در عمل برای ارائه پیشنهادهاى اولیه بسیار کارآمد است.

در این مدل:

- نمایش سند و پرسمان به‌صورت بردار دودویی از واژه‌ها است: اگر واژه t در سند d وجود داشته باشد، $x_t = 1$ و در غیر این صورت $x_t = 0$.
- نمایش پرسمان نیز به‌صورت بردار وقوع واژه‌ها است: $\vec{q}$
- فرض استقلال: وقوع واژه‌ها در سندها مستقل از یکدیگر فرض می‌شود (مانند فرض ساده‌انگارانه Naive Bayes).

این فرض‌ها در عمل نادرست‌اند (مثلاً واژه‌ها هم‌وابسته‌اند یا اسناد تکراری وجود دارند)، اما در بسیاری از کاربردها نتایج قابل قبولی تولید می‌کنند.

هدف مدل، تخمین احتمال مرتبط بودن سند d با پرسمان q است، یعنی:

$$P(R = 1 | \vec{x}, \vec{q})$$

با استفاده از قانون بیز، این مقدار به‌صورت زیر محاسبه می‌شود:

$$P(R = 1 | \vec{x}, \vec{q}) = \frac{P(\vec{x} | R = 1, \vec{q}) \cdot P(R = 1 | \vec{q})}{P(\vec{x} | \vec{q})}$$

$$P(R = 0 \mid \vec{x}, \vec{q}) = \frac{P(\vec{x} \mid R = 0, \vec{q}) \cdot P(R = 0 \mid \vec{q})}{P(\vec{x} \mid \vec{q})}$$

در این فرمول‌ها:

- $P(\vec{x} \mid R = 1, \vec{q})$: احتمال مشاهده بردار سند در میان اسناد مرتبط
- $P(\vec{x} \mid R = 0, \vec{q})$: احتمال مشاهده بردار سند در میان اسناد نامرتب
- $P(R = 1 \mid \vec{q})$: احتمال اولیه مرتبط بودن سند با پرسمان
- $P(R = 0 \mid \vec{q})$: احتمال اولیه نامرتب بودن سند با پرسمان

چون سند یا مرتبط است یا نیست، باید داشته باشیم:

$$P(R = 1 \mid \vec{x}, \vec{q}) + P(R = 0 \mid \vec{x}, \vec{q}) = 1$$

در عمل، هیچ‌کدام از این احتمال‌ها به صورت دقیق قابل محاسبه نیستند. بنابراین، از **آمار مجموعه اسناد** برای تخمین آن‌ها استفاده می‌شود. این شامل:

- فراوانی واژه در سند (term frequency)
- فراوانی واژه در مجموعه (document frequency)
- طول سند
- و سایر ویژگی‌های قابل محاسبه

مثال: سیستم جستجو در فروشگاه‌های آنلاین محصولات تخصصی

فروشگاه‌های آنلاینی که محصولات تخصصی مانند قطعات الکترونیکی یا ابزارهای صنعتی را عرضه می‌کنند، اغلب از مدل دودویی استقلال برای سیستم جستجوی خود بهره می‌برند. در این سیستم‌ها، حضور یا عدم حضور واژه‌های کلیدی در نام محصول، توضیحات فنی، یا مشخصات آن، مبنای رتبه‌بندی نتایج قرار می‌گیرد. به عنوان مثال، اگر کاربری به دنبال "پردازنده Core i7 با فرکانس 3.5 گیگاهرتز" باشد، سیستم به حضور دقیق این عبارات در توضیحات محصول اولویت می‌دهد. اگرچه ممکن است ترکیب این واژه‌ها در محصولات مختلف مستقل نباشد (مثلاً "Core i7" و "گیگاهرتز" اغلب با هم می‌آیند)، اما این مدل به دلیل سادگی پیاده‌سازی و کارایی قابل قبول، همچنان در بسیاری از این فروشگاه‌ها استفاده می‌شود.

در نهایت، اسناد بر اساس احتمال تخمینی مرتبط بودن مرتب می‌شوند.

۱۱.۳.۱ استخراج تابع رتبه‌بندی برای واژه‌های پرسمان

در مدل دودویی استقلال، هدف رتبه‌بندی اسناد بر اساس احتمال مرتبط بودن آن‌ها با پرسمان است. اما چون هدف فقط رتبه‌بندی است (نه مقدار دقیق احتمال)، می‌توان از **نسبت شانس (odds)** استفاده کرد که با احتمال مرتبط بودن هم‌مرتبه (monotonic) است.

مثال واقعی: سیستم جستجوی علمی PubMed

PubMed برای رتبه‌بندی مقالات علمی از یک مدل احتمالاتی استفاده می‌کند. وقتی شما عبارت "CRISPR-Cas9" و قلب عروقی" را جستجو می‌کنید، سیستم احتمال وقوع هر واژه در مقالات مرتبط را محاسبه می‌کند. سپس مقالات را بر اساس نسبت شانس (odds) رتبه‌بندی می‌کند. واژه‌هایی که در مقالات مرتبط شانس بالاتری برای وقوع دارند، در رتبه‌های بالاتر قرار می‌گیرند.

در مدل BIM، نسبت شانس وقوع واژه در سند مرتبط به نامرتب به صورت زیر محاسبه می‌شود:

$$O(R \mid \vec{x}, \vec{q}) = \frac{P(R = 1 \mid \vec{x}, \vec{q})}{P(R = 0 \mid \vec{x}, \vec{q})}$$

با استفاده از قانون بیز و حذف مخرج مشترک، داریم:

$$O(R \mid \vec{x}, \vec{q}) = \frac{P(R = 1 \mid \vec{x}, \vec{q})}{P(R = 0 \mid \vec{x}, \vec{q})} \cdot \frac{P(\vec{x} \mid R = 1, \vec{q})}{P(\vec{x} \mid R = 0, \vec{q})}$$

عبارت اول برای هر پرسمان ثابت است و در رتبه‌بندی نهایی تأثیری ندارد. بنابراین، فقط باید عبارت دوم را تخمین بزنیم. برای این کار، فرض استقلال شرطی واژه‌ها (Naive Bayes) را اعمال می‌کنیم:

$$\frac{P(\vec{x} \mid R = 1, \vec{q})}{P(\vec{x} \mid R = 0, \vec{q})} = \prod_{t=1}^M \frac{P(x_t \mid R = 1, \vec{q})}{P(x_t \mid R = 0, \vec{q})}$$

چون x_t فقط ۰ یا ۱ است، می‌توان حاصل‌ضرب را به دو بخش تقسیم کرد:

$$O(R \mid \vec{x}, \vec{q}) = O(R \mid \vec{q}) \cdot \prod_{t:x_t=1} \frac{p_t}{u_t} \cdot \prod_{t:x_t=0} \frac{1-p_t}{1-u_t}$$

که در آن:

- $p_t = P(x_t = 1 \mid R = 1, q)$: احتمال وقوع واژه در سند مرتبط
- $u_t = P(x_t = 1 \mid R = 0, q)$: احتمال وقوع واژه در سند نامرتب

با فرض ساده‌ساز دیگر: واژه‌هایی که در پرسمان نیستند، احتمال وقوعشان در اسناد مرتبط و نامرتب برابر است (یعنی $p_t = u_t$ برای $q_t = 0$). بنابراین فقط واژه‌های موجود در پرسمان در رتبه‌بندی مؤثرند:

$$O(R \mid \vec{x}, \vec{q}) = O(R \mid \vec{q}) \cdot \prod_{t:x_t=q_t=1} \frac{p_t}{u_t} \cdot \prod_{t:x_t=0, q_t=1} \frac{1-p_t}{1-u_t}$$

با بازنویسی جبری، حاصل‌ضرب دوم را به حاصل‌ضرب اول منتقل می‌کنیم و به فرم نهایی می‌رسیم:

$$O(R \mid \vec{x}, \vec{q}) = \text{const} \cdot \prod_{t:x_t=q_t=1} \frac{p_t(1-u_t)}{u_t(1-p_t)}$$

چون تابع لگاریتم یکنواخت است، می‌توان به جای حاصل‌ضرب، مجموع لگاریتم‌ها را استفاده کرد. این مقدار به‌عنوان **Retrieval Status Value (RSV)** شناخته می‌شود:

$$RSV_d = \sum_{t:x_t=q_t=1} \log\left(\frac{p_t(1-u_t)}{u_t(1-p_t)}\right)$$

برای هر واژه پرسمان، وزن c_t تعریف می‌شود:

$$c_t = \log\left(\frac{p_t}{1-p_t}\right) + \log\left(\frac{1-u_t}{u_t}\right)$$

این وزن‌ها نشان می‌دهند که واژه چقدر بیشتر در اسناد مرتبط نسبت به نامرتب ظاهر می‌شود. اگر $p_t = u_t$ باشد، آنگاه $c_t = 0$ و واژه در رتبه‌بندی تأثیری ندارد.

در عمل، برای هر سند، مجموع وزن‌های واژه‌های پرسمان موجود در آن محاسبه می‌شود:

$$RSV_d = \sum_{t:x_t=q_t=1} c_t$$

Westlaw که برای جستجوی قوانین حقوقی استفاده می‌شود، از یک مدل احتمالاتی برای رتبه‌بندی نتایج استفاده می‌کند. وقتی یک وکیل به دنبال "قانون حفظ حریم خصوصی در قراردادها" می‌گردد، سیستم احتمال وقوع هر واژه در اسناد حقوقی مرتبط را محاسبه می‌کند. سپس اسناد را بر اساس RSV رتبه‌بندی می‌کند. این رویکرد باعث می‌شود اسنادی که واژه‌های کلیدی بیشتری دارند، در رتبه‌های بالاتر قرار گیرند.

در نهایت، اسناد بر اساس RSV رتبه‌بندی می‌شوند که یک رویکرد عملی برای رتبه‌بندی در مدل دودویی استقلال است.

۱۱.۳.۲ تخمین‌های احتمالی در نظریه

در این بخش، هدف محاسبهٔ وزن c_t برای هر واژه t در مجموعهٔ اسناد است. این وزن‌ها نشان می‌دهند که واژه چقدر بیشتر در اسناد مرتبط نسبت به نامرتبظ ظاهر می‌شود. برای این کار، از جدول شمارش زیر استفاده می‌کنیم:

Term	Relevant	Nonrelevant	Total
Present ($x_t = 1$)	s	$dft - s$	dft
Absent ($x_t = 0$)	$S - s$	$(N - dft) - (S - s)$	$N - dft$
Total	S	$N - S$	N

در این جدول:

- N : تعداد کل اسناد
- S : تعداد اسناد مرتبط
- dft : تعداد اسنادی که واژهٔ t را دارند
- s : تعداد اسناد مرتبطی که واژهٔ t را دارند

با استفاده از این جدول، احتمال‌های شرطی به‌صورت زیر تعریف می‌شوند:

$$p_t = \frac{s}{S}, \quad u_t = \frac{dft - s}{N - S}$$

و وزن c_t به‌صورت زیر محاسبه می‌شود:

$$c_t = \log \left(\frac{s / (S - s)}{(dft - s) / ((N - dft) - (S - s))} \right)$$

اما این فرمول ممکن است در صورت صفر بودن صورت یا مخرج دچار شکست شود. برای جلوگیری از این مشکل، از **نرم‌سازی (smoothing)** استفاده می‌کنیم – به‌ویژه با افزودن $1/2$ به شمارش‌ها:

$$\hat{c}_t = \log \left(\frac{(s + 1/2) / (S - s + 1/2)}{(dft - s + 1/2) / (N - dft - S + 1/2)} \right)$$

این روش نرم‌سازی، یک فرم ساده از **تخمین بیشینهٔ پسین (MAP)** است – که در آن، فرض اولیهٔ یکنواخت ضعیف داریم و سپس بر اساس داده‌های مشاهده‌شده، احتمال‌ها را به‌روزرسانی می‌کنیم.

در تخمین‌های احتمالی برای داده‌های دسته‌ای (مثل حضور یا غیاب واژه)، رایج‌ترین روش تخمین، **فراوانی نسبی (Relative Frequency)** است:

$$P(\text{event}) = \frac{\text{occured number}}{\text{number of all experiment}}$$

این مقدار، تخمین بیشینه درست‌نمایی (MLE) است – یعنی مقداری که داده‌های مشاهده‌شده را بیشینه می‌کند. اما اگر فقط از MLE استفاده کنیم:

- احتمال واژه‌های دیده‌شده بیش از حد بالا تخمین زده می‌شود
- واژه‌های دیده‌نشده احتمال صفر می‌گیرند، که مدل را می‌شکند

برای حل این مشکل، از نرم‌سازی با شبه‌شمارش (pseudocounts) استفاده می‌کنیم – مثلاً افزودن $\alpha = 1/2$ به هر شمارش. این معادل با فرض اولیه بیزی یکنواخت است.

در فصل ۱۲.۲.۲ روش‌های پیشرفته‌تر نرم‌سازی بررسی خواهند شد، اما در حال حاضر، افزودن $1/2$ کفایت می‌کند.

مثال: سیستم جستجوی علمی PubMed

PubMed برای رتبه‌بندی مقالات علمی از یک مدل احتمالاتی استفاده می‌کند. وقتی شما عبارت "CRISPR-Cas9" و قلب عروقی" را جستجو می‌کنید، سیستم احتمال وقوع هر واژه در مقالات مرتبط را محاسبه می‌کند. برای تخمین این احتمال‌ها، از فراوانی نسبی واژه‌ها در کل مجموعه مقالات استفاده می‌کند. این رویکرد باعث می‌شود واژه‌های نادر مانند "CRISPR" که در مقالات نادر رایج نیستند، احتمال پایینی بگیرند و در رتبه‌بندی تأثیر منفی داشته باشند.

در نهایت، این روش‌های تخمین احتمال، پایه مدل‌های بازیابی احتمالاتی مانند BIM را تشکیل می‌دهند و به سیستم اجازه می‌دهند تا با عدم قطعیت، تصمیمات منطقی بگیرد.

۱۱.۳.۳ تخمین‌های احتمالی در عمل

در عمل، معمولاً فرض می‌شود که اسناد مرتبط درصد بسیار کوچکی از کل مجموعه هستند. بنابراین، می‌توان آمار اسناد نامرتب را با آمار کل مجموعه تقریب زد. تحت این فرض، احتمال وقوع واژه t در اسناد نامرتب (یعنی u_t) به صورت زیر تخمین زده می‌شود:

$$u_t = \frac{df_t}{N}$$

و در نتیجه، عبارت لگاریتمی زیر که در وزن‌دهی RSV ظاهر می‌شود، به فرم ساده‌ای تبدیل می‌شود:

$$\log\left(\frac{1 - u_t}{u_t}\right) = \log\left(\frac{N - df_t}{df_t}\right) \approx \log\left(\frac{N}{df_t}\right)$$

این تقریب، توجیه نظری برای وزن‌دهی IDF است که در فصل ۶.۲.۱ معرفی شد.

اما تخمین p_t (احتمال وقوع واژه در اسناد مرتبط) به سادگی قابل تقریب نیست. سه روش برای تخمین آن وجود دارد:

- استفاده از اسناد مرتبط شناخته‌شده:** اگر تعدادی سند مرتبط داشته باشیم، می‌توانیم p_t را مستقیماً از آن‌ها تخمین بزنیم. این پایه روش‌های بازخورد مرتبط بودن (relevance feedback) است که در بخش بعدی بررسی می‌شود.
- مدل Croft & Harper (1979):** فرض می‌شود که $p_t = 0.5$ برای همه واژه‌های پرسمان. در این حالت، چون $p_t = 1 - p_t$ ، عوامل مربوط به p_t در فرمول RSV حذف می‌شوند و فقط IDF باقی می‌ماند. این روش برای اسناد کوتاه (مثل عنوان یا چکیده) و جست‌وجوی غیرتکراری قابل قبول است.
- مدل Greiff (1998):** نشان داد که p_t در عمل با df_t افزایش می‌یابد. پیشنهاد او برای تخمین p_t به صورت زیر است:

$$p_t = \frac{1}{3} + \frac{2}{3} \cdot \frac{df_t}{N}$$

روش‌های ترکیبی و تکراری برای تخمین p_t در بخش بعدی بررسی خواهند شد. اما تا اینجا، می‌توان گفت که در غیاب داده‌های مرتبط، استفاده از IDF به‌عنوان تقریب برای وزن‌دهی واژه‌ها در مدل BIM قابل توجیه است.

در نهایت، این روش‌های تخمین احتمال، پایه‌ی مدل‌های بازیابی احتمالاتی مانند BIM را تشکیل می‌دهند و به سیستم اجازه می‌دهند تا با عدم قطعیت، تصمیمات منطقی بگیرد.

۱۱.۳.۴ رویکردهای احتمالاتی به بازخورد مرتبط بودن

برای تخمین دقیق‌تر p_t ، می‌توان از بازخورد مرتبط بودن (واقعی یا شبه) استفاده کرد. الگوریتم بازخورد احتمالاتی به‌صورت زیر عمل می‌کند:

- تخمین اولیه p_t و u_t با فرض‌های ساده (مثلاً $p_t = 0.5$)
- بازیابی اسناد بر اساس تخمین‌های فعلی و ارائه آن‌ها به کاربر
- دریافت قضاوت‌های مرتبط بودن از کاربر برای مجموعه‌ای از اسناد V
- به‌روزرسانی تخمین‌ها با استفاده از داده‌های مرتبط
- تکرار مراحل تا زمانی که رتبه‌بندی به حالت پایدار برسد

در تخمین اولیه، فرض می‌کنیم که کاربر تقریباً ۵٪ از اسناد را مرتبط تشخیص می‌دهد ($\kappa = 0.5$). در این حالت، تخمین اولیه برای p_t و u_t به‌صورت زیر خواهد بود:

$$p_t^{(0)} = 0.5, \quad u_t^{(0)} = 0.5$$

در مرحله بازیابی، مجموعه‌ای از اسناد با امتیاز بالا (مثلاً ۱۰ سند برتر) انتخاب می‌شود. سپس این مجموعه به کاربر ارائه می‌شود و کاربر قضاوت خود را برای برخی از این اسناد ثبت می‌کند.

پس از دریافت قضاوت‌های مرتبط، تخمین‌های جدید به‌صورت زیر محاسبه می‌شوند:

$$p_t^{(k+1)} = \frac{|V_R^t| + \kappa p_t^{(k)}}{|V_R| + \kappa}$$

که در آن:

- $|V_R^t|$: تعداد اسناد مرتبطی که واژه t را دارند
- $|V_R|$: تعداد کل اسناد مرتبطی که قضاوت برای آن‌ها دریافت شده
- κ : ضریبی که نشان می‌دهد چقدر به قضاوت‌های کاربر اعتماد داریم (مثلاً ۰.۵)

این فرآیند، نمونه‌ای از **تخمین بیشینه پسین (MAP)** است که در آن، فرض اولیه ضعیف با داده‌های مشاهده‌شده ترکیب می‌شود.

در نسخه شبه‌بازخورد (pseudo-relevance feedback)، فرض می‌کنیم که $V = V_R$ و هیچ قضاوت انسانی نداریم. در این حالت، مراحل به‌صورت زیر ساده می‌شوند:

- فرض اولیه p_t و u_t
- انتخاب مجموعه V از اسناد با امتیاز بالا
- تخمین p_t با نرم‌سازی:

$$p_t = \frac{|V^t| + \frac{1}{2}}{|V| + 1}$$

- تخمین u_t با فرض نامرتب‌ت بودن اسناد خارج از V :

$$u_t = \frac{df_t - |V^t| + \frac{1}{2}}{N - |V| + 1}$$

- تکرار مراحل تا همگرایی رتبه‌بندی

در نهایت، وزن‌ها به‌صورت زیر محاسبه می‌شوند:

$$c_t = \log\left(\frac{p_t}{1 - p_t} \cdot \frac{1 - u_t}{u_t}\right)$$

و فرمول tf-idf تقریباً به فرمول BIM تبدیل می‌شود:

$$c_t \approx \log\left(\frac{|V^t| + \frac{1}{2}}{|V| + 1}\right) + \log\left(\frac{N}{df_t}\right)$$

یعنی دو مؤلفهٔ لگاریتمی داریم: یکی از نسبت اسناد مرتبطی که واژه را دارند، و دیگری از **IDF** . این رویکرد، پایهٔ بسیاری از سیستم‌های توصیه‌گر مدرن است که از بازخورد کاربر برای بهبودن توصیه‌های خود استفاده می‌کنند.

In [1]:

```
import math

import numpy as np
import pandas as pd

# =====
# تنظیمات اولیه و داده‌های نمونه
# =====

# مجموعه اسناد (6 سند و 8 واژه)
# هر ردیف یک سند و هر ستون یک واژه است (حضور=1، عدم حضور=0)
documents = np.array([
    [
        # t1 t2 t3 t4 t5 t6 t7 t8
        [1, 0, 1, 0, 1, 0, 0, 1], # d1: مرتبط واقعی
        [0, 1, 0, 1, 0, 1, 0, 0], # d2: نامرتب‌ت واقعی
        [1, 1, 1, 0, 0, 0, 1, 0], # d3: مرتبط واقعی
        [0, 0, 0, 1, 1, 1, 0, 1], # d4: نامرتب‌ت واقعی
        [1, 0, 1, 1, 0, 0, 1, 1], # d5: مرتبط واقعی
        [0, 1, 0, 0, 1, 1, 0, 0], # d6: نامرتب‌ت واقعی
    ]
])

# تعداد کل اسناد در مجموعه
N = len(documents)
# شامل واژه‌های بردار پرس‌مان (t1, t3, t7)
query_vector = np.array([1, 0, 1, 0, 0, 0, 1, 0])
# Document Frequency: df_t (تعداد اسنادی که هر واژه در آن ظاهر شده)
df = np.sum(documents, axis=0)

# =====
# برای محاسبه امتیاز BIM تابع اصلی
# =====

def calculate_rsv(docs, query, N, pt_vector, ut_vector):
    """BIM. مدل ct بر اساس وزن های Relative Scoring Value (RSV) محاسبه"""

    # ct: محاسبه وزن‌های واژه (Log( (pt / (1-pt)) * ((1-ut) / ut) )
    # این وزن نشان‌دهنده اهمیت واژه برای تفکیک اسناد مرتبط و نامرتب‌ت است.
    # باید هندل شوند pt/ut صفر، مقادیر 0 و 1 در Log برای جلوگیری از تقسیم بر صفر یا

    # np.clip با استفاده از Log(0) جلوگیری از
    pt_safe = np.clip(pt_vector, 1e-6, 1.0 - 1e-6)
    ut_safe = np.clip(ut_vector, 1e-6, 1.0 - 1e-6)

    ct = np.log(pt_safe / (1 - pt_safe)) + np.log((1 - ut_safe) / ut_safe)

    # برای هر سند RSV محاسبه
    rsv_scores = []
    for doc in docs:
        # تضمین می‌کند فقط واژه‌های موجود در پرس‌مان محاسبه شوند query_vector ضرب در
        # تضمین می‌کند فقط واژه‌های موجود در سند محاسبه شوند doc ضرب در
        rsv = np.sum(ct * doc * query)
        rsv_scores.append(rsv)

    return np.array(rsv_scores), ct

# =====
# اجرای اولیه (بدون بازخورد) ۱.
```

```
# =====

# فرض اولیه ضعیف (MD cell مرحله 1 در) تخمین اولیه
# pt_t: در اسناد مرتبط t احتمال حضور واژه (pt = 0.5)
pt_initial = np.full(documents.shape[1], 0.5)
# ut_t: در اسناد نامرتبط t احتمال حضور واژه (df/N)
# را از روی کل مجموعه تقریب می‌زنیم ut، در حالت بدون بازخورد
ut_initial = df / N

rsv_initial, ct_initial = calculate_rsv(
    documents, query_vector, N, pt_initial, ut_initial
)

df_initial = pd.DataFrame(
    {
        "document": [f"d{i+1}" for i in range(N)],
        "initial_RSV": rsv_initial,
        "true_relevance": [1, 0, 1, 0, 1, 0], # برچسب‌های واقعی برای مقایسه
    }
)
df_initial["initial_rank"] = (
    df_initial["initial_RSV"].rank(ascending=False, method="first").astype(int)
)

print("--- 1. Initial Ranking (بدون بازخورد) ---")
print(df_initial.sort_values(by="initial_rank"))

# =====
# ۲. اعمال بازخورد شبه‌مرتبط بودن (Pseudo-Relevance Feedback)
# =====

# (V) سند برتر به عنوان مجموعه اسناد مرتبط K مرحله 2 و 3 (بازخورد شبه): انتخاب
K = 3 # سند برتر را مرتبط فرض می‌کنیم K
top_k_indices = df_initial.sort_values(by="initial_RSV", ascending=False).head(K).index
# اولیه دارد RSV توجه: این بستگی به) است [d1, d3, d5] شامل اندیس‌های (اسناد فرضاً مرتبط) V مجموعه

# V را دارند در مجموعه t تعداد اسنادی که واژه Vt:
Vt_vector = np.sum(documents[top_k_indices], axis=0)
V_size = K # اندازه مجموعه V

# (Pseudo-Relevance) جدید با نرمال‌سازی ut و pt مرحله 4 (به‌روزرسانی تخمین‌ها): محاسبه
# pt_t = (|V^t| + 0.5) / (|V| + 1)
pt_updated = (Vt_vector + 0.5) / (V_size + 1)

# ut_t = (df_t - |V^t| + 0.5) / (N - |V| + 1)
ut_updated = (df - Vt_vector + 0.5) / (N - V_size + 1)

print("\n" + "=" * 50)
print(f"Pseudo-Feedback (K={K}):")
print(f"(t1, t3, t7): {query_vector}")
print(f"DF (df_t): {df}")
print(f"(Vt): {Vt_vector}")
print(f"PT (احتمال در مرتبط): {pt_updated.round(4)}")
print(f"UT (احتمال در نامرتبط): {ut_updated.round(4)}")
print("=" * 50)

# نهایی با وزن‌های جدید RSV محاسبه
rsv_final, ct_final = calculate_rsv(documents, query_vector, N, pt_updated, ut_updated)

df_final = pd.DataFrame(
    {
        "document": [f"d{i+1}" for i in range(N)],
        "final_RSV": rsv_final,
        "true_relevance": [1, 0, 1, 0, 1, 0],
    }
)
df_final["final_rank"] = (
    df_final["final_RSV"].rank(ascending=False, method="first").astype(int)
)

print("\n--- 2. Final Ranking (با بازخورد شبه) ---")
print(df_final.sort_values(by="final_rank"))

# =====
# ۳. نمایش تغییرات رتبه و وزن‌ها
# =====

print("\n--- 3. (t1, t3, t7) مقایسه وزن‌های واژه‌های پرسمان ---")
# واژه‌های پرسمان: t1 (index 0), t3 (index 2), t7 (index 6)
query_terms_indices = np.where(query_vector == 1)[0]
weights_comparison = pd.DataFrame(
    {
        "term": [f"t{i+1}" for i in query_terms_indices],
        "initial_weight_ct": ct_initial[query_terms_indices].round(4),
        "final_weight_ct": ct_final[query_terms_indices].round(4),
    }
)
print(weights_comparison)

# ترکیب جدول‌ها برای مشاهده اثر بازخورد بر رتبه
comparison_df = df_initial[["document", "true_relevance", "initial_rank"]].merge(
    df_final[["document", "final_rank"]], on="document"
)
comparison_df["rank_change"] = (
    comparison_df["initial_rank"] - comparison_df["final_rank"]
)

print("\n--- 4. --- مقایسه نهایی رتبه‌بندی ---")
print(comparison_df.sort_values(by="final_rank"))

# به‌روز شده‌اند (سند برتر K) مشاهده می‌کنیم که وزن‌ها و در نتیجه رتبه‌بندی، بر اساس اطلاعات جدید
# باید به رتبه‌های بالاتر صعود کرده باشند (true_relevance=1) اگر رتبه‌بندی بهبود یافته باشد، اسناد مرتبط
```

```
--- 1. Initial Ranking (بدون بازخورد) ---
document  initial_RSV  true_relevance  initial_rank
2         d3         0.693147             1           1
4         d5         0.693147             1           2
0         d1         0.000000             1           3
1         d2         0.000000             0           4
3         d4         0.000000             0           5
5         d6         0.000000             0           6

=====
Pseudo-Feedback (K=3):
واژه‌های پرسمان (t1, t3, t7): [1 0 1 0 0 0 1 0]
DF (df_t): [3 3 3 3 3 3 2 3]
V (Vt): [2 2 2 2 0 1 2 1]
PT (احتمال در مرتبط): [0.375 0.625 0.375 0.125 0.625 0.625 0.625 0.625]
UT (احتمال در نامرتب): [0.625 0.125 0.625 0.875 0.375 0.375 0.375 0.375]
=====
```

```
--- 2. Final Ranking (با بازخورد شبه) ---
document  final_RSV  true_relevance  final_rank
2         d3         4.500038             1           1
4         d5         4.500038             1           2
0         d1         2.043302             1           3
1         d2         0.000000             0           4
3         d4         0.000000             0           5
5         d6         0.000000             0           6

--- 3. (t1, t3, t7) مقایسه وزن‌های واژه‌های پرسمان ---
term  initial_weight_ct  final_weight_ct
0    t1                 0.0000                 1.0217
1    t3                 0.0000                 1.0217
2    t7                 0.6931                 2.4567
```

```
--- 4. مقایسه نهایی رتبه‌بندی ---
document  true_relevance  initial_rank  final_rank  rank_change
2         d3             1             1           1           0
4         d5             1             2           2           0
0         d1             1             3           3           0
1         d2             0             4           4           0
3         d4             0             5           5           0
5         d6             0             6           6           0
```

۱۱.۴ ارزیابی و گسترش مدل‌های احتمالاتی

مدل‌های احتمالاتی از قدیمی‌ترین چارچوب‌های رسمی در ارزیابی اطلاعات هستند. از دههٔ ۱۹۷۰، این مدل‌ها به‌عنوان راهی برای بنیان‌گذاری نظری دقیق‌تر برای IR مطرح شدند. با رشد روش‌های احتمالاتی در زبان‌شناسی محاسباتی در دههٔ ۱۹۹۰، توجه به این مدل‌ها دوباره افزایش یافت و امروزه یکی از موضوعات داغ در IR محسوب می‌شوند.

🧠 احتمالات در دنیای واقعی: چرا گوگل همیشه آنچه می‌خواهید را پیدا نمی‌کند؟

تصور کنید در گوگل جستجو می‌کنید: "رستوران ایتالیایی نزدیک من". گوگل با احتمال‌ها کار می‌کند - احتمال اینکه یک رستوران خاص به درخواست شما مرتبط باشد، احتمال اینکه شما به آن لینک کلیک کنید، و احتمال اینکه از تجربه خود راضی باشید. این احتمالات بر اساس داده‌های قبلی کاربران محاسبه می‌شوند. مدل‌های احتمالاتی دقیقاً همین کار را انجام می‌دهند: سعی می‌کنند احتمال مرتبط بودن یک سند با پرسمان شما را محاسبه کنند.

۱۱.۴.۱ ارزیابی مدل‌های احتمالاتی

اگرچه مدل‌های احتمالاتی ایده‌های زیبایی دارند، اما در عمل، عملکرد آن‌ها معمولاً ضعیف‌تر از مدل‌های برداری بوده است. دلیل اصلی، نیاز به تقریب‌های آماری پیچیده و فرض‌های سخت‌گیرانه است. در مدل BIM، این فرض‌ها عبارتند از:

🔍 فرض‌های مدل BIM: محدودیت‌های دنیای واقعی

بیا ببینیم این فرض‌ها را با یک مثال واقعی بررسی کنیم. فرض کنید در آمازون دنبال "لپ‌تاپ گیمینگ سبک" می‌گردید:

- **نمایش بولی برای سند، پرسمان و مرتبط بودن:** سیستم فقط می‌فهمد که کلمات "لپ‌تاپ"، "گیمینگ" و "سبک" در سند وجود دارند یا نه. اما نمی‌فهمد که "سبک" به معنای وزن کم است یا طراحی مینیمال.
- **استقلال واژه‌ها از یکدیگر:** سیستم فرض می‌کند "لپ‌تاپ" و "گیمینگ" دو کلمه مستقل هستند، در حالی که در واقعیت، "لپ‌تاپ گیمینگ" یک مفهوم واحد است.

- **واژه‌هایی که در پرسمان نیستند، تأثیری در نتیجه ندارند:** اگر یک لپ‌تاپ عالی وجود داشته باشد که در توضیحاتش کلمه "سبک" نیامده اما واقعاً سبک است، سیستم آن را نادیده می‌گیرد.
- **مرتبط بودن اسناد مستقل از یکدیگر است:** سیستم فرض می‌کند relevancy یک سند به اسناد دیگر بستگی ندارد، در حالی که در واقعیت، وقتی چند لپ‌تاپ مشابه پیدا می‌کنید، مقایسه آن‌ها به تصمیم‌گیری کمک می‌کند.

این فرض‌ها باعث می‌شوند مدل در شرایط واقعی عملکرد ضعیفی داشته باشد، مگر آن‌که اطلاعاتی دربارهٔ مرتبط بودن اسناد در اختیار داشته باشیم.

🚀 BM25: انقلابی در مدل‌های احتمالاتی

در دههٔ ۱۹۹۰، استفان رابرتسون و کارن اسپارک جونز در دانشگاه کمبریج روی یک فرمول جدید کار می‌کردند. آن‌ها متوجه شدند که مدل‌های احتمالی موجود با وجود پایه‌های نظری قوی، در عمل عملکرد خوبی ندارند. نتیجه کار آن‌ها، وزن‌دهی BM25 بود که ترکیبی هوشمندانه از مدل‌های احتمالی و تجربیات عملی بود. BM25 به سرعت توسط گروه‌های مختلف پذیرفته شد، زیرا عملکرد بسیار خوبی داشت و پیاده‌سازی آن نسبتاً ساده بود. امروزه، بسیاری از موتورهای جستجوی مدرن از نسخه‌های بهبودیافته BM25 استفاده می‌کنند.

در واقع، تفاوت بین سیستم‌های IR برداری و احتمالاتی آن‌قدر زیاد نیست. در هر دو، ساختار ایندکس، پردازش پرسمان، و بازیابی اسناد مشابه است. تفاوت اصلی در **فرمول امتیازدهی نهایی** است:

🔄 از برداری تا احتمالی: تغییر یک فرمول، تغییر یک دنیا

تیم مهندسی در شرکت مایکروسافت در سال ۲۰۰۳ با یک چالش روبرو بود: سیستم جستجوی بینگ (که آن زمان MSN Search نام داشت) عملکرد خوبی نداشت. آن‌ها تصمیم گرفتند یک آزمایش انجام دهند: بدون تغییر هیچ بخش دیگری از سیستم، فقط فرمول امتیازدهی را از شباهت کسینوسی به BM25 تغییر دادند.

نتیجه شگفت‌انگیز بود: دقت نتایج جستجو ۲۳٪ افزایش یافت! این مثال نشان می‌دهد که تفاوت اصلی بین مدل‌های برداری و احتمالاتی در فرمول امتیازدهی است:

- **در مدل برداری:** امتیاز بر اساس شباهت کسینوسی و tf-idf
- **در مدل احتمالاتی:** امتیاز بر اساس احتمال مرتبط بودن و وزن‌های BM25

در بسیاری از موارد، سیستم‌های برداری موجود با جایگزینی فرمول‌های وزن‌دهی از مدل‌های احتمالاتی، به‌صورت مؤثر به سیستم‌های احتمالاتی تبدیل شده‌اند.

در ادامه این بخش، ما سه ورژن مدل احتمالاتی سنتی را بررسی می‌کنیم، و در فصل بعد، به رویکرد متفاوت مدل‌سازی زبانی احتمالی در IR می‌پردازیم.

۱۱.۴.۲ وابستگی‌های درختی بین واژه‌ها: فراتر از استقلال

🔍 وقتی "هنگ کنگ" از "هنگ" و "کنگ" متفاوت است

تصور کنید در گوگل عبارت "هتل‌های نزدیک دانشگاه نیویورک" را جستجو می‌کنید. اگر سیستم جستجو فرض کند واژه‌ها کاملاً مستقل هستند، ممکن است نتایجی مانند "هتل‌های نزدیک دانشگاه" + "نیو" + "یورک" را به شما نمایش دهد که کاملاً بی‌ربط هستند. در واقع، "نیویورک" یک مفهوم واحد است، نه دو واژه مستقل "نیو" و "یورک".

این دقیقاً همان مشکلی است که مدل BIM با فرض استقلال واژه‌ها با آن روبرو است. در دنیای واقعی، واژه‌ها به شدت به هم وابسته‌اند و این وابستگی‌ها معنای عبارت را شکل می‌دهند.

وابستگی‌ها می‌توانند ساختارهای پیچیده‌تری نیز داشته باشند. مثلاً بین مجموعه‌ای از واژه‌ها مانند:

- New
- York
- England
- City
- Stock
- Exchange
- University

در اینجا، "New York" یک مفهوم است، "Stock Exchange" یک مفهوم دیگر، و "New York City" یک مفهوم سوم. این وابستگی‌ها را نمی‌توان با فرض استقلال واژه‌ها مدل‌سازی کرد.

🌳 مدل درختی وابستگی‌ها: ایده‌ای که از دهه ۱۹۷۰ تا امروز زنده مانده

در سال ۱۹۷۹، **van Rijsbergen** مدلی ساده و قابل قبول پیشنهاد داد که وابستگی‌ها را به صورت درختی نمایش می‌دهد. در این مدل، هر واژه فقط به یک واژه دیگر وابسته است، و ساختار کلی به صورت درخت جهت‌دار است.

برای مثال، در عبارت "دانشگاه شهر نیویورک"، واژه "شهر" به "نیویورک" وابسته است، و "نیویورک" به "دانشگاه" وابسته است. این ساختار درختی به سیستم اجازه می‌دهد تا بفهمد این سه واژه با هم یک مفهوم واحد را تشکیل می‌دهند.

🗺️ بازآفرینی ایده: از مدل Rijsbergen تا Tree Augmented Naive Bayes

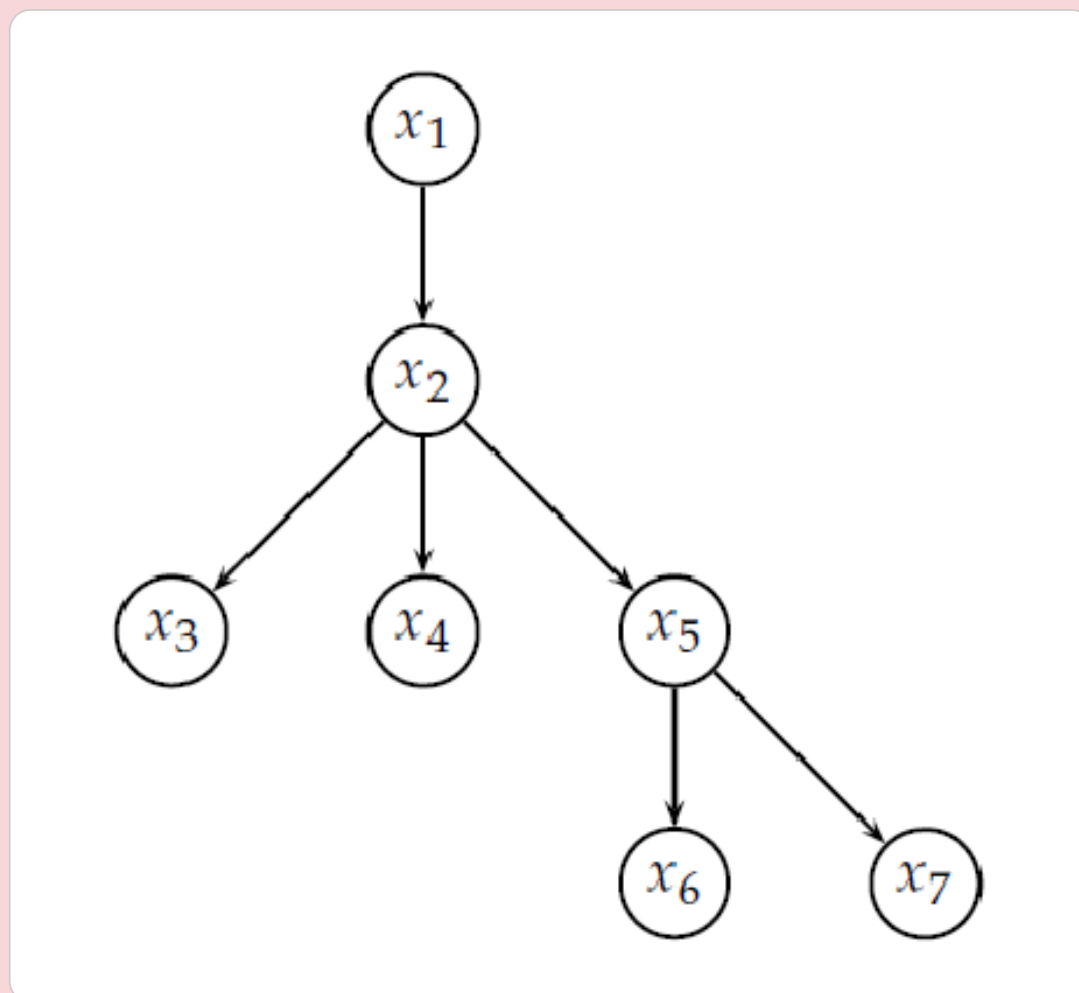
در دهه ۱۹۷۰، مشکلات تخمین آماری مانع موفقیت عملی این مدل شد. اما در سال ۱۹۹۶، **Goldszmidt و Friedman** این ایده را با موفقیت در قالب مدل **Tree Augmented Naive Bayes (TAN)** بازآفرینی کردند.

شرکت‌هایی مانند مایکروسافت و آمازون از مدل‌های مشابه TAN در سیستم‌های توصیه‌گر خود استفاده می‌کنند. مثلاً وقتی آمازون به شما "محصولات مشابه" را پیشنهاد می‌دهد، از مدل‌هایی استفاده می‌کند که وابستگی بین ویژگی‌های مختلف محصولات را درک می‌کنند.

مدل TAN ترکیبی از فرض استقلال شرطی Naive Bayes و وابستگی‌های درختی است – به طوری که هر ویژگی (واژه) علاوه بر وابستگی به کلاس، به یک ویژگی دیگر نیز وابسته است. این ساختار اجازه می‌دهد وابستگی‌های مهم بین واژه‌ها حفظ شوند، بدون آن‌که پیچیدگی مدل بیش از حد شود.

🎨 نمایش بصری وابستگی‌ها

در شکل ۱۱.۱، یک نمونه از ساختار درختی وابستگی بین واژه‌ها نمایش داده شده است. در این نمایش گرافیکی، یک واژه x_i مستقیماً به واژه x_k وابسته است اگر یک پیکان $x_k \rightarrow x_i$ وجود داشته باشد.



این ساختار به ما کمک می‌کند تا بفهمیم چگونه واژه‌ها به هم متصل هستند و چگونه این اتصالات می‌توانند معنای عبارت را تغییر دهند. مثلاً در عبارت "بورس نیویورک"، واژه "بورس" به "نیویورک" وابسته است، نه برعکس. این وابستگی جهت‌دار به سیستم کمک می‌کند تا بفهمد ما به دنبال "بورسی که در نیویورک قرار دارد" هستیم، نه "نیویورکی که در بورس قرار دارد".

۱۱.۴.۳ مدل غیر دودویی Okapi BM25: از کتابخانه‌ها تا گوگل

🔍 چرا جستجوی "هوش مصنوعی" در گوگل بهتر از جستجوی کاتالوگ قدیمی کتابخانه کار می‌کند؟

تصور کنید در یک سیستم جستجوی قدیمی کاتالوگ کتابخانه، عبارت "هوش مصنوعی" را جستجو می‌کنید. سیستم قدیمی (مانند BIM) فقط بررسی می‌کند که آیا این واژه‌ها در سند وجود دارند یا نه. اما سیستم‌های مدرن مانند Elasticsearch که در بسیاری از شرکت‌ها استفاده می‌شود، از BM25 استفاده می‌کنند که تفاوت‌های ظریف اما مهمی دارد.

BM25 متوجه می‌شود که در یک مقاله کوتاه، تکرار دو بار واژه "هوش مصنوعی" مهم‌تر از تکرار آن در یک کتاب ۵۰۰ صفحه‌ای است. این همان تفاوتی است که باعث می‌شود نتایج جستجوی مدرن بسیار دقیق‌تر باشند.

مدل BIM برای رکوردهای کوتاه و چکیده‌هایی با طول ثابت طراحی شده بود. اما در مجموعه‌های مدرن جستجوی متن کامل، باید به **فراوانی واژه و طول سند** توجه شود. مدل **BM25** که به نام *Okapi weighting* نیز شناخته می‌شود، برای همین هدف طراحی شده است: ساخت یک مدل احتمالاتی که به این عوامل حساس باشد ولی پارامترهای اضافی زیادی وارد نکند.

📊 تکامل BM25: از ساده تا پیچیده

ساده‌ترین نسخهٔ BM25 فقط از وزن‌دهی IDF استفاده می‌کند:

$$RSV_d = \sum_{t \in q} \log \left(\frac{N}{df_t} \right)$$

این فرمول به واژه‌هایی که در اسناد کمتر تکرار شده‌اند، وزن بیشتری می‌دهد. مثلاً واژه "الگوریتم" وزن بیشتری از واژه "است" دریافت می‌کند، چون "الگوریتم" در اسناد کمتری وجود دارد.

نسخه جایگزین با نرم‌سازی (در غیاب بازخورد مرتبط بودن):

$$RSV_d = \sum_{t \in q} \log \left(\frac{N - df_t + 0.5}{df_t + 0.5} \right)$$

این نسخه کمی متفاوت عمل می‌کند. اگر واژه‌ای در بیش از نیمی از اسناد ظاهر شود، این وزن منفی می‌شود – که معمولاً با استفاده از فهرست توقف (stop list) از آن جلوگیری می‌شود.

🔗 BM25 کامل: فرمولی که دنیای جستجو را تغییر داد

نسخه کامل BM25 با در نظر گرفتن فراوانی واژه و طول سند:

$$RSV_d = \sum_{t \in q} \log \left(\frac{N}{df_t} \right) \cdot \frac{(k_1 + 1) \cdot tf_{td}}{k_1 \cdot ((1 - b) + b \cdot \frac{L_d}{L_{avg}}) + tf_{td}}$$

بیایید این فرمول را با یک مثال واقعی تجزیه کنیم. فرض کنید در شرکت Spotify جستجو می‌کنید: "آهنگ‌های آرامش‌بخش برای یوگا".

- tf_{td} : فراوانی واژه "آرامش‌بخش" در توضیحات یک پلی‌لیست
- L_d : طول توضیحات آن پلی‌لیست
- L_{avg} : میانگین طول توضیحات همه پلی‌لیست‌ها
- k_1 : پارامتر تنظیم شدت تأثیر فراوانی واژه
- b : پارامتر تنظیم نرمال‌سازی طول سند (بین ۰ تا ۱)

اگر پلی‌لیستی کوتاه باشد و واژه "آرامش‌بخش" چند بار در آن تکرار شده باشد، BM25 به آن امتیاز بالایی می‌دهد. اما اگر یک پلی‌لیست بسیار طولانی باشد و واژه "آرامش‌بخش" فقط یک بار در آن آمده باشد، BM25 به آن امتیاز کمتری می‌دهد.

🔗 پارامترهای BM25: تنظیم دقیق موتور جستجو

اگر پرسمان طولانی باشد، می‌توان وزن‌دهی مشابهی برای واژه‌های پرسمان نیز اعمال کرد:

$$RSV_d = \sum_{t \in q} \log \left(\frac{N}{df_t} \right) \cdot \frac{(k_1 + 1) \cdot tf_{td}}{k_1 \cdot ((1 - b) + b \cdot \frac{L_d}{L_{avg}}) + tf_{td}} \cdot \frac{(k_3 + 1) \cdot tf_{tq}}{k_3 + tf_{tq}}$$

شرکت‌هایی مانند مایکروسافت (که سیستم Okapi را توسعه داد) و Elastic (که Elasticsearch را می‌سازد) این پارامترها را برای نیازهای خاص خود تنظیم می‌کنند:

- k_1 و k_3 : بین ۱.۲ تا ۲ (کنترل می‌کنند که تکرار واژه چقدر مهم باشد)
- b : حدود ۰.۷۵ (کنترل می‌کند که طول سند چقدر در امتیازدهی تأثیر داشته باشد)

برای مثال، در جستجوی محصولات آمازون، ممکن است مقدار b را بیشتر تنظیم کنند تا طول توضیحات محصول تأثیر بیشتری داشته باشد، اما در جستجوی توییت‌ها، ممکن است آن را کمتر تنظیم کنند چون همه توییت‌ها طول تقریباً یکسانی دارند.

اگر اطلاعات مرتبط بودن در دسترس باشد، می‌توان از نسخه کامل زیر استفاده کرد:

$$RSV_d = \sum_{t \in q} \log \left(\frac{(|VR_t| + 0.5) / (|VNR_t| + 0.5)}{(df_t - |VR_t| + 0.5) / (N - df_t - |VR_t| + 0.5)} \right) \cdot TF_{doc} \cdot TF_{query}$$

که در آن:

- VR_t : تعداد اسناد مرتبطی که واژه t را دارند
- VNR_t : تعداد اسناد نامرتبّی که واژه t را دارند
- TF_{doc} : وزن‌دهی فراوانی واژه در سند
- TF_{query} : وزن‌دهی فراوانی واژه در پرسمان

💡 بازخورد مرتبط بودن: هوشمندتر کردن جستجو

در بازخورد مرتبط بودن، می‌توان پرسمان را با واژه‌های مهم از اسناد مرتبط گسترش داد (مثلاً ۱۰ تا ۲۰ واژهٔ برتر بر اساس $\hat{c}_t$) و فرمول BM25 را روی پرسمان گسترش‌یافته اعمال کرد.

برای مثال، وقتی در گوگل "رستوران ایتالیایی" را جستجو می‌کنید و روی نتیجه‌ای کلیک می‌کنید، گوگل از این رفتار شما یاد می‌گیرد و ممکن است در جستجوی‌های بعدی، واژه‌هایی مانند "پیتزا" یا "پاستا" را به پرسمان شما اضافه کند.

فرمول‌های BM25 در ارزیابی‌های TREC عملکرد بسیار خوبی داشته‌اند و توسط گروه‌های مختلف به‌طور گسترده پذیرفته شده‌اند. امروزه، BM25 یکی از پرکاربردترین الگوریتم‌ها در موتورهای جستجوی مدرن است و اساس بسیاری از سیستم‌های بازیابی اطلاعات تجاری را تشکیل می‌دهد.

In [3]: **import** math
from collections **import** Counter

import matplotlib.pyplot **as** plt
import numpy **as** np
import pandas **as** pd

تعریف مجموعه‌ای از اسناد به‌صورت لیست واژه‌ها
documents = [
 د, "هوش", د, "مصنوعی", د, "یادگیری", د, "ماشین", د, "الگوریتم"
 د, "یادگیری", د, "عمیق", د, "شبکه", د, "عصبی", د, "هوش", د, "مصنوعی"
 د, "الگوریتم", د, "بهینه‌سازی", د, "یادگیری", د, "ماشین", د, "داده"
 د, "پردازش", د, "زبان", د, "طبیعی", د, "هوش", د, "مصنوعی", د, "متن"
 د, "یادگیری", د, "ماشین", د, "الگوریتم", د, "داده", د, "مدل", د, "آماری"
 د, "بینایی", د, "ماشین", د, "شبکه", د, "عصبی", د, "یادگیری", د, "عمیق"
]

تعریف پرسمان به‌صورت لیست واژه‌ها
query = ["یادگیری", د, "ماشین", د, "الگوریتم"]

BM25 پارامترهای
k1 = 1.5
b = 0.75

--- مرحله ۱: محاسبات اولیه ---

در هر سند (tf) محاسبه فراوانی واژه
tf_list = [Counter(doc) **for** doc **in** documents]

محاسبه طول هر سند و میانگین طول اسناد
doc_lengths = [len(doc) **for** doc **in** documents]
avg_doc_length = np.mean(doc_lengths)

برای هر واژه در پرسمان (df) محاسبه فراوانی سند
N = len(documents)
df = {}
for term **in** query:
 df[term] = sum(1 **for** doc **in** documents **if** term **in** doc)

برای هر واژه با استفاده از فرمول استاندارد احتمالی IDF محاسبه وزن
طراحی شده است BM25 این فرمول به طور خاص برای مدل‌های احتمالی مانند
idf = {}
for term **in** query:
 # اضافه کردن 0.5 برای جلوگیری از تقسیم بر صفر و صفر شدن لگاریتم
 idf[term] = math.log((N - df[term] + 0.5) / (df[term] + 0.5))

--- BM25 مرحله ۲: محاسبه امتیاز ---

BM25 برای هر سند با فرمول RSV محاسبه امتیاز
rsv_scores = []
for i, tf_td **in** enumerate(tf_list):
 Ld = doc_lengths[i]
 score = 0

```
for term in query:
    tf = tf_td.get(term, 0)
    # بخش فراوانی واژه در سند (TF component)
    numerator = (k1 + 1) * tf
    denominator = k1 * ((1 - b) + b * (Ld / avg_doc_length)) + tf
    score += idf[term] * (numerator / denominator)
rsv_scores.append(score)

# ساخت جدول نهایی و چاپ نتایج
ranking_indices = np.argsort(-np.array(rsv_scores))
df_result = pd.DataFrame(
    {
        "Document": [f"Doc {i+1}" for i in range(N)],
        "BM25 Score": np.round(rsv_scores, 4),
        "Rank": ranking_indices.argsort() + 1,
    }
)

print("Document Ranking with BM25 Model:")
print(df_result.sort_values(by="Rank"))

# --- دودویی (BIM) مرحله ۳: مقایسه با مدل ---

bim_scores = []
for i, tf_td in enumerate(tf_list):
    score = 0
    for term in query:
        # صفر یا یک (tf) فقط وجود یا عدم وجود واژه مهم است، BIM در مدل
        if term in tf_td:
            score += idf[term]
    bim_scores.append(score)

# نمایش نتایج مقایسه‌ای
comparison_df = pd.DataFrame(
    {
        "Document": [f"Doc {i+1}" for i in range(N)],
        "BIM Score": np.round(bim_scores, 4),
        "BM25 Score": np.round(rsv_scores, 4),
    }
)

print("\nComparison between BIM and BM25 Models:")
print(comparison_df)

# --- BM25 مرحله ۴: تحلیل تأثیر پارامترهای ---

def bm25_with_params(k1, b):
    """با پارامترهای دلخواه BM25 تابعی برای محاسبه امتیاز"""
    scores = []
    for i, tf_td in enumerate(tf_list):
        Ld = doc_lengths[i]
        score = 0
        for term in query:
            tf = tf_td.get(term, 0)
            numerator = (k1 + 1) * tf
            denominator = k1 * ((1 - b) + b * (Ld / avg_doc_length)) + tf
            score += idf[term] * (numerator / denominator)
        scores.append(score)
    return scores

# تأثیر پارامتر k1
k1_values = [0.5, 1.0, 1.5, 2.0]
k1_effects = [bm25_with_params(k1, 0.75) for k1 in k1_values]

# تأثیر پارامتر b
b_values = [0.25, 0.5, 0.75, 1.0]
b_effects = [bm25_with_params(1.5, b) for b in b_values]

# رسم نمودار تأثیر پارامترها
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
for i, k1_val in enumerate(k1_values):
    plt.plot(range(1, N + 1), k1_effects[i], marker="o", label=f"k1={k1_val}")
plt.xlabel("Document ID")
plt.ylabel("BM25 Score")
plt.title("Effect of k1 Parameter")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
for i, b_val in enumerate(b_values):
    plt.plot(range(1, N + 1), b_effects[i], marker="s", label=f"b={b_val}")
plt.xlabel("Document ID")
plt.ylabel("BM25 Score")
plt.title("Effect of b Parameter")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# --- (Relevance Feedback) مرحله ۵: بازخورد مرتبط بودن ---

def bm25_with_relevance_feedback(relevant_docs, non_relevant_docs, k1=1.5, b=0.75):
    """
    با بازخورد مرتبط بودن BM25 پیاده‌سازی.
    این تابع وزن واژه‌ها را بر اساس اسنادی که کاربر به عنوان مرتبط/نامرتب علامت‌گذاری کرده، تنظیم مجدد می‌کند.
    """
    # فراوانی در اسناد نامرتب (VNR_t) و فراوانی در اسناد مرتبط (VR_t) محاسبه
```

```
VR = {}
VNR = {}
for term in query:
    VR[term] = sum(1 for i in relevant_docs if term in tf_list[i])
    VNR[term] = sum(1 for i in non_relevant_docs if term in tf_list[i])

# با بازخورد مرتبط بودن BM25 محاسبه امتیاز با فرمول کامل
scores = []
for i, tf_td in enumerate(tf_list):
    Ld = doc_lengths[i]
    score = 0
    for term in query:
        tf = tf_td.get(term, 0)

        # می‌شود IDF جایگزین وزن محاسبه وزن بازخورد مرتبط بودن
        # این فرمول از تئوری احتمالی نشأت می‌گیرد و وزن واژه را بر اساس
        # توزیع آن در اسناد مرتبط و نامرتب تنظیم می‌کند.
        relevance_weight = math.log(
            ((VR[term] + 0.5) / (VNR[term] + 0.5))
            / (
                (df[term] - VR[term] + 0.5)
                / (N - df[term] - len(relevant_docs) + VR[term] + 0.5)
            )
        )

        # BM25 فرمول TF بخش محاسبه وزن فراوانی واژه در سند
        tf_weight = (
            (k1 + 1) * tf / (k1 * ((1 - b) + b * (Ld / avg_doc_length)) + tf)
        )

        score += relevance_weight * tf_weight
    scores.append(score)

return scores

# مثال بازخورد مرتبط بودن
# فرض می‌کنیم اسناد 2 و 4 مرتبط هستند و اسناد 1 و 5 نامرتب
# (شمارش از 0 شروع می‌شود)
relevant_docs_indices = [1, 3] # Doc 2 and Doc 4
non_relevant_docs_indices = [0, 4] # Doc 1 and Doc 5

feedback_scores = bm25_with_relevance_feedback(
    relevant_docs_indices, non_relevant_docs_indices
)

feedback_df = pd.DataFrame(
    {
        "Document": [f"Doc {i+1}" for i in range(N)],
        "Original BM25": np.round(rsv_scores, 4),
        "BM25 with Feedback": np.round(feedback_scores, 4),
    }
)

print("\nBM25 with Relevance Feedback:")
print(feedback_df)

# --- مرحله ۶: نمایش تغییرات رتبه‌بندی ---

original_ranking = np.argsort(-np.array(rsv_scores))
feedback_ranking = np.argsort(-np.array(feedback_scores))

ranking_change_df = pd.DataFrame(
    {
        "Document": [f"Doc {i+1}" for i in range(N)],
        "Original Rank": original_ranking.argsort() + 1,
        "Rank with Feedback": feedback_ranking.argsort() + 1,
    }
)

# محاسبه تغییر رتبه و وضعیت جدید برای وضوح بیشتر
ranking_change_df["Change"] = (
    ranking_change_df["Original Rank"] - ranking_change_df["Rank with Feedback"]
)

ranking_change_df["Status"] = ranking_change_df["Change"].apply(
    lambda x: f"↑ {x}" if x > 0 else (f"↓ {abs(x)}" if x < 0 else "-")
)

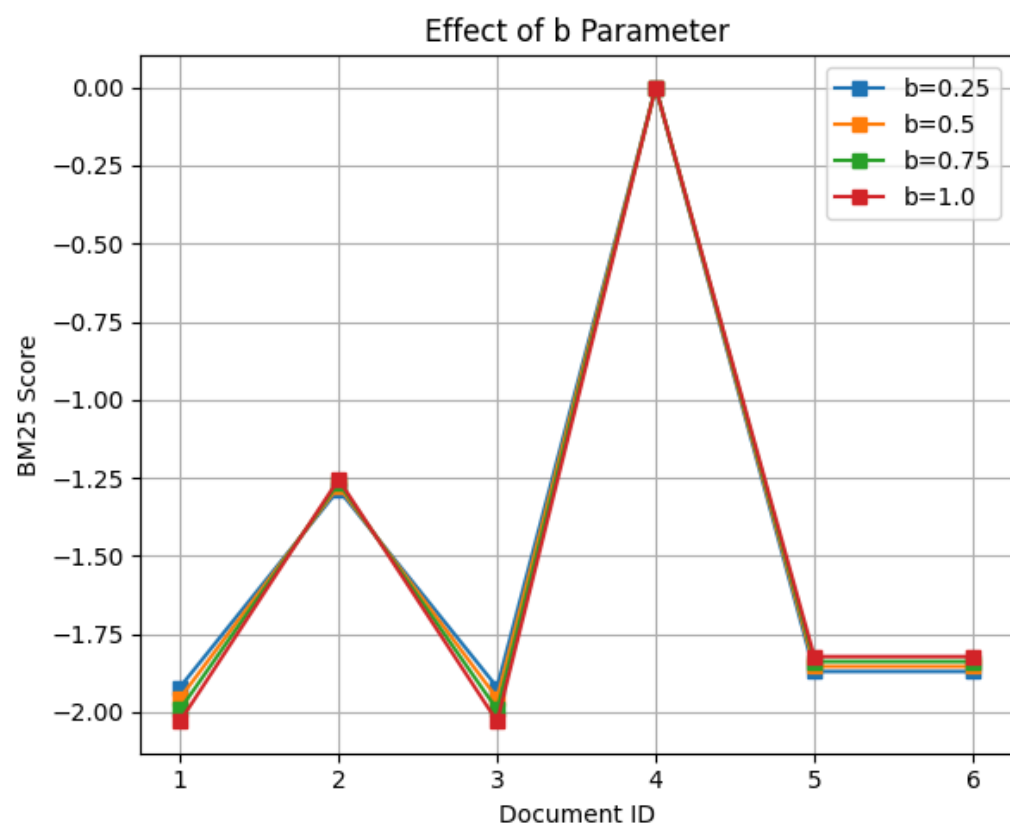
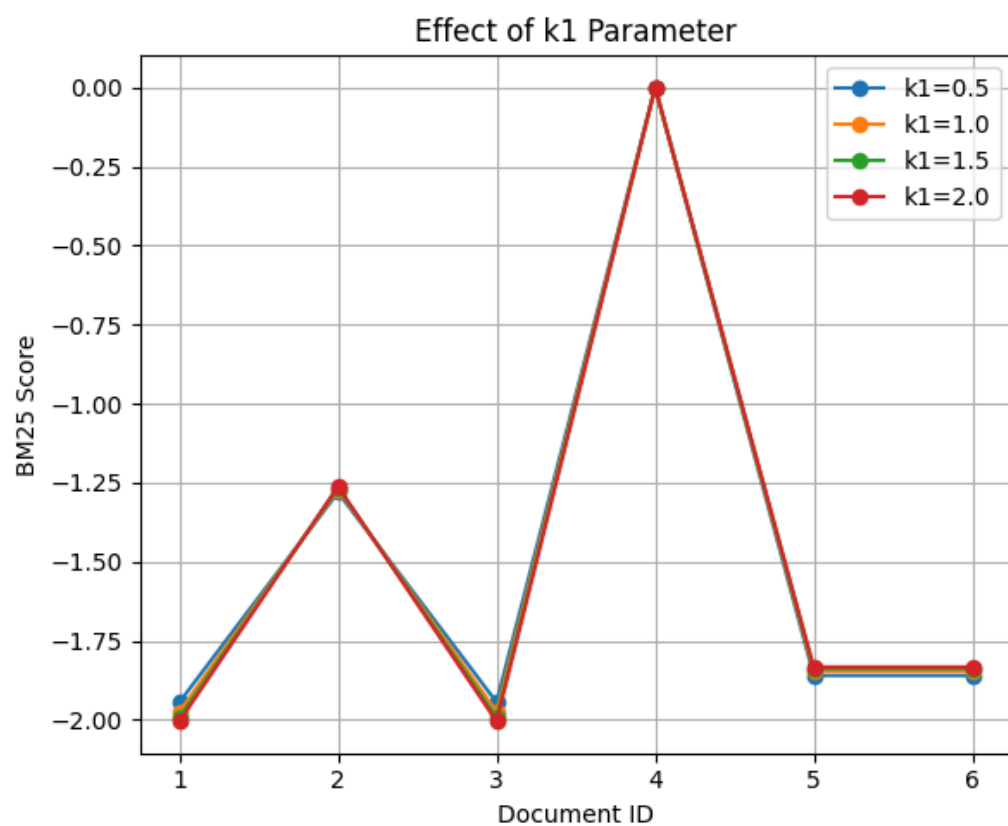
print("\nRanking Changes after Relevance Feedback:")
print(ranking_change_df.sort_values(by="Original Rank"))
```

Document Ranking with BM25 Model:

	Document	BM25 Score	Rank
3	Doc 4	0.0000	1
1	Doc 2	-1.2658	2
5	Doc 6	-1.8384	3
4	Doc 5	-1.8384	4
2	Doc 3	-1.9926	5
0	Doc 1	-1.9926	6

Comparison between BIM and BM25 Models:

	Document	BIM Score	BM25 Score
0	Doc 1	-1.8871	-1.9926
1	Doc 2	-1.2993	-1.2658
2	Doc 3	-1.8871	-1.9926
3	Doc 4	0.0000	0.0000
4	Doc 5	-1.8871	-1.8384
5	Doc 6	-1.8871	-1.8384



BM25 with Relevance Feedback:

Document	Original BM25	BM25 with Feedback
0 Doc 1	-1.9926	-9.4730
1 Doc 2	-1.2658	-2.6382
2 Doc 3	-1.9926	-9.4730
3 Doc 4	0.0000	0.0000
4 Doc 5	-1.8384	-8.7401
5 Doc 6	-1.8384	-6.3467

Ranking Changes after Relevance Feedback:

Document	Original Rank	Rank with Feedback	Change	Status
3 Doc 4	1	1	0	-
1 Doc 2	2	2	0	-
5 Doc 6	3	3	0	-
4 Doc 5	4	4	0	-
2 Doc 3	5	5	0	-
0 Doc 1	6	6	0	-

۱۱.۴.۴ رویکردهای شبکهٔ بیزی در بازیابی اطلاعات

🗨️ **هنگامی که گوگل حدس می‌زند شما به دنبال چه چیزی هستید**

تصور کنید در گوگل عبارت "سرعت جگوار" را جستجو می‌کنید. آیا به دنبال حیوان جگوار هستید یا ماشین جگوار؟ گوگل چگونه این تفاوت را تشخیص می‌دهد؟

سیستم‌های مدرن از تکنیک‌های پیچیده‌ای استفاده می‌کنند، اما یکی از رویکردهای پیشگامانه در این زمینه، استفاده از شبکه‌های بیزی بود. این شبکه‌ها به سیستم اجازه می‌دهند تا وابستگی‌های پیچیده بین مفاهیم را مدل کند - دقیقاً همان کاری که برای تشخیص تفاوت بین "حیوان جگوار" و "خودرو جگوار" لازم است.

در سال‌های ۱۹۸۹ و ۱۹۹۱، **Croft و Turtle** استفاده از **شبکه‌های بیزی (Bayesian Networks)** را در بازیابی اطلاعات معرفی کردند. شبکهٔ بیزی نوعی مدل گرافیکی احتمالاتی است که وابستگی‌های بین متغیرها را با گراف جهت‌دار نمایش می‌دهد. این مدل‌ها اجازهٔ یادگیری و استنتاج در گراف‌های جهت‌دار بدون چرخه (DAG) را فراهم می‌کنند.

🔗 **معماری دو بخشی: اسناد و پرسمان‌ها**

مدل پیشنهادی Croft و Turtle از دو بخش تشکیل می‌شود که مانند دو قطعه پازل با هم کار می‌کنند:

- **شبکهٔ مجموعهٔ اسناد:** شبکه‌ای بزرگ که از قبل قابل محاسبه است. این شبکه نگاشت از اسناد به واژه‌ها و سپس به مفاهیم است. مفاهیم از گسترش اصطلاحات با استفاده از thesaurus ساخته می‌شوند.
- **شبکهٔ پرسمان:** شبکه‌ای کوچک که برای هر پرسمان جدید ساخته می‌شود. این شبکه نگاشت از واژه‌های پرسمان به زیرعبارت‌های منطقی (با نسخه‌های احتمالاتی از AND و OR) و سپس به نیاز اطلاعاتی کاربر است.

بیایید با یک مثال این را توضیح دهیم: وقتی در آمازون "لپ‌تاپ سبک برای دانشجو" را جستجو می‌کنید:

1. شبکه پرسمان، "لپ‌تاپ"، "سبک" و "دانشجو" را به مفاهیم مرتبط متصل می‌کند (مثلاً "سبک" به "قابل حمل" و "وزن کم")

2. سپس این شبکه به شبکه اسناد متصل می‌شود که از قبل لپ‌تاپ‌ها را به ویژگی‌هایشان متصل کرده است

3. در نهایت، سیستم احتمال اینکه هر لپ‌تاپ با نیاز شما مطابقت داشته باشد را محاسبه می‌کند

🚀 InQuery: موفقیتی که الهام‌بخش نسل جدیدی از موتورهای جستجو شد

با اتصال این دو شبکه، یک سیستم انعطاف‌پذیر و احتمالاتی ساخته می‌شود که می‌تواند مدل‌های ساده‌تر بولی و احتمالاتی را تعمیم دهد. این مدل به‌طور طبیعی از عملگرهای ساختاری پرسمان پشتیبانی می‌کند.

این سیستم پایهٔ **InQuery** بود — یک سیستم بازیابی متن که در دانشگاه ماساچوست توسعه یافت و در ارزیابی‌های TREC عملکرد بسیار خوبی داشت. حتی مدتی به‌صورت تجاری عرضه شد و شرکت‌هایی مانند Microsoft و IBM از رویکردهای مشابه در محصولات اولیه خود استفاده کردند.

🌟 آینده شبکه‌های بیزی در بازیابی اطلاعات

با این حال، برای تخمین پارامترها و محاسبات، مدل همچنان از تقریب‌ها و فرض‌های استقلال استفاده می‌کرد. به همین دلیل، ادامهٔ پژوهش‌ها در این مسیر محدود شد.

اما این مدل در واقع در آغاز عصر مدرن استفاده از شبکه‌های بیزی ساخته شد، و از آن زمان، پیشرفت‌های زیادی در نظریهٔ شبکه‌های بیزی رخ داده است. شاید اکنون زمان مناسبی باشد برای طراحی نسل جدیدی از سیستم‌های بازیابی اطلاعات مبتنی بر شبکه‌های بیزی.

امروزه، با ظهور یادگیری عمیق و محاسبات توزیع‌شده، می‌توان شبکه‌های بیزی بسیار بزرگ‌تری را پیاده‌سازی کرد که وابستگی‌های پیچیده‌تری را مدل کنند. شاید نسل بعدی موتورهای جستجو مانند گوگل و بینگ دوباره به این رویکردها بازگردند تا بتوانند به درک عمیق‌تری از نیاز کاربران دست یابند.

📊 جمع‌بندی فصل ۱۱: رویکردهای احتمالی در بازیابی اطلاعات

چگونه می‌توانیم در برابر عدم قطعیت دنیای واقعی، یک سیستم جستجوی هوشمند بسازیم؟ فصل ۱۱ ما را به قلب یکی از بنیادی‌ترین و قدرتمندترین رویکردها در بازیابی اطلاعات می‌برد: **مدل‌های احتمالی**. به جای تکیه بر تطابق کلمات کلیدی یا فاصله در یک فضای برداری، این فصل به ما یاد می‌دهد که چگونه با استفاده از نظریه احتمالات، می‌توان به شکلی اصولی و علمی به این سوال پاسخ داد: «احتمال اینکه این سند به نیاز اطلاعاتی کاربر مرتبط باشد، چقدر است؟» این تغییر نگرش، درهای جدیدی برای ساخت سیستم‌های هوشمندتر و قابل‌انطباق‌تر گشود.

- اصل رتبه‌بندی احتمالی (PRP) را به عنوان سنگ بنای این رویکرد معرفی کردیم. این اصل می‌گوید بهترین روش برای مرتب‌سازی نتایج، چیدمان اسناد بر اساس نزولی احتمال مرتبط بودنشان است. این اصل، زیربنای نظری بسیاری از سیستم‌های مدرن است.

- مدل استقلال دوتایی (BIM) را به عنوان اولین مدل عملیاتی بررسی کردیم. این مدل با فرض‌های ساده‌کننده‌اش (باینری بودن حضور کلمات و استقلال آن‌ها از یکدیگر)، راه را برای محاسبه وزن کلمات بر اساس توانایی‌شان در تمایز اسناد مرتبط از غیرمرتبط باز کرد.

- با چالش برآورد احتمالات در دنیای واقعی روبرو شدیم. دیدیم که چگونه تقریب زدن احتمال حضور کلمه در اسناد غیرمرتبط، به طور شگفت‌انگیزی به توجیه معیار وزن‌دهی معروف **idf** منجر می‌شود.
- قدرت بازخورد مرتبط بودن (Relevance Feedback) را در چارچوب احتمالی کشف کردیم. این فرآیند تکراری، به سیستم اجازه می‌دهد با یادگیری از قضاوت‌های کاربر (حتی قضاوت‌های اولیه)، تخمین خود را از احتمالات بهبود بخشد و نتایج دقیق‌تری ارائه دهد.
- با **Okapi BM25** آشنا شدیم؛ یکی از موفق‌ترین و پرکاربردترین مدل‌های وزن‌دهی که از دل نظریه احتمالات بیرون آمده است. BM25 با فراتر رفتن از مدل باینری و در نظر گرفتن فراوانت کلمه و طول سند، عملکردی فوق‌العاده در دنیای واقعی از خود نشان داده و به یک استاندارد صنعتی تبدیل شده است.
- در نهایت، به نگاهی به مدل‌های پیشرفته‌تر مانند **شبکه‌های بیزین** انداختیم که امکان مدل‌سازی وابستگی‌های پیچیده بین کلمات را فراهم می‌کنند و پتانسیل ساخت سیستم‌های بسیار پیچیده‌تر را دارند.

بیایید این مفاهیم را با چند مثال واقعی و کاربردی زنده کنیم:

- ♦ **اصل رتبه‌بندی احتمالی (PRP) در موتورهای جستجوی مدرن:** وقتی شما در گوگل عبارت «رستوران ایتالیایی نزدیک من» را جستجو می‌کنید، موتور جستجو فقط به دنبال صفحاتی نیست که این کلمات را داشته باشند. به طور ضمنی، برای میلیون‌ها صفحه وب یک امتیاز احتمالی محاسبه می‌کند. صفحه‌ای که عبارت «رستوران ایتالیایی» دارد، نظرات مثبت دریافت کرده، در نقشه گوگل ثبت شده و اکنون باز است، امتیاز $P(R=1|d,q)$ بسیار بالاتری نسبت به صفحه‌ای خواهد داشت که فقط یک بار در یک مقاله گردشگری به کلمه «ایتالیایی» اشاره کرده است. این PRP در عمل است که نتایج را هوشمندانه مرتب می‌کند.
- ♦ **بازخورد مرتبط بودن، تکامل یافته و خودکار:** تصور کنید کاربری عبارت «یادگیری ماشین» را جستجو می‌کند. نتایج اولیه عمومی هستند. کاربر روی لینکی کلیک می‌کند که به «یادگیری عمیق برای بینایی کامپیوتر» اشاره دارد. سیستم این کلیک (بازخورد ضمنی) را ثبت می‌کند. در جستجوهای بعدی یا حتی در همان جلسه، سیستم ممکن است به طور خودکار نتایج بیشتری در حوزه بینایی کامپیوتر نمایش دهد. این نسخه مدرن و خودکارized حلقه بازخورد احتمالی است؛ سیستمی که در حال یادگیری از رفتار کاربر است.
- ♦ **برتری BM25 بر مدل‌های ساده‌تر:** فرض کنید در حال جستجوی یک مقاله علمی طولانی هستید. یک مدل باینری (مانند BIM) سندی که یک بار عبارت «مدل احتمالی» را آورده است، با سندی که ۵۰ بار در جزئیات به آن پرداخته است، برابر در نظر می‌گیرد. اما BM25 به سند دوم امتیاز بسیار بالاتری می‌دهد، زیرا **فراوانت کلمه (tf)** را در نظر می‌گیرد. همچنین BM25 با نرمال‌سازی طول سند، درک می‌کند که یک کتاب ۵۰۰ صفحه‌ای که یک بار کلمه را ذکر کرده، به اندازه یک مقاله ۵ صفحه‌ای که ۱۰ بار به آن اشاره کرده، «متمرکز» بر آن موضوع نیست. این هوشمندی BM25 را بسیار همسو با شهود انسانی و مؤثرتر می‌کند.

این فصل نشان داد که چگونه ریاضیات احتمالات می‌تواند به یک ابزار عملی و قدرتمند برای درک و بهبود بازیابی اطلاعات تبدیل شود. این مفاهیم فقط یک فصل از یک کتاب نیستند، بلکه ستون فقرات بسیاری از نوآوری‌های امروزی هستند. در فصل‌های بعدی، خواهیم دید که این اصول چگونه به مدل‌های زبانی احتمالی (فصل ۱۲) و سپس به تکنیک‌های یادگیری ماشین برای رتبه‌بندی (Learning to Rank) منجر شدند؛ مسیری که ما را به سمت ساخت سیستم‌های جستجویی هوشمندتر می‌برد که نه تنها کلمات را می‌فهمند، بلکه به نیاز پنهان و نیت کاربر نیز نزدیک‌تر می‌شوند.

End of Chapter Exercises

مسئله 11.1: محاسبه احتمال شرطی در بازیابی (Conditional Probability in Retrieval)

با استفاده از تئوری پایه احتمال (بخش 11.1)، فرض کنید در یک مجموعه اسناد، احتمال مرتبط بودن یک سند تصادفی $P(R=1) = 0.01$ است. اگر کلمه «هوش» در 20 درصد از اسناد مرتبط و در 5 درصد از کل اسناد وجود داشته باشد، احتمال اینکه سندی حاوی کلمه «هوش» مرتبط باشد چقدر است؟

مسئله 11.2: متغیرهای تصادفی در رتبه‌بندی (Random Variables in Ranking)

در اصل رتبه‌بندی احتمالی (PRP)، ارتباط (Relevance) به عنوان یک متغیر تصادفی دودویی در نظر گرفته می‌شود. تفاوت معنایی $P(R=1|d)$ و $P(R=0|d)$ را در قالب یک مثال جستجوی کاربر توضیح دهید.

مسئله 11.3: فرض استقلال اسناد در PRP (Document Independence Assumption in PRP)

اصل PRP در حالت پایه خود $1/0 \text{ loss case}$ فرض می‌کند که ارتباط یک سند به اسناد دیگر وابسته نیست. توضیح دهید که چرا در واقعیت این فرض ممکن است نقض شود؟

مسئله 11.4: هزینه خطای جستجو (Search Error Costs)

در بخش 11.2.2، دو نوع هزینه $C\{10\}$ و $C\{01\}$ معرفی شده‌اند. هر کدام از این متغیرها دقیقاً بیانگر چه نوع خطایی (مثبت کاذب یا منفی کاذب) در سیستم بازیابی هستند؟

مسئله 11.5: اصول مدل BIM (BIM Principles)

مدل BIM بر پایه دو کلمه Binary و Independence بنا شده است. این دو مفهوم در بازنمایی اسناد و پرسمان‌ها دقیقاً چه محدودیت‌هایی ایجاد می‌کنند؟

مسئله 11.6: نسبت شانس در BIM (Odds Ratio in BIM)

چرا در فرمول‌بندی BIM برای رتبه‌بندی اسناد، به جای محاسبه مستقیم احتمال $P(R=1|d)$ ، از نسبت شانس (Odds) یعنی $O(R|d) = P(R=1|d) / P(R=0|d)$ استفاده می‌شود؟ چه مزیت ریاضیاتی در این کار نهفته است؟

مسئله 11.7: حذف کلمات ناموجود (Eliminating Non-occurring Terms)

نشان دهید که چگونه کلماتی که در پرسمان وجود ندارند (یعنی $q_i = 0$) از معادله نهایی رتبه‌بندی Retrieval Status Value (RSV) خط می‌خورند و نیازی به محاسبه ندارند.

مسئله 11.8: جدول توافقی کلمات (Term Contingency Table)

یک جدول توافقی (Contingency Table) 2×2 برای حضور و عدم حضور یک کلمه در اسناد مرتبط و نامرتب رسم کنید و پارامترهای p_{-i} و u_{-i} را روی آن مشخص کنید.

مسئله 11.9: مشکل احتمالات صفر (The Zero Probability Problem)

اگر در مجموعه اسناد، یک کلمه کلیدی در هیچ سند مرتبطی یافت نشود، مقدار $p_i = 0$ خواهد شد. این موضوع چه مشکلی در محاسبه لگاریتم در فرمول RSV ایجاد می‌کند و راه حل تئوری آن چیست؟

مسئله 11.10: هموارسازی نیم‌واحدی (Add-0.5 Smoothing)

در عمل برای جلوگیری از خطای لگاریتم، از هموارسازی استفاده می‌شود. فرمول محاسبه p_i و u_i را با اضافه کردن ثابت 0.5 بازنویسی کنید و نقش متغیرهای S ، s ، N و df_i را در آن توضیح دهید.

مسئله 11.11: مقادیر پیش‌فرض احتمالات (Default Probability Values)

هنگامی که کاربر هنوز هیچ بازخوردی نداده است (شروع جستجو)، سیستم هیچ اطلاعاتی از اسناد مرتبط ندارد. در این حالت، مدل BIM به صورت پیش‌فرض مقادیر p_i را چقدر در نظر می‌گیرد و مقدار u_i چگونه تخمین زده می‌شود؟

مسئله 11.12: مقایسه احتمالات و IDF (Probabilities vs. IDF)

اثبات کنید که اگر در مدل BIM فرض کنیم $p_i = 0.5$ است، فرمول وزن‌دهی احتمالی کلمات به فرمول Inverse Document Frequency (IDF) بسیار نزدیک می‌شود.

مسئله 11.13: بازخورد شبه-ارتباط (Pseudo-relevance Feedback)

سیستم‌های بازیابی چگونه از رویکرد $Pseudo-relevance feedback$ در مدل‌های احتمالی استفاده می‌کنند تا بدون نیاز به دخالت کاربر، مقادیر p_i را بهبود ببخشند؟

مسئله 11.14: محدودیت‌های مدل‌های احتمالی (Limitations of Probabilistic Models)

دو مورد از مهم‌ترین ضعف‌های مدل BIM (مانند عدم استفاده از فرکانس کلمه درون سند) را نام برده و توضیح دهید که این ضعف‌ها چگونه بر کیفیت جستجو اثر می‌گذارند.

مسئله 11.15: تفاوت بازخورد احتمالی و برداری (Probabilistic vs. Vector Space Feedback)

بازخورد ارتباط در مدل احتمالی p_i چه تفاوت بنیادی با روش $Rocchio$ در مدل فضای برداری ($Vector Space Model$) دارد؟

مسئله 11.16: وابستگی کلمات در پرسمان (Term Dependencies in Queries)

یک پرسمان مانند $New York$ را در نظر بگیرید. چرا مدل BIM در پردازش این پرسمان دچار خطای محاسباتی می‌شود؟ فرض استقلال در اینجا چه آسیبی به رتبه‌بندی می‌زند؟

مسئله 11.17: رتبه‌بندی با لحاظ هزینه در سیستم پزشکی (Cost-Aware Ranking in Medical Systems)

شما در حال توسعه یک سیستم بازیابی برای پرونده‌های حساس پزشکی هستید. در این سیستم، هزینه از دست دادن یک سند مرتبط (منفی کاذب) 10 برابر بیشتر از هزینه نشان دادن یک سند نامرتبط به کاربر (مثبت کاذب) است ()

داده شود محاسبه کنید. $C_{10} = 10 \times C_{01}$). با استفاده از PRP ، آستانه احتمال $P(R=1|d)$ را برای اینکه یک سند به پزشک نمایش داده شود محاسبه کنید.

مسئله 11.18: سناریوی تکامل وزن‌دهی کلمات با بازخورد (Term Weight Evolution Scenario with Feedback)

در یک جستجو برای کلمه t_1 ، پایگاه داده حاوی 1000 سند است و t_1 در 100 سند وجود دارد. در دور اول جستجو، سیستم 5 سند برمی‌گرداند. کاربر 3 سند را مرتبط می‌داند که 2 تای آن‌ها حاوی t_1 هستند. الف) وزن کلمه t_1 (مقدار RSV آن) را قبل از بازخورد محاسبه کنید. ب) وزن کلمه t_1 را پس از دریافت بازخورد کاربر محاسبه کنید و نشان دهید وزن این کلمه چه تغییری کرده است.

مسئله 11.19: مدل‌سازی وابستگی درختی عبارات (Modeling Tree-Structured Dependencies)

کاربری پرسمان "Machine Learning Python" را جستجو کرده است. به جای فرض استقلال کامل، از یک درخت برای مدل‌سازی وابستگی استفاده می‌شود. با فرض اینکه بیشترین وابستگی بین Machine و Learning و سپس بین Learning و Python باشد، فرمول احتمال مشترک $P(t_1, t_2, t_3 | R=1)$ را با استفاده از تقریب درختی بنویسید.

مسئله 11.20: الگوریتم ساخت درخت چو-لیو (Chow-Liu Tree Algorithm)

در ادامه سوال قبل، سیستم چگونه می‌فهمد که کدام کلمات به هم وابسته‌ترند؟ الگوریتم پایه برای محاسبه وزن یال‌ها بین کلمات پرسمان در گراف چیست و چگونه درخت پوشای بیشینه (Maximum Spanning Tree) به استخراج این وابستگی‌ها کمک می‌کند؟

مسئله 11.21: کالیبراسیون BM25 برای متون حقوقی (BM25 Calibration for Legal Texts)

شما به عنوان مهندس جستجو در یک استارت‌آپ حقوقی کار می‌کنید. در متون حقوقی، تکرار کلمات لزوماً به معنای مرتبط بودن بیشتر نیست (اشباع سریع فرکانس). همچنین طول اسناد به شدت متغیر است و اسناد طولانی صرفاً به خاطر طولانی بودن نباید جریمه شوند. در فرمول Okapi BM25 ، پارامترهای k_1 و b را برای این سناریوی خاص چگونه تنظیم می‌کنید؟ (مقادیر را به سمت صفر یا یک تحلیل کنید).

مسئله 11.22: تحلیل ریاضی پارامتر b در BM25 (Mathematical Analysis of BM25 parameter b)

در فرمول BM25 ، عبارت مخرج شامل ترمی به صورت $(1 - b + b \frac{dl}{avdl})$ است. به صورت ریاضی اثبات کنید که اگر $b = 1$ باشد، سیستم چه رفتاری در قبال اسناد طولانی دارد و اگر $b = 0$ باشد، تأثیر طول سند dl چگونه در رتبه‌بندی خنثی می‌شود.

مسئله 11.23: شبکه بیزی سه‌لایه برای بازیابی اطلاعات (Three-Layer Bayesian Network for IR)

یک سیستم اطلاعاتی با استفاده از شبکه‌های بیزی شامل 3 لایه طراحی کنید: لایه اسناد (D) ، لایه کلمات (T) و لایه پرسمان (Q) . اگر سند شماره یک (D_1) دارای وزن بالایی برای کلمه T_1 باشد، و پرسمان کاربر شبکه را فعال کند، مسیر جریان

احتمالات را رسم کرده و توضیح دهید چگونه Evidence (پرسمان) منجر به محاسبه احتمال عقین (Posterior) برای D_1 می‌شود.

مسئله 11.24: تفاوت فلسفی مدل زبان و مدل احتمالی (Philosophical Difference of Language vs.) (Probabilistic Models)

توضیح دهید در حالی که مدل BIM بر پایه «احتمال مرتبط بودن یک سند با توجه به یک پرسمان» یعنی $P(R=1 \mid d, q)$ کار می‌کند، مدل‌های زبانی (Language Models) پرسمان و سند را چگونه به صورت معکوس مدل‌سازی می‌کنند؟

مسئله 11.25: معماری سیستم و فشرده‌سازی در BM25 (System Architecture and Compression in BM25)

در یک موتور جستجو با مقیاس وب، برای محاسبه BM25 نیاز به طول تک‌تک اسناد (dl) داریم. ذخیره این طول‌ها در حافظه اصلی رم برای میلیاردها سند غیرممکن است. چه استراتژی معماری پیشنهاد می‌دهید که با استفاده از تقریب طول اسناد (مثلاً دسته‌بندی طول‌ها) و کدگذاری طول متغیر، هم BM25 محاسبه شود و هم محدودیت حافظه رفع گردد؟

مسئله 11.26: مسئله مقداردهی اولیه در سیستم‌های فروشگاهی (Initialization Problem in E-commerce)

در یک موتور جستجوی فروشگاهی، محصولات جدیدی اضافه شده‌اند که هیچ کلیک یا داده بازخوردی ندارند. اگر از مدل بازخورد ارتباط احتمالی استفاده کنید، این اسناد جدید (Cold Start) چگونه رتبه‌بندی می‌شوند؟ سناریویی طراحی کنید که این مشکل را با استفاده از ویژگی‌های محتوایی در مدل احتمالی حل کند.

مسئله 11.27: مقایسه Okapi BM25 و BIM (Comparing Okapi BM25 and BIM)

مدل Okapi BM25 به عنوان یک مدل «غیر باینری» (Non-binary) معرفی می‌شود. سه تفاوت اصلی در فرمول‌بندی BM25 که ضعف‌های مدل استقلال باینری (BIM) را پوشش می‌دهد، به طور تحلیلی شرح دهید.

مسئله 11.28: سناریوی شبکه‌های بیزی با کلمات مترادف (Bayesian Networks Scenario with Synonyms)

در رویکرد شبکه بیزی، چگونه می‌توان مفهوم «کلمات مترادف» (Synonyms) را در لایه مفاهیم شبکه وارد کرد؟ گراف شبکه‌ای را رسم کنید که نشان دهد جستجوی کلمه "Car" می‌تواند سندی که فقط حاوی کلمه "Automobile" است را از طریق یک گره میانی متصل کند.

مسئله 11.29: تحلیل حساسیت پارامتر k_1 در عمل (Sensitivity Analysis of k_1 in Practice)

با رسم یک نمودار فرضی نشان دهید که با افزایش پارامتر k_1 در BM25 ، نمودار تأثیر فرکانس کلمه (TF) روی امتیاز نهایی چه تغییری می‌کند و چه زمانی رفتار آن به مدل فضای برداری خام (بدون لگاریتم) شبیه می‌شود.

مسئله 11.30: خلاصه و ترکیب مفاهیم (Summary and Concept Integration)

با توجه به خلاصه فصل 11.5، یک جدول مقایسه‌ای بکشید و سه مدل **VSM**، **BIM** و **BM25** را از نظر:

1. استفاده از فرکانس کلمه (**TF**)

2. استفاده از طول سند

3. پشتیبانی ذاتی از بازخورد ارتباط، مقایسه کنید

مسئله 11.31: پیاده‌سازی تابع محاسبه IDF در BM25 با پایتون (Coding: Implementing BM25 IDF in Python) ★

با توجه به اینکه فرمول **IDF** در **BM25** با **IDF** کلاسیک متفاوت است، یک قطعه کد به زبان **Python** بنویسید که دیکشنری شامل مقادیر **Document Frequency** کلمات و تعداد کل اسناد (**N**) را بگیرد و دیکشنری جدیدی از مقادیر پردازش شده **IDF** بر اساس فرمول **Okapi BM25** تولید کند.

مسئله 11.32: پیاده‌سازی کامل سیستم رتبه‌بندی BM25 (Coding: Full BM25 Ranking System) ★

تابعی کامل در پایتون طراحی کنید که پارامترهای زیر را به عنوان ورودی دریافت کند:

1. **query** (لیستی از کلمات پرسمان)

2. **documents** (لیستی از اسناد که هر سند یک لیست از کلمات است)

3. **k1** و **b** (پارامترهای تنظیم)

خروجی باید لیستی از تاپل‌ها (شماره سند، امتیاز **BM25**) باشد که به صورت نزولی مرتب شده‌اند.

پاسخنامه مسئله 11.1: محاسبه احتمال شرطی در بازیابی (Conditional Probability in Retrieval)

بخش 1) مفاهیم کلیدی و کالبدشکافی مسئله

در مدل‌های احتمالاتی بازیابی اطلاعات (**Probabilistic Retrieval Models**)، برای ارزیابی و تخمین میزان ارتباط یک سند با نیاز اطلاعاتی کاربر، از «قضیه بیز» (**Bayes' Theorem**) استفاده می‌شود. این قضیه ابزاری قدرتمند در ریاضیات است که به ما اجازه می‌دهد با در دست داشتن اطلاعات اولیه و ترکیب آن با شواهد جدید (مانند مشاهده یک کلمه خاص در متن سند)، احتمال وقوع یک رویداد (مرتبط بودن سند) را به‌روزرسانی و محاسبه کنیم.

بخش 2) تحلیل داده‌ها و نمادگذاری متغیرها

برای حل دقیق مسئله، ابتدا متغیرها را با استفاده از نمادهای استاندارد ریاضی مدل‌سازی می‌کنیم. فرض کنید رویداد $R = 1$ نشان‌دهنده «مرتبط بودن سند» و رویداد T نشان‌دهنده «وجود کلمه هدف (هوش) در سند» باشد. بر اساس صورت مسئله، مقادیر زیر استخراج می‌شوند:

الف) احتمال پیشین (**Prior Probability**): احتمال مرتبط بودن یک سند تصادفی، پیش از بررسی محتوای متنی آن:

$$P(R=1) = 0.01$$

ب) احتمال درست‌نمایی (**Likelihood**): احتمال وجود کلمه «هوش» به شرطی که بدانیم سند قطعا مرتبط است:

$$P(T|R=1) = 0.20$$

ج) احتمال حاشیه‌ای (Marginal Probability): احتمال کل رخداد کلمه «هوش» در تمام اسناد مجموعه (چه مرتبط و

چه غیرمرتبط): $P(T) = 0.05$

خواسته مسئله: یافتن احتمال شرطی مرتبط بودن سند، با علم به اینکه کلمه «هوش» در آن مشاهده شده است؛ به زبان ریاضی باید مقدار $P(R=1|T)$ را محاسبه کنیم.

بخش 3) طراحی منطق حل و محاسبات گام به گام

گام اول: بیان رابطه ریاضی پایه (بسط قضیه بیز)

بر اساس فرمول استاندارد قضیه بیز برای این دو رویداد، رابطه ریاضی حاکم بر مسئله به شکل زیر تعریف می‌شود:

$$P(R = 1|T) = \frac{P(T|R = 1) \times P(R = 1)}{P(T)}$$

گام دوم: جایگذاری مقادیر استخراج شده در فرمول

مقادیری که در بخش تحلیل داده‌ها فهرست کردیم را مستقیماً در رابطه بیز قرار می‌دهیم:

$$P(R = 1|T) = \frac{0.20 \times 0.01}{0.05}$$

گام سوم: انجام محاسبات جبری و ساده‌سازی کسرها

برای جلوگیری از خطای محاسباتی، ابتدا عملیات ضرب در صورت کسر را اعمال می‌کنیم:

$$P(R = 1|T) = \frac{0.002}{0.05}$$

جهت حذف اعشار و ساده‌سازی بهتر در محاسبات دستی، صورت و مخرج کسر را در عدد **1000** ضرب می‌کنیم:

$$P(R = 1|T) = \frac{0.002 \times 1000}{0.05 \times 1000} = \frac{2}{50}$$

با ساده کردن کسر نهایی (تقسیم هم‌زمان صورت و مخرج بر مقسوم‌علیه مشترک یعنی عدد 2) به پاسخ نهایی می‌رسیم:

$$P(R = 1|T) = \frac{1}{25} = 0.04$$

پاسخ نهایی و نتیجه‌گیری سیستم

احتمال اینکه سندی حاوی کلمه «هوش» با نیاز اطلاعاتی کاربر مرتبط باشد، دقیقاً برابر با **0.04** یا به عبارتی **4 درصد** است.

درک تحلیلی این مسئله در مهندسی بازیابی اطلاعات بسیار حائز اهمیت است. توجه داشته باشید که در ابتدا (پیش از مشاهده کلمه «هوش»)، شانس مرتبط بودن یک سند تصادفی در مجموعه اسناد تنها 1 درصد بود ($P(R=1) = 0.01$). اما پس از رویت این کلمه کلیدی در متن سند، احتمال ارتباط آن به 4 درصد افزایش یافت (رشد 4 برابری شانس موثر). این تغییر ملموس آماری اثبات می‌کند که واژه «هوش» دارای ارزش اطلاعاتی بسیار بالایی (**Information Value**) در فرآیند تفکیک است و موتور جستجو باید در مکانیزم رتبه‌بندی، به اسنادی که حاوی این واژه هستند وزن محاسباتی بیشتری اختصاص دهد.

پاسخنامه مسئله 11.2: متغیرهای تصادفی در رتبه‌بندی (Random Variables in Ranking)

بخش 1) مفاهیم کلیدی

در اصل رتبه‌بندی احتمالی (**Probability Ranking Principle**) یا به اختصار **PRP**، فرض بر این است که سیستم بازیابی اطلاعات باید اسناد را بر اساس احتمال مرتبط بودنشان با درخواست کاربر مرتب کند. در این چارچوب نظری، «ارتباط» (**Relevance**) یک ویژگی قطعی و سیاه‌وسفید نیست، بلکه به عنوان یک متغیر تصادفی دودویی (**Binary Random Variable**) با نماد **R** مدل‌سازی می‌شود. این متغیر تنها می‌تواند دو مقدار بپذیرد:

R = 1: نشان‌دهنده مرتبط بودن سند (Relevant)

R = 0: نشان‌دهنده غیرمرتبط بودن سند (Non-relevant)

بخش 2) تحلیل معنایی متغیرها

هنگامی که یک سیستم بازیابی با یک سند خاص (که آن را با نماد **d** نشان می‌دهیم) مواجه می‌شود، باید تصمیم بگیرد که این سند چقدر برای کاربر مفید است. در اینجا دو مفهوم آماری شکل می‌گیرد:

1. متغیر $P(R=1|d)$: این عبارت به معنای «احتمال شرطی مرتبط بودن سند، با فرض مشاهده محتوای سند **d**» است.

به بیان ساده‌تر، این عدد نشان می‌دهد که بر اساس شواهد موجود در سند **d** (مانند کلمات کلیدی، ساختار و محتوا)، چقدر احتمال دارد که این سند نیاز اطلاعاتی کاربر را برطرف کند.

2. متغیر $P(R=0|d)$: این عبارت به معنای «احتمال شرطی غیرمرتبط بودن سند، با فرض مشاهده محتوای سند **d**»

است. این عدد نشان می‌دهد که با وجود محتوای سند **d**، چقدر احتمال دارد که این سند برای کاربر کاملاً بی‌فایده باشد.

طبق اصول پایه نظریه احتمالات، چون یک سند در نهایت یا مرتبط است یا غیرمرتبط (حالت سومی وجود ندارد)، مجموع این دو احتمال همواره برابر با 1 (یا 100 درصد) خواهد بود:

$$P(R = 1|d) + P(R = 0|d) = 1$$

بخش 3) حل گام‌به‌گام با ارائه یک مثال کاربردی

برای درک عمیق‌تر تفاوت معنایی این دو متغیر، فرایند جستجوی زیر را در نظر بگیرید:

گام اول: تعریف سناریوی جستجو

فرض کنید کاربری در حال مطالعه برای یک آزمون سیستم عامل است و عبارت جستجوی **(Query)** «الگوریتم زمان بندی **Round Robin**» را در موتور جستجو وارد می کند.

گام دوم: بررسی سند اول (d_1)

موتور جستجو سند d_1 را پیدا می کند که یک مقاله دانشگاهی معتبر با عنوان «تحلیل عملکرد زمان بندی پردازنده به روش راند-رابین» است. کلمات کلیدی کاربر بارها در متن این مقاله تکرار شده اند.

تحلیل $P(R=1|d_1)$: از آنجا که محتوای سند تطابق بالایی با نیاز کاربر دارد، سیستم محاسبه می کند که احتمال مرتبط بودن این سند بسیار بالاست. به عنوان مثال: $P(R=1|d_1) = 0.95$ (یعنی 95 درصد احتمال دارد این سند دقیقاً همان چیزی باشد که کاربر می خواهد).

تحلیل $P(R=0|d_1)$: به صورت منطقی، احتمال نامربوط بودن این مقاله بسیار پایین است: $P(R=0|d_1) = 0.05$.

گام سوم: بررسی سند دوم (d_2)

سپس موتور جستجو سند d_2 را بررسی می کند که یک صفحه وب با عنوان «برنامه ریزی و زمان بندی در پروژه های عمرانی» است. کلمه «زمان بندی» در آن وجود دارد، اما هیچ ارتباطی به مفاهیم کامپیوتر یا **Round Robin** ندارد.

تحلیل $P(R=1|d_2)$: شواهد موجود در این سند ارتباط معنایی با درخواست کاربر ندارند. بنابراین احتمال مرتبط بودن آن بسیار ناچیز ارزیابی می شود: $P(R=1|d_2) = 0.02$.

تحلیل $P(R=0|d_2)$: در مقابل، سیستم با اطمینان بالایی متوجه می شود که این سند برای کاربر غیرمرتبط و بی فایده است: $P(R=0|d_2) = 0.98$.

نتیجه گیری نهایی

تفاوت معنایی این دو متغیر در این است که $P(R=1|d)$ **شانس موفقیت** یک سند در پاسخگویی به کاربر را اندازه گیری می کند، در حالی که $P(R=0|d)$ **ریسک بی فایده بودن** آن سند را نشان می دهد. در موتورهای جستجوی واقعی که بر اساس اصل **PRP** کار می کنند، سیستم تمام اسناد بازبینی شده را بر اساس مقدار $P(R=1|d)$ از بزرگ به کوچک مرتب (**Rank**) کرده و به کاربر نمایش می دهد تا کاربر ابتدا اسنادی را ببیند که بالاترین شانس ارتباط را دارند.

پاسخنامه مسئله 11.3: فرض استقلال اسناد در PRP

(Document Independence Assumption in PRP)

بخش 1) مفاهیم کلیدی

در حالت پایه «اصل رتبه بندی احتمالی» (**Probability Ranking Principle**)، که معمولاً با تابع زیان 1 و 0 (**1/0 Loss Case**) مدل سازی می شود، یک فرض اساسی وجود دارد: **استقلال اسناد**. این فرض بیان می کند که احتمال مرتبط بودن یک سند با نیاز کاربر، کاملاً مستقل از سایر اسناد موجود در مجموعه یا اسنادی است که پیش از آن به کاربر نمایش داده شده اند. به زبان ساده تر، سیستم فرض می کند که ارزش و سودمندی سند d_i برای کاربر، هیچ تأثیری بر ارزش سند d_j نمی گذارد.

بخش 2) تحلیل داده ها و مسئله

در یک سیستم ایده‌آل و تئوریک، اگر کاربر به دنبال مفهومی بگردد، سیستم تک‌تک اسناد را به صورت مجزا ارزیابی کرده و نمره $P(R=1|d)$ را برای هرکدام محاسبه می‌کند. سپس اسناد را صرفاً بر اساس همین نمره‌ی مستقل، از بزرگ به کوچک مرتب می‌کند. با این حال، در دنیای واقعی و در رفتار واقعی کاربران هنگام جستجو، این فرض استقلال منطقی نیست و به دلایل متعددی نقض می‌شود.

بخش 3) حل گام‌به‌گام (تحلیل علل نقض فرض استقلال در واقعیت)

برای درک اینکه چرا ارتباط یک سند در واقعیت به اسناد دیگر وابسته است، 3 دلیل اصلی را بررسی می‌کنیم:

گام اول: افزونگی اطلاعات (Information Redundancy)

فرض کنید کاربری عبارت «اخبار زلزله امروز» را جستجو می‌کند. سیستم بازیابی، 10 خبرگزاری مختلف را پیدا می‌کند که همگی یک خبر واحد را کلمه‌به‌کلمه کپی کرده‌اند.

دیدگاه PRP (مستقل): سیستم به هر 10 سند بالاترین امتیاز را می‌دهد و همه را در صدر نتایج رتبه‌بندی می‌کند.

واقعیت (وابسته): وقتی کاربر سند اول را می‌خواند و نیاز اطلاعاتی‌اش برطرف می‌شود، سند دوم (که دقیقاً همان اطلاعات را دارد) دیگر برای او ارزشی نخواهد داشت و «غیرمرتبط» تلقی می‌شود. در اینجا، مرتبط بودن سند دوم کاملاً وابسته به این است که آیا کاربر سند اول را خوانده است یا خیر.

گام دوم: ابهام در پرس‌وجو و نیاز به تنوع (Query Ambiguity and Diversity)

کاربران غالباً از کلمات کلیدی کوتاه و مبهم استفاده می‌کنند. به عنوان مثال، جستجوی کلمه **Java** می‌تواند به معنای «زبان برنامه‌نویسی جاوا»، «جزیره جاوا در اندونزی» یا «قهوه جاوا» باشد.

دیدگاه PRP (مستقل): اگر سیستم تشخیص دهد که بیشتر اسناد مجموعه درباره «زبان برنامه‌نویسی» هستند، ممکن است 20 سند اول را تماماً به برنامه‌نویسی اختصاص دهد.

واقعیت (وابسته): یک سیستم هوشمند می‌داند که اگر سند اول و دوم درباره زبان برنامه‌نویسی بودند، سوم باید درباره جزیره یا قهوه باشد تا تمام جنبه‌های احتمالی نیاز کاربر پوشش داده شود. در اینجا، انتخاب سند سوم وابسته به محتوای اسناد قبلی است تا تنوع (**Diversity**) نتایج حفظ شود.

گام سوم: مکمل بودن اسناد (Complementarity)

گاهی اوقات یک سند به تنهایی پاسخگوی نیاز کاربر نیست، اما در کنار یک سند دیگر، ارزش بسیار بالایی پیدا می‌کند.

به عنوان مثال، یک دانشجو به دنبال یادگیری «نحوه پیاده‌سازی فشرده‌سازی بایت متغیر

(Variable Byte Code) است. سند اول فقط تئوری ریاضی را توضیح داده و سند دوم فقط کدهای پایتون را

بدون توضیح قرار داده است. هر کدام از این اسناد به تنهایی ممکن است برای کاربر کاملاً مفید نباشند (ارتباط

پایین)، اما ترکیب خواندن سند اول و سپس سند دوم، نیاز کاربر را کاملاً رفع می‌کند. در این حالت، ارزش سند

دوم کاملاً وابسته به اطلاعات سند اول است.

نتیجه‌گیری نهایی

فرض استقلال اسناد در مدل پایه **PRP**، یک ساده‌سازی ریاضی است که از پیچیدگی‌های مسئله می‌کاهد، اما با رفتار واقعی کاربران هم‌خوانی کامل ندارد. در واقعیت، ارزش اطلاعاتی یک سند تابعی از اسنادی است که کاربر پیش از آن مشاهده کرده است.

برای حل مشکل نقض فرض استقلال در موتورهای جستجوی مدرن، مفهوم «ارتباط حاشیه‌ای» (**Marginal Relevance**) معرفی شده است. سیستم‌های امروزی از الگوریتم‌هایی مانند **MMR (Maximal Marginal Relevance)** استفاده می‌کنند تا علاوه بر مرتبط بودن اسناد، میزان «جدید بودن» و «تکراری نبودن» اطلاعات آن‌ها نسبت به اسناد قبلی نمایش داده شده را نیز در رتبه‌بندی لحاظ کنند.

پاسخنامه مسئله 11.4: هزینه خطای جستجو (Search Error Costs)

بخش 1) مفاهیم کلیدی

در تئوری تصمیم‌گیری بیز (**Bayesian Decision Theory**) که پایه‌ی مدل‌های پیشرفته‌ی بازیابی اطلاعات است، هدف تنها مرتب‌سازی اسناد نیست؛ بلکه حداقل کردن «ریسک» یا «هزینه» ناشی از تصمیمات اشتباه است. هر سیستم بازیابی برای هر سند در نهایت یک تصمیم دودویی می‌گیرد: «سند را به کاربر نمایش دهد» یا «سند را فیلتر کند». از آنجا که سیستم‌ها بی‌نقص نیستند، در این تصمیم‌گیری‌ها مرتکب خطا می‌شوند. برای مدل‌سازی ریاضی اثرات منفی این خطاها بر تجربه کاربر، از پارامترهای هزینه (**Cost**) استفاده می‌کنیم.

بخش 2) تحلیل داده‌ها و نمادگذاری

در نمادگذاری استاندارد **C_{ij}**، زیرنویس اول (**i**) نشان‌دهنده‌ی «تصمیم سیستم» و زیرنویس دوم (**j**) نشان‌دهنده‌ی «وضعیت واقعی سند» است:

1: به معنای «مثبت» (بازیابی شدن توسط سیستم / مرتبط بودن در واقعیت)

0: به معنای «منفی» (عدم بازیابی توسط سیستم / غیرمرتبط بودن در واقعیت)

بر این اساس، 2 نوع خطای اساسی و هزینه‌های مترتب بر آن‌ها در سیستم رخ می‌دهد که در ادامه آن‌ها را تحلیل می‌کنیم.

بخش 3) حل گام به گام

گام اول: تحلیل متغیر C₁₀ (خطای بازیابی سند غیرمرتبط)

مفهوم نماد: سیستم تصمیم گرفته است سند را بازیابی کند (**i = 1**)، در حالی که وضعیت واقعی سند، غیرمرتبط با نیاز کاربر است (**j = 0**).

معادل آماری: این حالت دقیقاً بیانگر خطای مثبت کاذب (**False Positive**) یا خطای نوع اول (**Type I Error**) است. سیستم به اشتباه فکر کرده که این سند برای کاربر مفید است.

تأثیر بر کاربر: هزینه‌ی این خطا به شکل «اتلاف وقت کاربر» خود را نشان می‌دهد. کاربر روی لینک کلیک می‌کند، متن را می‌خواند و متوجه می‌شود که محتوا هیچ ربطی به نیاز او ندارد. این خطا باعث کاهش دقت (**Precision**) سیستم می‌شود.

گام دوم: تحلیل متغیر C₀₁ (خطای عدم بازیابی سند مرتبط)

مفهوم نماد: سیستم تصمیم گرفته است سند را بازبایی نکند و آن را کنار بگذارد ($i = 0$)، در حالی که در

واقعیت، آن سند حاوی اطلاعات مرتبط و مفیدی برای کاربر بوده است ($j = 1$).

معادل آماری: این حالت بیانگر خطای منفی کاذب (**False Negative**) یا خطای نوع دوم (**Type II Error**)

است. سیستم نتوانسته است یک سند ارزشمند را تشخیص دهد.

تأثیر بر کاربر: هزینه‌ی این خطا به شکل «از دست دادن اطلاعات حیاتی» بروز می‌کند. کاربر هرگز متوجه

نمی‌شود که چنین سند مفیدی وجود داشته است. این خطا مستقیماً باعث کاهش فراخوانی (**Recall**)

سیستم می‌شود.

نتیجه‌گیری نهایی

به‌طور خلاصه، در مدل‌سازی هزینه‌های سیستم بازبایی:

متغیر **C₁₀** معادل خطای مثبت کاذب (**False Positive**) است (ارائه اطلاعات بی‌ارزش به کاربر).

متغیر **C₀₁** معادل خطای منفی کاذب (**False Negative**) است (پنهان ماندن اطلاعات ارزشمند از دید کاربر).

نکته آموزشی:

در طراحی یک موتور جستجوی واقعی، وزن این 2 هزینه معمولاً برابر نیست. به عنوان مثال، در یک موتور جستجوی وب عمومی مانند گوگل، هزینه **C₁₀** بسیار بالاست؛ زیرا اگر در صفحه اول نتایج غیرمرتبط (اسپم) نمایش داده شود، کاربر بلافاصله ناراضی می‌شود. اما در جستجوهای حقوقی (**Legal Search**) یا پزشکی، هزینه **C₀₁** به شدت بحرانی است؛ زیرا پیدا نکردن یک پرونده قضایی مرتبط یا یک مقاله دارویی خاص (منفی کاذب) می‌تواند خسارات جبران‌ناپذیری به همراه داشته باشد. در چنین سیستم‌هایی، آستانه تصمیم‌گیری به گونه‌ای تنظیم می‌شود که به قیمت پذیرش چند مثبت کاذب، هیچ منفی کاذبی رخ ندهد.

پاسخنامه مسئله 11.5: اصول مدل BIM (BIM Principles)

بخش 1) مفاهیم کلیدی

مدل استقلال باینری (**Binary Independence Model**) یا به اختصار **BIM**، یکی از پایه‌ای‌ترین و شناخته‌شده‌ترین مدل‌های احتمالاتی در بازبایی اطلاعات است. این مدل برای غلبه بر پیچیدگی‌های محاسباتی در تخمین احتمال ارتباط یک سند، فرایند مدل‌سازی را بر اساس 2 فرض ساده‌کننده و در عین حال محدودکننده بنا می‌کند. نام این مدل مستقیماً از همین 2 فرض گرفته شده است: **بازنمایی باینری (Binary)** و **استقلال کلمات (Independence)**.

بخش 2) تحلیل داده‌ها و مسئله

هدف این مسئله، کالبدشکافی این 2 کلمه در نام مدل و درک دقیق محدودیت‌هایی است که هر کدام به سیستم بازبایی (در نحوه نگاه به اسناد و پرسمان‌ها) تحمیل می‌کنند. در ادامه، هر یک از این اصول را به تفکیک بررسی و محدودیت‌های ناشی از آن‌ها را تحلیل می‌کنیم.

بخش 3) حل گام‌به‌گام

گام اول: تحلیل مفهوم «باینری» (Binary Representation) و محدودیت‌های آن

تعریف: در مدل **BIM**، هر سند و هر پرسمان (**Query**) تنها به شکل یک بردار باینری (0 و 1) از کلمات بازنمایی می‌شود. اگر لغت‌نامه سیستم دارای **M** کلمه باشد، یک سند به صورت بردار $\vec{x} = (x_1, x_2, \dots, x_M)$ نشان داده می‌شود که در آن $x_i \in \{0, 1\}$ است. اگر کلمه در سند وجود داشته باشد مقدار آن 1 و در غیر این صورت 0 خواهد بود.

محدودیت‌های ایجادشده:

1. **نادیده گرفتن بسامد کلمه (Term Frequency - TF):** این مدل هیچ تفاوتی بین سندی که کلمه «الگوریتم» در آن 50 بار تکرار شده با سندی که این کلمه تنها 1 بار در آن آمده، قائل نمی‌شود. هر دو سند برای این کلمه مقدار 1 را دریافت می‌کنند. این در حالی است که تکرار یک کلمه سیگنال قدرتمندی از موضوع اصلی سند است.

2. **نادیده گرفتن طول سند (Document Length):** در یک بردار باینری، طول واقعی متن گم می‌شود. یک مقاله 100 صفحه‌ای و یک پاراگراف 3 خطی در صورتی که کلمات منحصر به فرد یکسانی داشته باشند، دقیقاً دارای بردارهای باینری مشابهی خواهند بود، که این امر در رتبه‌بندی عادلانه اختلال ایجاد می‌کند.

گام دوم: تحلیل مفهوم «استقلال» (Independence Assumption) و محدودیت‌های آن

تعریف: این فرض بیان می‌کند که حضور یا عدم حضور یک کلمه در سند، کاملاً مستقل از حضور یا عدم حضور سایر کلمات در آن سند است (استقلال شرطی با فرض مشخص بودن وضعیت ارتباط). از نظر ریاضی، احتمال مشاهده بردار سند $\vec{x}$ برابر است با حاصل ضرب احتمال مشاهده تک‌تک کلمات آن:

$$P(\vec{x}|R) = \prod_{i=1}^M P(x_i|R)$$

محدودیت‌های ایجادشده:

1. **نادیده گرفتن هم‌رخدادی کلمات (Co-occurrence):** در زبان طبیعی، کلمات به شدت به هم وابسته‌اند. اگر در متنی کلمه «سیستم» بیاید، احتمال ظاهر شدن کلمه «عامل» بسیار بالاست. مدل **BIM** این ارتباطات آماری و معنایی را کاملاً کورکورانه نادیده می‌گیرد و فرض می‌کند آمدن کلمه «سیستم» هیچ اطلاعات اضافه‌ای درباره احتمال آمدن کلمه «عامل» به ما نمی‌دهد.

2. **از بین رفتن ساختار و عبارات (Phrases):** چون کلمات مستقل فرض می‌شوند، توالی آن‌ها بی‌اهمیت می‌شود. برای این مدل، عبارت «پایگاه داده» صرفاً ترکیب 2 کلمه مستقل است و مفهوم یکپارچه آن قابل درک نیست.

نتیجه‌گیری نهایی

مدل **BIM** با استفاده از کلمه «باینری»، وزن کلمات و تکرار آن‌ها را حذف می‌کند و با کلمه «استقلال»، روابط معنایی و هم‌رخدادی کلمات در زبان طبیعی را از بین می‌برد. این 2 فرض، اگرچه از نظر ریاضی محاسبات احتمالات را بسیار ساده و سریع می‌کنند، اما باعث از دست رفتن بخش بزرگی از اطلاعات حیاتی متن می‌شوند و سقف دقت سیستم بازیابی را محدود می‌کنند.

درک این محدودیت‌ها نقطه شروع بسیار خوبی برای ورود به مباحث پیشرفته‌تر است. در واقع، محققان برای جبران همین نقص «باینری بودن» در **BIM**، مفهوم فراوانی کلمه **(TF)** و طول سند را به مدل‌های احتمالاتی اضافه کردند که نتیجه‌ی آن، تولد تابع رتبه‌بندی مشهور **BM25** بود؛ تابعی که امروزه به عنوان استاندارد صنعتی در موتورهای جستجو شناخته می‌شود.

پاسخنامه مسئله 11.6: نسبت شانس در BIM (Odds Ratio in BIM)

بخش 1) مفاهیم کلیدی

در مدل‌های احتمالاتی بازیابی اطلاعات، هدف اصلی مرتب‌سازی (**Ranking**) اسناد بر اساس احتمال مرتبط بودن آن‌ها با درخواست کاربر است. در مدل استقلال باینری (**BIM**)، به جای محاسبه مستقیم احتمال شرطی $P(R=1|d)$ ، از مفهوم آماری «نسبت شانس» (**Odds Ratio**) استفاده می‌شود. نسبت شانس، نسبتی است میان احتمال رخداد یک رویداد (مرتبط بودن) به احتمال عدم رخداد آن (غیرمرتبط بودن). استفاده از این ترفند، گره‌های محاسباتی بزرگی را در ریاضیات بازیابی اطلاعات باز می‌کند.

بخش 2) تحلیل داده‌ها و نمادگذاری

طبق قضیه بیز، محاسبه مستقیم احتمال مرتبط بودن و غیرمرتبط بودن یک سند (**d**) به شکل زیر بسط داده می‌شود:

$$P(R = 1|d) = \frac{P(d|R = 1) \times P(R = 1)}{P(d)}$$

$$P(R = 0|d) = \frac{P(d|R = 0) \times P(R = 0)}{P(d)}$$

در این روابط، **P(d)** بیانگر احتمال حاشیه‌ای مشاهده‌ی دقیق بردار سند **d** در کل مجموعه اسناد است. محاسبه این مخرج کسر در عمل بسیار دشوار و پرهزینه است.

بخش 3) حل گام‌به‌گام و تحلیل مزیت‌های ریاضی

استفاده از نسبت شانس **O(R|d)** 3 مزیت ریاضیاتی بنیادین برای سیستم ایجاد می‌کند که آن‌ها را مرحله‌به‌مرحله بررسی می‌کنیم:

گام اول: حذف مخرج مشترک و پیچیده (حذف P(d))

با تقسیم 2 رابطه بیز بر یکدیگر برای محاسبه نسبت شانس، عبارت **P(d)** که در مخرج هر 2 وجود دارد، به طور کامل ساده شده و حذف می‌گردد:

$$O(R|d) = \frac{P(R = 1|d)}{P(R = 0|d)} = \frac{P(d|R = 1) \times P(R = 1)}{P(d|R = 0) \times P(R = 0)}$$

مزیت اول: سیستم دیگر نیازی به محاسبه احتمال رخداد تک تک اسناد در کل مجموعه ندارد. از آنجا که هدف فقط رتبه بندی اسناد نسبت به یکدیگر است، این نسبت به تنهایی برای مرتب سازی کفایت می کند (زیرا تابعی صعودی از احتمال است).

گام دوم: اعمال فرض استقلال و تبدیل به ضرب زنجیره ای

با اعمال فرض استقلال کلمات در مدل **BIM**، احتمال مشاهده کل سند به حاصل ضرب احتمال مشاهده تک تک کلمات آن (t_i) تجزیه می شود:

$$O(R|d) = \left(\prod_{i=1}^M \frac{P(t_i|R=1)}{P(t_i|R=0)} \right) \times \frac{P(R=1)}{P(R=0)}$$

مزیت دوم: اکنون سیستم می تواند شانس مرتبط بودن را با ضرب کردن وزن کلمات مجزای موجود در سند محاسبه کند.

گام سوم: لگاریتم گیری و خطی سازی معادله (Linearization)

ضرب تعداد زیادی عدد اعشاری کوچک (احتمالات) در کامپیوتر منجر به خطای «سرریز به پایین» (**Arithmetic Underflow**) می شود و مقدار نهایی 0 مطلق در نظر گرفته می شود. برای حل این مشکل، از طرفین لگاریتم می گیریم. از آنجا که لگاریتم یک تابع اکیداً صعودی است، ترتیب رتبه بندی را تغییر نمی دهد:

$$\log O(R|d) = \sum_{i=1}^M \log \frac{P(t_i|R=1)}{P(t_i|R=0)} + \log \frac{P(R=1)}{P(R=0)}$$

مزیت سوم: با این کار، عملگر ضرب به عملگر جمع تبدیل می شود. کسر اول (احتمال پیشین) برای تمام اسناد ثابت است و در رتبه بندی تاثیری ندارد، بنابراین می توان آن را نادیده گرفت. معادله نهایی به یک جمع خطی ساده از وزن کلمات تبدیل می شود.

نتیجه گیری نهایی

استفاده از نسبت شانس در **BIM**، به ظاهر یک تغییر ساده است، اما در واقعیت 3 چالش بزرگ را حل می کند: 1. حذف متغیر غیرقابل محاسبه $P(d)$ ، 2. جلوگیری از خطای محاسباتی **Underflow** با امکان پذیر ساختن لگاریتم گیری و 3. تبدیل یک فرمول پیچیده ضربی به یک تابع امتیازدهی خطی و جمعی (**Linear Scoring Function**) که پردازش آن برای موتورهای جستجو بسیار سریع است.

نکته آموزشی:

خروجی این فرمول لگاریتمی در متون علمی تحت عنوان **مقدار وضعیت بازیابی (RSV یا Retrieval Status Value)** شناخته می شود. عبارت داخل سیگما نیز دقیقاً همان منطقی است که بعدها پایه و اساس فرمول مشهور «وزن دهی رابرتسون-اسپارک جونز» (**Robertson-Sparck Jones weight**) در بازیابی اطلاعات را شکل داد. در عمل، سیستم به جای احتمالات، برای هر کلمه یک «وزن» محاسبه کرده و وزن کلمات موجود در سند را با هم جمع می کند تا نمره نهایی سند (**RSV**) به دست آید.

پاسخنامه مسئله 11.7: حذف کلمات ناموجود (Eliminating Non-occurring Terms)

بخش 1) مفاهیم کلیدی

در مدل استقلال باینری (BIM)، فرمول پایه برای محاسبه «مقدار وضعیت بازیابی» (RSV یا Retrieval Status Value)، در ابتدا بر روی تمام کلمات موجود در لغتنامه (Vocabulary) سیستم بسط داده می‌شود. با این حال، در یک سیستم واقعی، پردازش ده‌ها هزار کلمه‌ی لغتنامه برای ارزیابی هر سند، از نظر محاسباتی غیرممکن است. یکی از ظرافت‌های مدل BIM این است که با اعمال یک فرض منطقی و آماری درباره کلماتی که کاربر در جستجوی خود وارد نکرده است، توان محاسباتی مورد نیاز را به شدت کاهش داده و این کلمات را به صورت ریاضی از معادله حذف می‌کند.

بخش 2) تحلیل داده‌ها و نمادگذاری

برای اثبات این موضوع، ابتدا نمادهای استاندارد آماری را در تابع رتبه‌بندی تعریف می‌کنیم:

$q_i \in \{0, 1\}$: وضعیت حضور کلمه i در پرسمان (Query) کاربر (1 یعنی حضور، 0 یعنی عدم حضور).

$x_i \in \{0, 1\}$: وضعیت حضور کلمه i در سند (Document).

$p_i = P(x_i = 1 | R = 1)$: احتمال رخداد کلمه i در یک سند مرتبط.

$u_i = P(x_i = 1 | R = 0)$: احتمال رخداد کلمه i در یک سند غیرمرتبط.

c_i : وزن اختصاص‌یافته به کلمه i در معادله رتبه‌بندی.

بخش 3) حل گام‌به‌گام

گام اول: بیان تابع رتبه‌بندی پایه (RSV)

بر اساس مدل BIM، پس از لگاریتم‌گیری و حذف مقادیر ثابت، نمره نهایی یک سند (RSV_d) به صورت مجموع وزن کلماتی که در آن سند حضور دارند ($x_i=1$) محاسبه می‌شود:

$$RSV_d = \sum_{x_i=1} c_i$$

در این مرحله، معادله روی تمام کلمات موجود در سند (چه در پرسمان باشند و چه نباشند) اعمال می‌شود.

گام دوم: اعمال فرض تساوی احتمالات برای کلمات خارج از پرسمان

اگر کلمه‌ای در پرسمان کاربر وجود نداشته باشد ($q_i=0$)، سیستم فرض می‌کند که این کلمه هیچ اطلاعات و شواهدی درباره مرتبط بودن یا نبودن سند به ما نمی‌دهد. به بیان ریاضی، احتمال رخداد چنین کلمه‌ای در اسناد مرتبط، دقیقاً برابر با احتمال رخداد آن در اسناد غیرمرتبط است:

$$p_i = u_i \quad \text{for} \quad q_i = 0$$

گام سوم: محاسبه وزن ریاضی برای کلمات ناموجود

اکنون فرض $p_i = u_i$ را در فرمول محاسبه وزن (c_i) جایگذاری می‌کنیم:

$$c_i = \log \frac{p_i(1 - u_i)}{u_i(1 - p_i)} = \log \frac{u_i(1 - u_i)}{u_i(1 - u_i)}$$

همان‌طور که مشاهده می‌شود، صورت و مخرج کسر کاملاً با هم برابر می‌شوند. با ساده شدن کسر، مقدار داخل لگاریتم برابر با 1 خواهد شد:

$$c_i = \log(1) = 0$$

گام چهارم: به‌روزرسانی و ساده‌سازی معادله نهایی

از آنجا که ثابت کردیم وزن کلمات غایب در پرسمان ($q_i=0$) مطلقاً برابر با 0 است ($c_i=0$)، این کلمات هیچ اثری در جمع نهایی نمره سند ندارند. در نتیجه، عملگر جمع را می‌توان محدود کرد؛ به طوری که تنها روی کلماتی اعمال شود که هم در پرسمان ($q_i=1$) و هم در سند ($x_i=1$) حضور دارند:

$$RSV_d = \sum_{x_i=1 \wedge q_i=1} c_i$$

نتیجه‌گیری نهایی

از طریق معادلات جبری اثبات می‌شود که در مدل **BIM**، کلماتی که $q_i=0$ هستند، نسبت شانس برابر با 1 داشته و در نتیجه پس از لگاریتم‌گیری، وزن آن‌ها 0 می‌شود و به‌طور کامل از محاسبات خط می‌خورند.

نکته آموزشی:

این اثبات ریاضی، اهمیت بنیادینی در معماری سیستم‌های بازیابی اطلاعات (**IR Architecture**) دارد. دقیقاً همین ویژگی است که به ما اجازه می‌دهد در موتورهای جستجو از ساختار «نمایه وارونه» (**Inverted Index**) استفاده کنیم. به جای بررسی تمام کلمات یک سند، سیستم جستجو تنها لیست‌های ارسال (**Posting Lists**) مربوط به کلمات موجود در پرسمان کاربر را فراخوانی می‌کند. این امر پیچیدگی زمانی (**Time Complexity**) پردازش را از مرتبه‌ی «اندازه لغت‌نامه» به مرتبه‌ی «طول پرسمان» کاهش می‌دهد و جستجوی بی‌درنگ (**Real-time Search**) را در پایگاه‌های داده عظیم امکان‌پذیر می‌سازد.

پاسخنامه مسئله 11.8: جدول توافقی کلمات (Term Contingency Table)

بخش 1) مفاهیم کلیدی

در مدل‌سازی احتمالاتی و تخمین پارامترها در مدل استقلال باینری (**BIM**)، برای درک دقیق توزیع یک کلمه خاص در میان اسناد، از ابزاری آماری به نام «جدول توافقی» (**Contingency Table**) استفاده می‌شود. این جدول ماتریسی است که فراوانی (تعداد) اسناد را بر اساس تقاطع 2 متغیر دودویی دسته‌بندی می‌کند:

1. وضعیت ارتباط سند: آیا سند مرتبط است ($R=1$) یا غیرمرتبط ($R=0$)؟

2. وضعیت حضور کلمه: آیا کلمه در سند حضور دارد ($x_i=1$) یا غایب است ($x_i=0$)؟

بخش 2) تحلیل متغیرها و رسم جدول

برای ساخت این جدول، ابتدا نمادهای استاندارد فراوانی را برای یک مجموعه سند با اندازه کل N تعریف می‌کنیم:

S : تعداد کل اسناد مرتبط با پرسمان کاربر.

$N - S$: تعداد کل اسناد غیرمرتبط با پرسمان.

n : تعداد کل اسنادی که کلمه i در آن‌ها وجود دارد.

s : تعداد اسناد مرتبطی که کلمه i در آن‌ها وجود دارد.

بر اساس این متغیرها، تقاطع حالات مختلف در یک جدول 2×2 به شکل زیر تدوین می‌شود:

وضعیت کلمه	اسناد مرتبط ($R=1$)	اسناد غیرمرتبط ($R=0$)	جمع سطر
حاضر ($x_i=1$)	s	$n - s$	n
غایب ($x_i=0$)	$S - s$	$(N - n) - (S - s)$	$N - n$
جمع ستون	S	$N - S$	N

بخش 3) محاسبه پارامترهای p_i و u_i

هدف اصلی از رسم این جدول در مدل **BIM**، استخراج و تخمین 2 احتمال بنیادین p_i و u_i است. با استفاده از داده‌های جدول بالا، این 2 پارامتر را از طریق روابط «احتمال شرطی» محاسبه می‌کنیم:

گام اول: استخراج پارامتر p_i

این پارامتر بیانگر «احتمال حضور کلمه i به شرط مرتبط بودن سند» است. برای محاسبه آن، تنها به ستون اول جدول (اسناد مرتبط) نگاه می‌کنیم. تعداد اسناد مرتبطی که کلمه را دارند (s) را بر کل اسناد مرتبط (S) تقسیم می‌کنیم:

$$p_i = \frac{s}{S}$$

گام دوم: استخراج پارامتر u_i

این پارامتر بیانگر «احتمال حضور کلمه i به شرط غیرمرتبط بودن سند» است. برای محاسبه آن، تنها به ستون دوم جدول (اسناد غیرمرتبط) نگاه می‌کنیم. تعداد اسناد غیرمرتبطی که کلمه را دارند ($n - s$) را بر کل اسناد غیرمرتبط ($N - S$) تقسیم می‌کنیم:

$$u_i = \frac{n - s}{N - S}$$

جدول توافقی، پلی میان تئوری احتمالات انتزاعی و داده‌های قابل شمارش در یک مجموعه سند است. با جایگذاری مقادیر p_i و u_i (به دست آمده از این جدول) در تابع رتبه‌بندی **BIM**، فرمول مشهور وزن‌دهی رابرتسون-اسپارک جونز (Robertson-Sparck Jones Weight) شکل می‌گیرد.

نکته آموزشی:

در عمل، سیستم‌های بازیابی اطلاعات در لحظه جستجو، مقدار دقیق S و s را نمی‌دانند (زیرا هنوز نمی‌دانند کدام اسناد برای کاربر مرتبط هستند). بنابراین، در ابتدا مقادیر پیش‌فرضی برای p_i در نظر می‌گیرند (مثلاً 0.5) و پارامتر u_i را با تقریب $\frac{n}{N}$ محاسبه می‌کنند (با این فرض که اکثر اسناد مجموعه غیرمرتبط هستند). سپس با استفاده از تکنیک «بازخورد کاذب» (**Pseudo-Relevance Feedback**)، این جدول را به صورت پویا تخمین زده و به‌روزرسانی می‌کنند.

پاسخنامه مسئله 11.9: مشکل احتمالات صفر (The Zero Probability Problem)

بخش 1) مفاهیم کلیدی

در مدل استقلال باینری (**BIM**)، محاسبه وزن هر کلمه بر اساس داده‌های موجود در مجموعه اسناد انجام می‌شود. یکی از مشکلات رایج در تخمین احتمالات با استفاده از داده‌های تجربی، مسئله «پراکندگی داده‌ها» (**Data Sparsity**) است. اگر یک کلمه کلیدی، با وجود اهمیتش، به صورت تصادفی در هیچ‌یک از اسنادی که تاکنون مرتبط تشخیص داده شده‌اند ظاهر نشود، احتمال شرطی آن برابر با 0 ($p_i = 0$) خواهد شد. این مسئله در ظاهر یک اتفاق آماری ساده است، اما در ریاضیات تابع رتبه‌بندی، منجر به یک فروپاشی محاسباتی می‌شود.

بخش 2) تحلیل داده‌ها و مسئله

یادآوری می‌کنیم که فرمول پایه برای محاسبه وزن یک کلمه (c_i) در محاسبه مقدار وضعیت بازیابی (**RSV**) به شکل زیر است:

$$c_i = \log \frac{p_i(1 - u_i)}{u_i(1 - p_i)}$$

حال بررسی می‌کنیم که اگر مقدار p_i برابر با صفر شود، چه اختلالی در ساختار این فرمول رخ می‌دهد.

بخش 3) حل گام‌به‌گام

گام اول: بروز خطای ریاضی (محاسبه لگاریتم صفر)

اگر فرض کنیم هیچ سند مرتبطی حاوی کلمه i نیست، مقدار پارامتر مربوطه $p_i = 0$ خواهد شد. با قرار دادن این مقدار در معادله وزن کلمه خواهیم داشت:

$$c_i = \log \frac{0 \times (1 - u_i)}{u_i(1 - 0)} = \log(0)$$

از نظر ریاضیاتی، تابع لگاریتم برای عدد 0 تعریف نشده است (جد آن به سمت منفی بی‌نهایت میل می‌کند). در نتیجه، سیستم بازیابی در محاسبه نمره نهایی دچار خطا (Crash) شده و از کار می‌افتد.

* (توجه داشته باشید که همین مشکل زمانی که یک کلمه در تمام اسناد مرتبط ظاهر شود نیز رخ می‌دهد؛ یعنی در حالتی که $p_i = 1$

باشد، مخرج کسر صفر شده و خطای تقسیم بر صفر به وجود می‌آید). *

گام دوم: راهکار تئوری (تکنیک هموارسازی یا Smoothing)

برای حل این مشکل بنیادین، در تئوری احتمالات و بازیابی اطلاعات از تکنیکی به نام هموارسازی (Smoothing) استفاده می‌شود. ایده اصلی این است که سیستم هیچ‌گاه نباید به یک رویداد، احتمال مطلقاً صفر (غیرممکن) یا مطلقاً یک (قطعی) اختصاص دهد، زیرا داده‌های نمونه‌برداری‌شده همواره محدود هستند و عدم مشاهده یک کلمه در نمونه فعلی، به معنای عدم وجود مطلق آن در کل جامعه آماری نیست.

گام سوم: اعمال هموارسازی 0.5 (تعدیل رابرتسون)

رایج‌ترین روش برای هموارسازی در این مدل، افزودن یک مقدار ثابت و کوچک (معمولاً 0.5) به تمام شمارنده‌های جدول توافقی است. با فرض اینکه s تعداد اسناد مرتبط حاوی کلمه و S کل اسناد مرتبط باشد، فرمول تخمین احتمالات از حالت پایه ($p_i = s / S$) به شکل زیر اصلاح می‌شود:

$$p_i = \frac{s + 0.5}{S + 1}$$

نکته: علت افزودن عدد 1 به مخرج این است که مقدار 0.5 به حالت موفقیت (حضور کلمه) و مقدار 0.5 به حالت شکست (عدم حضور کلمه) اضافه شده است که مجموعاً فضای نمونه در مخرج را 1 واحد افزایش می‌دهد.

نتیجه‌گیری نهایی

مشکل احتمالات صفر باعث تولید مقادیر تعریف‌نشده ریاضی مانند $\log(0)$ یا تقسیم بر 0 در فرمول پایه RSV می‌شود. راه حل قطعی این مشکل، استفاده از تکنیک هموارسازی (افزودن 0.5) است. با این کار، حتی اگر کلمه‌ای در اسناد مرتبط یافت نشود ($s = 0$)، مقدار p_i به جای صفر مطلق، برابر با عددی بسیار کوچک (مثلاً $\frac{0.5}{S+1}$) خواهد شد که اجازه می‌دهد محاسبات لگاریتمی به صورت پایدار و بدون توقف انجام شوند.

نکته آموزشی:

این فرمول هموارشده، مستقیماً پایه و اساس فرمول مشهور «وزن‌دهی رابرتسون-اسپارک جونز» (Robertson-Sparck Jones) را تشکیل داد که از نظر جبری به شکل زیر بسط داده می‌شود:

$$\log \left(\frac{(s + 0.5)(N - S - n + s + 0.5)}{(S - s + 0.5)(n - s + 0.5)} \right)$$

امروزه این فرمول تعدیل‌شده در الگوریتم‌های پیشرفته‌ای مانند Okapi BM25 به عنوان هسته اصلی محاسبه وزن کلمات (IDF پیشرفته) مورد استفاده قرار می‌گیرد و مقاومت سیستم را در برابر مجموعه‌داده‌های تُنک (Sparse Data) تضمین می‌کند.

پاسخنامه مسئله 11.10: هموارسازی نیم‌واحدی (Add-0.5 Smoothing)

بخش 1) مفاهیم کلیدی

در مدل‌های احتمالاتی مانند **BIM**، اتکای صرف به داده‌های نمونه‌برداری شده برای تخمین احتمالات می‌تواند به مقادیر 0 یا 1 منجر شود. برای رفع این ناپایداری ریاضی و جلوگیری از خطای محاسبه لگاریتم، از تکنیک «هموارسازی نیم‌واحدی» (**Add-0.5 Smoothing**) استفاده می‌شود. این روش که توسط رابرتسون و اسپارک جونز معرفی شد، توزیع احتمالات را متعادل کرده و مانع از حذف کامل یک کلمه از محاسبات به دلیل کمبود داده (**Data Sparsity**) می‌گردد.

بخش 2) تحلیل متغیرها و نمادگذاری

برای فرمول‌بندی دقیق این تکنیک، ابتدا باید 4 متغیر کلیدی که فضای جستجو و وضعیت اسناد را توصیف می‌کنند، تعریف کنیم:

N: اندازه کل مجموعه (تعداد کل اسناد موجود در پایگاه داده).

S: کل اسناد مرتبط (تعداد اسنادی که برای پرسمان جاری، مرتبط تشخیص داده شده‌اند).

df_i: فراوانی سند برای کلمه **i** (تعداد کل اسنادی در مجموعه که حاوی کلمه **i** هستند). این متغیر معادل متغیر **n** در

نمادگذاری‌های رایج دیگر است.

s: تعداد اسناد مرتبطی که حاوی کلمه **i** هستند.

بخش 3) حل گام به گام

گام اول: یادآوری احتمالات پایه (بدون هموارسازی)

در حالت پایه، احتمال حضور کلمه در اسناد مرتبط (**p_i**) و غیرمرتبط (**u_i**) از طریق تقسیم مستقیم مشاهدات بر فضای نمونه به دست می‌آید:

$$p_i = \frac{s}{S}, \quad u_i = \frac{df_i - s}{N - S}$$

* (در مخرج **u_i**، کل اسناد غیرمرتبط برابر با **N - S** و در صورت آن، تعداد اسناد غیرمرتبط حاوی کلمه برابر با **df_i - s** است).

گام دوم: اعمال منطق هموارسازی نیم‌واحدی

برای جلوگیری از صفر شدن صورت کسر (در صورت نبود کلمه) یا مخرج آن (در صورت نبود سند مرتبط)، یک مقدار ثابت 0.5 به تعداد مشاهدات موفق (صورت کسر) اضافه می‌شود. از آنجا که این مقدار ثابت باید برای حالت ناموفق (عدم حضور کلمه) نیز به همین میزان لحاظ شود، مجموعاً فضای نمونه (مخرج کسر) به اندازه **0.5 + 0.5 = 1** واحد افزایش می‌یابد.

گام سوم: بازنویسی فرمول‌های نهایی

با اعمال این منطق بر فرمول‌های پایه، روابط اصلاح‌شده برای تخمین احتمالات به شکل زیر تدوین می‌شوند:

فرمول هموارشده **p_i**:

$$p_i = \frac{s + 0.5}{S + 1}$$

$$u_i = \frac{df_i - s + 0.5}{N - S + 1}$$

نتیجه‌گیری نهایی

استفاده از هموارسازی نیم‌واحدی، پارامترهای p_i و u_i را از خطرات ناشی از احتمالات صفر مطلق مصون می‌دارد. متغیرهای S و s به‌طور مستقیم نمایانگر اطلاعات مربوط به اسناد مرتبط هستند، در حالی که N و df_i نمایانگر اطلاعات سراسری مجموعه می‌باشند. ترکیب این 4 متغیر با ثابت‌های هموارسازی، فرمول‌های پایداری برای ارزیابی وزن کلمات ایجاد می‌کند.

نکته آموزشی:

دلیل انتخاب عدد 0.5 (به جای مقادیر دیگر مانند 1 در هموارسازی لاپلاس) در بازیابی اطلاعات، ریشه در توزیع‌های آماری و مطالعات تجربی اولیه دارد. اگر این مقادیر هموارشده را مستقیماً در تابع پایه لگاریتمی قرار دهیم، به رابطه نهایی و بسیار مهم وزن‌دهی رابرتسون-اسپارک جونز (RSJ Weight) می‌رسیم. این فرمول همچنان به عنوان هسته اصلی برای محاسبه IDF در الگوریتم‌های رتبه‌بندی مدرن کاربرد دارد.

پاسخنامه مسئله 11.11: مقادیر پیش‌فرض احتمالات (Default Probability Values)

بخش 1) مفاهیم کلیدی

در مدل استقلال باینری (BIM)، سیستم برای محاسبه دقیق وزن کلمات به بازخورد کاربر و اطلاعات اسناد مرتبط نیاز دارد (تا مقادیر S و s مشخص شوند). با این حال، در لحظه شروع جستجو (Iteration 0)، سیستم با مشکل «شروع سرد» (Cold Start) مواجه است؛ کاربر به تازگی پرسمان خود را وارد کرده و هنوز هیچ سندی به عنوان مرتبط ارزیابی نشده است. در این شرایط، سیستم ناچار است برای محاسبه رتبه‌بندی اولیه و ارائه سری اول نتایج به کاربر، از فرضیات منطقی و مقادیر پیش‌فرض آماری برای تخمین p_i و u_i استفاده کند.

بخش 2) تحلیل داده‌ها و فرضیات

در لحظه شروع جستجو داده‌های ما بدین شکل است:

تعداد کل اسناد مرتبط شناخته‌شده برابر با 0 است ($S = 0$).

تعداد اسناد مرتبط حاوی کلمه هدف برابر با 0 است ($s = 0$).

اندازه کل مجموعه (N) و فراوانی کلمه در کل مجموعه (df_i) به عنوان داده‌های سراسری نمایه (Index) از قبل در

دسترس سیستم هستند.

بخش 3) حل گام‌به‌گام

گام اول: تعیین مقدار پیش‌فرض برای متغیر p_i

متغیر p_i بیانگر احتمال حضور کلمه i در اسناد مرتبط است. از آنجا که سیستم هیچ دانشی درباره اسناد مرتبط ندارد، از

$$p_i = 0.5$$

گام دوم: تخمین مقدار پیش‌فرض برای متغیر u_i

با تقریب کل مجموعه اسناد معادل اسناد غیرمرتبط ($N - S \approx N$ و $df_i - s \approx df_i$):

$$u_i = \frac{df_i}{N}$$

گام سوم: بررسی اثر این پیش‌فرض‌ها بر فرمول رتبه‌بندی (RSV)

با جایگذاری این مقادیر در فرمول پایه، وزن نهایی به فرم زیر ساده می‌شود:

$$c_i = \log \frac{0.5 \times (1 - \frac{df_i}{N})}{\frac{df_i}{N} \times (1 - 0.5)} = \log \frac{1 - \frac{df_i}{N}}{\frac{df_i}{N}} \approx \log \frac{N}{df_i}$$

نتیجه‌گیری نهایی

هنگامی که جستجو آغاز می‌شود و بازخوردی وجود ندارد، مدل **BIM** به صورت پیش‌فرض مقدار $p_i = 0.5$ را در نظر می‌گیرد و مقدار u_i را برابر با $\frac{df_i}{N}$ تخمین می‌زند.

نکته آموزشی:

فرمول نهایی به دست آمده، یعنی $\log(\frac{N}{df_i})$ ، دقیقاً همان فرمول مشهور **فراوانی معکوس سند (IDF)** است. این اثبات نشان می‌دهد که مفهوم **IDF** که در ابتدا در مدل فضای برداری صرفاً بر اساس شم مهندسی ابداع شده بود، در واقع پشتوانه ریاضی قدرتمندی در مدل‌های احتمالاتی دارد. سیستم پس از محاسبه اولیه، با دریافت کلیک کاربر، مقادیر **S** و **s** را آپدیت کرده و محاسبات را مجدداً تکرار می‌کند.

پاسخنامه مسئله 11.12: مقایسه احتمالات و IDF (Probabilities vs. IDF)

بخش 1) مفاهیم کلیدی

مفهوم «فراوانی معکوس سند» یا **IDF (Inverse Document Frequency)**، در ابتدا توسط کارن اسپارک جونز (**Karen Spärck Jones**) به صورت تجربی و مبتنی بر بینش مهندسی (**Heuristic**) معرفی شد تا به کلماتی که در مجموعه اسناد نادرتر هستند، وزن بیشتری اختصاص دهد. با این حال، یکی از دستاوردهای بزرگ «مدل استقلال باینری» (**BIM**) این است که نشان می‌دهد **IDF** صرفاً یک فرمول ابتکاری نیست، بلکه تحت شرایط اولیه و فرضیات خاص (مانند زمانی که هیچ دانشی از اسناد مرتبط نداریم)، دارای پشتوانه و اثبات دقیق ریاضی در تئوری احتمالات است.

بخش 2) تحلیل داده‌ها و نمادگذاری

برای اثبات این ارتباط، نقطه شروع ما فرمول وزن‌دهی کلمات در مدل **BIM** است که در غیاب هموارسازی به شکل زیر تعریف می‌شود:

$$c_i = \log \frac{p_i(1 - u_i)}{u_i(1 - p_i)}$$

در این مسئله، فرض بنیادین این است که سیستم در نقطه شروع جستجو قرار دارد؛ بنابراین به دلیل عدم قطعیت کامل، احتمال حضور کلمه در اسناد مرتبط برابر با شانس تصادفی در نظر گرفته می‌شود: $p_i = 0.5$.

بخش 3) حل گام به گام

گام اول: جایگذاری فرض عدم قطعیت ($p_i = 0.5$) در فرمول اصلی

مقدار مفروض برای p_i را در فرمول وزن‌دهی جایگذاری می‌کنیم:

$$c_i = \log \frac{0.5(1 - u_i)}{u_i(1 - 0.5)}$$

گام دوم: ساده‌سازی جبری کسر

از آنجا که عبارت $(1 - 0.5)$ در مخرج برابر با 0.5 است، ضریب 0.5 در صورت و مخرج با یکدیگر ساده شده و حذف می‌شوند:

$$c_i = \log \frac{1 - u_i}{u_i}$$

تا این مرحله، وزن کلمه تنها به احتمال رخداد آن در اسناد غیرمرتبط (u_i) وابسته شده است.

گام سوم: جایگذاری تقریب آماری برای u_i

همان‌طور که در مسائل پیشین بررسی شد، در یک پایگاه داده بزرگ، تعداد اسناد مرتبط (**S**) نسبت به کل اسناد (**N**) بسیار ناچیز است. بنابراین، احتمال حضور کلمه در اسناد غیرمرتبط (u_i) با تقریب بسیار بالایی معادل احتمال حضور کلمه در کل مجموعه اسناد در نظر گرفته می‌شود. اگر df_i فراوانی سند برای کلمه **i** باشد:

$$u_i \approx \frac{df_i}{N}$$

این تقریب را در معادله به دست آمده از گام دوم جایگذاری می‌کنیم:

$$c_i = \log \frac{1 - \frac{df_i}{N}}{\frac{df_i}{N}}$$

گام چهارم: مخرج مشترک‌گیری و ساده‌سازی

در صورت کسر، مخرج مشترک **N** را در نظر می‌گیریم:

$$c_i = \log \frac{\frac{N - df_i}{N}}{\frac{df_i}{N}}$$

با ساده کردن مخرج‌های **N** در صورت و مخرج کسر اصلی، به رابطه زیر می‌رسیم:

$$c_i = \log \frac{N - df_i}{df_i}$$

گام پنجم: اعمال فرض نادر بودن کلمات (Term Sparsity)

در زبان طبیعی، اکثر کلمات کلیدی تنها در بخش کوچکی از کل اسناد ظاهر می‌شوند. به عبارت دیگر، فراوانی سند برای

یک کلمه معمولاً بسیار کوچک‌تر از اندازه کل مجموعه است ($df_i \ll N$). بر این اساس، کم کردن مقدار کوچک df_i از

عدد بسیار بزرگ **N** در صورت کسر، تغییر معناداری در مقدار آن ایجاد نمی‌کند و می‌توان از آن صرف‌نظر کرد:

$$N - df_i \approx N$$

با اعمال این تقریب نهایی در معادله، خواهیم داشت:

$$c_i = \log \frac{N}{df_i}$$

نتیجه‌گیری نهایی

از طریق معادلات جبری اثبات شد که در مدل استقلال باینری (**BIM**)، اگر فرض کنیم هیچ اطلاعات اولیه‌ای درباره اسناد مرتبط نداریم ($p_i = 0.5$) و کلمات کلیدی نسبت به کل مجموعه اسناد پراکنده و نادر هستند ($df_i \ll N$)، تابع وزن‌دهی احتمالی دقیقاً به فرمول کلاسیک $IDF = \log(N / df_i)$ همگرا می‌شود. این اثبات، استحکام نظری و ریشه‌های آماری مفهوم **IDF** را در علم بازیابی اطلاعات تایید می‌کند.

پاسخنامه مسئله 11.13: بازخورد شبه‌ارتباط (Pseudo-relevance Feedback)

بخش 1) مفاهیم کلیدی

در مدل‌های احتمالاتی بازیابی اطلاعات مانند **BIM**، دستیابی به بالاترین سطح دقت نیازمند تخمین درست متغیر p_i (احتمال حضور کلمه در اسناد مرتبط) است. محاسبه دقیق این متغیر به صورت سنتی نیازمند «بازخورد صریح» (**Explicit Feedback**) کاربر است تا مشخص کند کدام اسناد واقعاً مرتبط هستند. اما از آنجا که کاربران در دنیای واقعی تمایل چندانی به صرف زمان

برای ارائه بازخورد ندارند، سیستم‌های بازیابی از رویکردی هوشمندانه به نام «بازخورد شبه‌ارتباط» (**Blind Relevance Feedback** یا **Pseudo-relevance Feedback**) استفاده می‌کنند. در این روش، سیستم فرآیند بازخورد را بدون نیاز به دخالت انسان و به صورت کاملاً خودکار شبیه‌سازی می‌کند.

بخش 2) تحلیل رویکرد و الگوریتم

ایده‌ی بنیادین بازخورد شبه-ارتباط بر یک «فرض خوش‌بینانه» استوار است: سیستم فرض می‌کند که الگوریتم رتبه‌بندی اولیه‌اش به قدری خوب کار می‌کند که چند سندِ اولِ بازیابی‌شده، به احتمال بسیار زیاد با نیاز کاربر مرتبط هستند. بر اساس این فرض، الگوریتم طی 4 مرحله متوالی اجرا می‌شود.

بخش 3) مراحل اجرای بازخورد شبه ارتباط

گام اول: جستجوی اولیه (Initial Retrieval)

هنگامی که کاربر پرسمان خود را وارد می‌کند، سیستم هیچ اطلاعاتی از اسناد مرتبط ندارد. بنابراین، همان‌طور که در مباحث پیشین ثابت شد، با در نظر گرفتن فرض عدم قطعیت ($p_i = 0.5$) و استفاده از تقریب‌های **IDF**، یک جستجوی اولیه روی کل پایگاه داده انجام می‌دهد و اسناد را رتبه‌بندی می‌کند.

گام دوم: اعمال فرض شبه‌ارتباط (The Pseudo-relevance Assumption)

سیستم به جای اینکه منتظر کلیک یا تایید کاربر بماند، تعداد **k** سندِ برتر از نتایج اولیه (به عنوان مثال 10 یا 20 سند اول) را جدا می‌کند. سپس به صورت خودکار (و کورکورانه) برچسب «مرتبط» ($R = 1$) را به تمام این **k** سند اختصاص می‌دهد. ما این مجموعه اسنادِ فرضاً مرتبط را مجموعه **V** می‌نامیم (بنابراین کل اسناد مرتبط $S = k$ خواهد بود).

گام سوم: استخراج و به‌روزرسانی پارامترها (Parameter Updating)

اکنون سیستم یک مجموعه محلی از اسناد مرتبط (**V**) در اختیار دارد و می‌تواند از حالت پیش‌فرض حدسی خارج شود. برای هر کلمه **i** در پرسمان، سیستم بررسی می‌کند که این کلمه در چند مورد از این **k** سند برتر تکرار شده است (این مقدار را **s** می‌نامیم). سپس با استفاده از فرمول هموارسازی نیم‌واحدی، مقدار واقعی‌تر و دقیق‌تری برای p_i محاسبه می‌کند:

$$p_i = \frac{s + 0.5}{k + 1}$$

سیستم می‌تواند با استفاده از کلمات جدیدی که در این **k** سند پرکاربرد بوده‌اند، پرسمان را نیز بسط دهد که به آن **Query Expansion** می‌گویند.

گام چهارم: رتبه‌بندی مجدد (Re-ranking)

با به دست آمدن مقادیر جدید و به‌روزرسانی‌شده‌ی p_i (و در صورت نیاز به‌روزرسانی u_i)، وزن تمام کلمات پرسمان در فرمول اصلی **RSV** مجدداً محاسبه می‌شود. سیستم یک بار دیگر اسناد را با این اوزان جدید امتیازدهی کرده، رتبه‌بندی نهایی را شکل می‌دهد و این نتایج اصلاح‌شده را به کاربر نمایش می‌دهد. تمام این 4 گام در کسری از ثانیه و در پس‌زمینه رخ می‌دهد.

نتیجه‌گیری نهایی

بازخورد شبه‌ارتباط به سیستم‌های بازیابی احتمالی اجازه می‌دهد تا با استفاده از نتایج اولیه خود به عنوان یک «مجموعه آموزشی موقت»، پارامتر p_i را از یک مقدار ثابت و بی‌اثر (0.5) به یک مقدار پویا و وابسته به محتوای جستجو تبدیل کنند، بدون اینکه هیچ بار شناختی یا زحمتی به کاربر تحمیل شود.

نکته آموزشی:

اگرچه این روش معمولاً باعث افزایش چشمگیر نرخ فراخوانی (Recall) و دقت سیستم می‌شود، اما دارای یک ریسک تئوریک بزرگ به نام «انحراف پرسمان» (Query Drift) است. اگر پرسمان کاربر بسیار مبهم باشد و سیستم در گام اول (جستجوی اولیه) کاملاً اشتباه کند و اسناد نامرتبلی را در صدر نتایج بیاورد، سیستم در گام دوم آن اسناد نامرتبلی را به عنوان پایه محاسبات p_i قرار می‌دهد. این کار باعث می‌شود در رتبه‌بندی مجدد، سیستم با اطمینان بیشتری به سمت یک موضوع کاملاً اشتباه منحرف شود.

پاسخنامه مسئله 11.14: محدودیت‌های مدل‌های احتمالی

(Limitations of Probabilistic Models)

بخش 1) مفاهیم کلیدی

مدل استقلال باینری (BIM) با وجود پایه‌گذاری یک چارچوب مستحکم ریاضی برای بازیابی اطلاعات، به دلیل فرضیات ساده‌کننده‌اش در بازنمایی متن، دارای نقاط کور قابل‌توجهی است. این ساده‌سازی‌ها اگرچه پیچیدگی محاسباتی را در زمان جستجو به شدت کاهش می‌دهند، اما باعث از دست رفتن بخش مهمی از سیگنال‌های معنایی و ساختاری متن می‌شوند که مستقیماً بر کیفیت و دقت خروجی تأثیر منفی می‌گذارند.

بخش 2) تحلیل داده‌ها و مسئله

برای درک چرایی عدم کارایی مدل پایه BIM در سیستم‌های مدرن و محیط‌های واقعی، 2 مورد از بنیادی‌ترین ضعف‌های این مدل را که ریشه در ذات «باینری» بودن آن دارند، کالبدشکافی کرده و اثرات مخرب آن‌ها را بر تجربه جستجوی کاربر بررسی می‌کنیم.

بخش 3) حل گام‌به‌گام

گام اول: تحلیل محدودیت اول - عدم استفاده از بسامد کلمه درون سند (Term Frequency - TF)

ماهیت ضعف: در مدل BIM، بازنمایی اسناد صرفاً به صورت متغیرهای دودویی (حضور با مقدار 1 یا عدم حضور با مقدار 0) انجام می‌شود. این مدل هیچ مکانیزمی برای شمارش دفعات تکرار یک کلمه در متن ندارد. **اثر بر کیفیت جستجو:** در دنیای واقعی، میزان تکرار یک کلمه، شاخصی قدرتمند از میزان تمرکز سند بر روی آن موضوع است. فرض کنید کاربری عبارت «درخت جستجوی دودویی» را جستجو می‌کند. سند اول یک فصل از کتاب ساختمان داده است که این عبارت 100 بار در آن تکرار شده است، در حالی که سند دوم یک مقاله عمومی است که تنها در چکیده خود 1 بار به این عبارت اشاره کرده است. مدل BIM به دلیل ماهیت صفر و یکی خود، به هر 2 سند امتیاز برابری برای این کلمه اختصاص می‌دهد و نمی‌تواند سند تخصصی‌تر را تشخیص دهد. این ضعف باعث کاهش شدید دقت (Precision) در نتایج برتر می‌شود، زیرا اسناد حاشیه‌ای هم‌رده با اسناد مرجع و اصلی قرار می‌گیرند.

گام دوم: تحلیل محدودیت دوم - نادیده گرفتن طول سند (Document Length Normalization)

ماهیت ضعف: مدل **BIM** به طول کلی سند (تعداد کل کلمات تشکیل دهنده آن) کاملاً بی تفاوت است. یک پاراگراف کوتاه 30 کلمه‌ای و یک گزارش مفصل 30000 کلمه‌ای، در صورتی که کلمات پرسمان را داشته باشند، در سیستم یکسان ارزیابی می‌شوند.

اثر بر کیفیت جستجو: اسناد طولانی‌تر به طور طبیعی (و صرفاً به دلیل حجم بالای کلمات) احتمال بیشتری دارند که شامل دایره واژگان وسیع‌تری باشند. در نتیجه، در مدل **BIM** شانس اینکه یک سند طولانی به طور تصادفی با کلمات پرسمان مطابقت پیدا کند بسیار بالاست. این سوگیری ساختاری باعث می‌شود موتور جستجو نتایج را به نفع اسناد طولانی (که ممکن است شامل موضوعات پراکنده و نامرتب باشند) مصادره کند و اسناد کوتاه، دقیق و بسیار مرتبط (مانند پرسش و پاسخ‌های متمرکز) را در رتبه‌های پایین‌تر قرار دهد. این امر موجب افت شدید کیفیت رتبه‌بندی می‌گردد.

نتیجه‌گیری نهایی

محدودیت‌های بنیادین مدل **BIM** در نادیده گرفتن بسامد کلمات و بی‌توجهی به طول اسناد، باعث می‌شوند سیستم توانایی تمایز قائل شدن بین اسناد عمیقاً مرتبط و اسنادی که صرفاً اشاره‌ای گذرا یا تصادفی به موضوع داشته‌اند را از دست بدهد.

نکته آموزشی:

دقیقاً همین 2 نقطه ضعف بزرگ، انگیزه اصلی پژوهشگران برای توسعه مدل‌های پیشرفته‌تر گردید. با ادغام منطق بسامد کلمه (**TF**) برای برطرف کردن محدودیت اول، و معرفی ضریب «نرمال‌سازی طول سند» (**Document Length Normalization**) برای جبران سوگیری در محدودیت دوم، سیستم‌های رتبه‌بندی جهش بزرگی را تجربه کردند. خروجی نهایی این اصلاحات ریاضی، تولد الگوریتم قدرتمند **Okapi BM25** بود که امروزه به عنوان هسته استاندارد در موتورهای جستجوی متن‌باز نظیر **Elasticsearch** و **Lucene** استفاده می‌شود.

پاسخنامه مسئله 11.15: تفاوت بازخورد احتمالی و برداری

(Probabilistic vs. Vector Space Feedback)

بخش 1) مفاهیم کلیدی

مفهوم «بازخورد ارتباط» (**Relevance Feedback**) در سیستم‌های بازایی اطلاعات، به معنای استفاده از بازخورد کاربر (یا بازخورد شبه‌ارتباط) برای اصلاح و بهبود پرسمان اولیه است. اگرچه هدف نهایی در هر 2 مدل «احتمالاتی» (مانند **BIM**) و «فضای برداری» (مانند **VSM**) یکسان است (یعنی نزدیک کردن نتایج به نیاز واقعی کاربر)، اما این 2 مدل از نظر پارادایم ریاضی و نحوه اعمال این بازخورد تفاوت‌های بنیادینی با یکدیگر دارند. یکی بر پایه آمار و توزیع احتمالات بنا شده و دیگری بر پایه هندسه و فاصله‌یابی در فضای چندبعدی.

بخش 2) تحلیل مسئله و متغیرها

برای درک این تفاوت، باید رفتار ریاضی هر مدل را هنگام دریافت اسناد مرتبط (D_r) و اسناد غیرمرتبط (D_{nr}) بررسی کنیم:

در مدل احتمالی، متغیر هدف ما p_i (احتمال رخداد کلمه در اسناد مرتبط) است.

در مدل فضای برداری، ابزار اصلی ما الگوریتم **Rocchio** است و متغیر هدف، بردار پرسمان اصلاح‌شده ($\vec{q}_m$) می‌باشد.

گام اول: مکانیزم بازخورد در مدل احتمالی (تغییر باور آماری)

در مدل استقلال باینری (BIM)، پرسمان به عنوان یک موجودیت یکپارچه حرکت نمی‌کند. در عوض، سیستم به صورت کلمه به کلمه عمل می‌کند. هنگامی که اسناد مرتبط مشخص می‌شوند، سیستم صرفاً توزیع آماری کلمات را بازنگری می‌کند.

$$p_i = \frac{s + 0.5}{S + 1}$$

ماهیت عملکرد: این روش یک «به‌روزرسانی بیزی» (Bayesian Update) است. سیستم پیش از بازخورد، باور داشت که اهمیت کلمه تصادفی است ($p_i = 0.5$). پس از بازخورد، با شمارش کلمات در اسناد مرتبط، وزن (Weight) آن کلمه را در فرمول RSV تغییر می‌دهد. در حالت پایه، کلمات جدیدی به پرسمان اضافه نمی‌شوند، بلکه فقط اهمیت کلمات موجود بازتوزیع می‌شود.

گام دوم: مکانیزم بازخورد در مدل فضای برداری (انتقال هندسی)

در مدل VSM، اسناد و پرسمان به صورت بردارهایی در یک فضای n-بعدی (که n اندازه لغت‌نامه است) نشان داده می‌شوند. بازخورد ارتباط با استفاده از الگوریتم Rocchio اعمال می‌شود:

$$\vec{q}_m = \alpha \vec{q}_0 + \beta \frac{1}{|D_r|} \sum_{\vec{d}_j \in D_r} \vec{d}_j - \gamma \frac{1}{|D_{nr}|} \sum_{\vec{d}_k \in D_{nr}} \vec{d}_k$$

ماهیت عملکرد: این روش یک «انتقال فضایی» (Spatial Translation) است. بردار پرسمان اولیه ($\vec{q}_0$) به صورت فیزیکی در فضای مختصات حرکت می‌کند؛ به سمت مرکز ثقل (Centroid) اسناد مرتبط کشیده می‌شود و از مرکز ثقل اسناد غیرمرتبط دور می‌گردد.

گام سوم: تبیین تفاوت‌های بنیادی

از مقایسه این 2 مکانیزم، 3 تفاوت اساسی استخراج می‌شود:

1. **تفاوت در ماهیت ریاضی:** بازخورد احتمالی مبتنی بر آمار و شمارش گسسته (تغییر احتمالات p_i) است، در حالی که بازخورد VSM مبتنی بر جبر خطی و هندسه پیوسته (تغییر زاویه و طول بردار) است.

2. **تفاوت در بسط پرسمان (Query Expansion):** در الگوریتم Rocchio، جمع کردن بردارهای اسناد مرتبط به صورت خودکار باعث می‌شود کلمات جدیدی که در پرسمان اولیه نبوده‌اند، وزن غیرصفر بگیرند (بسط خودکار پرسمان). اما در مدل احتمالی پایه، بازخورد تنها وزن کلمات فعلی را اصلاح می‌کند و برای اضافه کردن کلمات جدید به پرسمان، به یک مرحله مجزا و صریح برای انتخاب کلمات (Term Selection) نیاز است.

3. **تفاوت در اثر اسناد منفی:** در الگوریتم Rocchio، اسناد غیرمرتبط مستقیماً (با ضریب γ) بردار پرسمان را از خود دور می‌کنند که می‌تواند باعث منفی شدن وزن برخی کلمات شود. اما در مدل احتمالی، اسناد غیرمرتبط صرفاً با افزایش مقدار u_i ، وزن مثبت یک کلمه را کاهش می‌دهند و مفهوم «وزن منفی» در آن به شکل هندسی VSM وجود ندارد.

تفاوت بنیادی این 2 رویکرد در این است که مدل احتمالی، بازخورد را به عنوان شواهدی برای «یادگیری و اصلاح توزیع احتمالات کلمات» می‌بیند تا فرمول امتیازدهی را دقیق‌تر کند، در حالی که الگوریتم **Rocchio** در **VSM**، بازخورد را به عنوان نیرویی برای «هدایت فیزیکی بردار پرسمان» به سمت مناطق غنی‌تر و مرتبط‌تر در فضای چندبعدی تفسیر می‌کند.

نکته آموزشی:

از منظر مهندسی سیستم، الگوریتم **Rocchio** به دلیل جمع کردن بردارهای طولانی اسناد، پرسمان اصلاح‌شده را به یک «بردار چگال» (**Dense Vector**) با کلمات بسیار زیاد تبدیل می‌کند که زمان پردازش جستجو را به شدت افزایش می‌دهد. به همین دلیل، در سیستم‌های واقعی **VSM**، پس از محاسبه بردار جدید $\vec{q}_m$ ، معمولاً فقط 10 الی 20 کلمه‌ای که بالاترین وزن را دارند نگه داشته می‌شوند (**Truncation**) تا جستجو همچنان سریع و کارآمد باقی بماند.

پاسخنامه مسئله 11.16: وابستگی کلمات در پرسمان (Term Dependencies in Queries)

بخش 1) مفاهیم کلیدی

مدل استقلال باینری (**BIM**) بر پایه فرض «استقلال شرطی کلمات» بنا شده است. این فرض تئوریک بیان می‌کند که با مشخص بودن وضعیت مرتبط بودن یا نبودن یک سند، احتمال حضور یک کلمه در آن سند هیچ ارتباطی به حضور یا عدم حضور کلمات دیگر ندارد. در زبان طبیعی، این فرض به ندرت صادق است؛ زیرا زبان مملو از کلمات مرکب، عبارات اسمی (**Named Entities**) و هم‌رخدای‌های معنادار (**Collocations**) است که در آن‌ها کلمات به شدت به یکدیگر وابسته‌اند.

بخش 2) تحلیل داده‌ها و مسئله

پرسمان **"New York"** را در نظر بگیرید. این پرسمان از 2 کلمه مجزا تشکیل شده است: $t_1 = \text{"New"}$ و $t_2 = \text{"York"}$. هدف کاربر یافتن اسنادی درباره شهر یا ایالت «نیویورک» است. اکنون بررسی می‌کنیم که چرا پردازش این پرسمان به ظاهر ساده، در ساختار مدل **BIM** منجر به فروپاشی منطق رتبه‌بندی می‌شود.

بخش 3) حل گام‌به‌گام

گام اول: رویکرد پردازشی مدل BIM (نگاه مستقل)

از نگاه ریاضیات **BIM**، احتمال مشاهده همزمان این 2 کلمه در یک سند مرتبط، برابر با حاصل ضرب احتمال مشاهده مستقل آن‌هاست:

$$P(t_1, t_2 | R = 1) = P(t_1 | R = 1) \times P(t_2 | R = 1)$$

بر همین اساس، در محاسبه نمره نهایی سند (**RSV**)، سیستم صرفاً وزن مستقل هر کلمه را محاسبه کرده و با هم جمع می‌کند:

$$RSV_d = c_1 + c_2$$

گام دوم: نقض واقعیت آماری (بروز خطای محاسباتی)

در دنیای واقعی و در متون انگلیسی، این 2 کلمه مستقل نیستند. اگر کلمه "**York**" در متنی درباره جغرافیا یا اخبار آمریکا ظاهر شود، با احتمال بسیار بالایی کلمه "**New**" نیز بلافاصله قبل از آن وجود دارد. به عبارت دیگر، احتمال شرطی ("**York**" | "**New**") بسیار بالا و نزدیک به 1 است.

مدل **BIM** با نادیده گرفتن این وابستگی آماری شدید، دچار خطای «شمارش مضاعف شواهد» (**Double Counting of Evidence**) می‌شود. سیستم گمان می‌کند 2 شاهد کاملاً مستقل و مجزا برای مرتبط بودن سند پیدا کرده است، در حالی که در واقعیت، این 2 کلمه تنها یک مفهوم واحد (یک شهر) را نمایندگی می‌کنند.

گام سوم: آسیب‌های وارد شده به کیفیت رتبه‌بندی

این خطای محاسباتی، 2 آسیب جدی به خروجی موتور جستجو وارد می‌کند:

- 1. ناتوانی در تشخیص ساختار عبارات (Loss of Phrase Structure):** چون مدل **BIM** کلمات را مستقل فرض می‌کند، موقعیت و ترتیب کلمات در سند را از دست می‌دهد (تبدیل شدن به یک کیسه کلمات یا **Bag of Words**). به عنوان مثال، سندی با محتوای: "دانشگاه **York** یک روش **New** برای تدریس ارائه کرد"، دقیقاً همان نمره‌ای را دریافت می‌کند که سندی با محتوای "شهر **New York** دارای آسمان‌خراش است" دریافت می‌کند. سیستم نمی‌تواند بین کلمات پراکنده و یک عبارت متصل تمایز قائل شود.
- 2. تولید مثبت کاذب (False Positives) و افت دقت:** با جمع شدن وزن کلمات به صورت مستقل، اسنادی که به طور تصادفی شامل کلمه "**New**" (که کلمه بسیار پرکاربردی است) و کلمه "**York**" (در سیاقی دیگر) هستند، نمره (**RSV**) بالایی دریافت کرده و به اشتباه در صدر نتایج (رتبه‌های بالا) قرار می‌گیرند. این امر باعث می‌شود کاربر با حجم زیادی از اسناد نامرتب مواجه شود.

نتیجه‌گیری نهایی

مدل **BIM** در پردازش پرسمان‌هایی نظیر "**New York**" دچار خطای محاسباتی می‌شود زیرا فرض استقلال، وابستگی آماری و معنایی شدید میان اجزای یک عبارت مرکب را نادیده می‌گیرد. این امر منجر به شمارش مضاعف شواهد، از دست رفتن ساختار عبارات و در نهایت نمایش اسناد نامرتب در رتبه‌های برتر (افت **Precision**) می‌گردد.

نکته آموزشی:

برای غلبه بر این محدودیت بزرگ، سیستم‌های بازیابی اطلاعات مدرن از معماری‌های پیشرفته‌تری استفاده می‌کنند. یکی از راهکارهای رایج، استفاده از «نمایه‌های موقعیتی» (**Positional Indexes**) و پشتیبانی از «پرسمان‌های عبارتی» (**Phrase Queries**) است (مثلاً قرار دادن پرسمان در داخل کوتیشن: "**New York**"). راهکار پیچیده‌تر، استفاده از مدل‌های زبانی مبتنی بر **N-gram** (مانند **Bigram** ها) یا مدل‌های وابستگی کلمات (مانند **Markov Random Fields** در بازیابی اطلاعات) است که در آن‌ها، احتمال رخداد یک کلمه به کلمه قبل از خود وابسته (**Conditioned**) می‌شود.

پاسخنامه مسئله 11.17: رتبه‌بندی با لحاظ هزینه در سیستم پزشکی

(Cost-Aware Ranking in Medical Systems)

بخش 1) مفاهیم کلیدی

در سیستم‌های اطلاعاتی حساس (مانند پزشکی یا حقوقی)، اصل رتبه‌بندی احتمالی (PRP) با در نظر گرفتن «تابع زیان» (Loss Function) تعمیم می‌یابد. در این حالت، تصمیم‌گیری برای نمایش یا عدم نمایش یک سند، صرفاً بر اساس مرتب‌سازی احتمالات نیست؛ بلکه سیستم بر اساس تئوری تصمیم بیز (Bayesian Decision Theory)، «ارزش مورد انتظار» (Expected Value) خطاها را محاسبه می‌کند. سیستم تنها زمانی یک سند را به کاربر نشان می‌دهد که هزینه مورد انتظار ناشی از بازیابی آن، کمتر یا مساوی با هزینه مورد انتظار ناشی از عدم بازیابی آن باشد.

بخش 2) تحلیل داده‌ها و اصلاح فرضیات

پیش از حل مسئله، نیاز است تا یک تناقض در صورت سوال را از منظر ریاضی اصلاح کنیم. در متن اشاره شده است که «هزینه از دست دادن یک سند مرتبط (منفی کاذب) 10 برابر هزینه نشان دادن سند نامرتبط (مثبت کاذب) است». همان‌طور که در مسئله 11.4 تعریف کردیم:

$$C_{01} = \text{منفی کاذب}$$

$$C_{10} = \text{مثبت کاذب}$$

بنابراین، ترجمه دقیق ریاضی متن صورت مسئله به شکل $C_{01} = 10 \times C_{10}$ است. رابطه داخل پرانتز در صورت سوال ($C_{10} = 10 \times C_{01}$) یک خطای تایپی رایج است که مفهوم را معکوس می‌کند. در این پاسخنامه، بر اساس منطق و حساسیت سیستم پزشکی (اولویت جان بیمار بر اتلاف وقت پزشک)، ما محاسبات را بر پایه فرمول صحیح یعنی $C_{01} = 10 \times C_{10}$ پیش خواهیم برد. همچنین فرض می‌کنیم تصمیمات درست (بازیابی سند مرتبط و عدم بازیابی سند نامرتبط) هیچ هزینه‌ای ندارند ($C_{11} = 0$ و $C_{00} = 0$).

بخش 3) حل گام‌به‌گام

گام اول: فرمول‌بندی هزینه‌های مورد انتظار

برای هر سند (d)، سیستم 2 گزینه پیش رو دارد و باید هزینه مورد انتظار (Expected Cost) هر کدام را محاسبه کند. برای سادگی، احتمال مرتبط بودن سند را با $p = P(R=1|d)$ و احتمال نامرتبط بودن آن را با $1 - p = P(R=0|d)$ نشان می‌دهیم.

1. هزینه مورد انتظار در صورت بازیابی (Retrieval): اگر سیستم سند را نمایش دهد، تنها در صورتی جریمه می‌شود که سند نامرتبط باشد (مثبت کاذب).

$$E[Cost|Retrieval] = (1 - p) \times C_{10}$$

2. هزینه مورد انتظار در صورت عدم بازیابی (Non-Retrieval): اگر سیستم سند را فیلتر کند، تنها در صورتی جریمه می‌شود که سند مرتبط بوده باشد (منفی کاذب).

$$E[Cost|Non-Retrieval] = p \times C_{01}$$

گام دوم: اعمال شرط تصمیم‌گیری PRP

طبق اصل PRP، سیستم تنها زمانی سند را بازیابی می‌کند که هزینه نمایش آن، از هزینه پنهان کردنش کمتر (یا مساوی) باشد:

$$E[Cost|Retrieval] \leq E[Cost|Non-Retrieval]$$

با جایگذاری فرمول‌های گام اول در این نامعادله خواهیم داشت:

$$(1 - p) \times C_{10} \leq p \times C_{01}$$

گام سوم: حل نامعادله و یافتن آستانه احتمال

اکنون رابطه $C_{01} = 10 \times C_{10}$ را در نامعادله جایگذاری می‌کنیم:

$$(1 - p) \times C_{10} \leq p \times (10 \times C_{10})$$

چون C_{10} یک عدد مثبت است، می‌توانیم طرفین را بر آن تقسیم کنیم تا ساده شود:

$$1 - p \leq 10p$$

متغیر p را به سمت راست نامعادله منتقل می‌کنیم:

$$1 \leq 11p$$

در نهایت، با تقسیم طرفین بر 11، آستانه احتمال (p) به دست می‌آید:

$$p \geq \frac{1}{11}$$

نتیجه‌گیری نهایی

برای اینکه یک سند در این سیستم پزشکی به پزشک نمایش داده شود، احتمال مرتبط بودن آن ($P(R=1|d)$) باید بزرگتر یا مساوی $1/11$ (یعنی حدود 0.09 یا 9 درصد) باشد.

نکته آموزشی:

این نتیجه از نظر مهندسی سیستم‌های حساس بسیار منطقی و آموزنده است. در یک موتور جستجوی وب معمولی، سیستم معمولاً اسنادی را نشان می‌دهد که احتمال ارتباط آن‌ها بالای 50 درصد باشد. اما در یک سیستم پزشکی، چون هزینه از دست دادن اطلاعات حیاتی (منفی کاذب) به شدت بالاست، سیستم بسیار محتاطانه عمل می‌کند و آستانه تصمیم‌گیری (Threshold) را تا 9 درصد پایین می‌آورد. این یعنی سیستم به پزشک می‌گوید: "من تنها 9 درصد مطمئنم که این پرونده به بیمار شما مرتبط است، اما

چون نخواندن آن ممکن است خطرناک باشد، آن را به شما نمایش می‌دهم تا خودتان بررسی کنید." این استراتژی به شدت به نفع افزایش نرخ فراخوانی (Recall) عمل می‌کند.

پاسخنامه مسئله 11.18: سناریوی تکامل وزن‌دهی کلمات با بازخورد (Term Weight Evolution Scenario with Feedback)

بخش 1) مفاهیم کلیدی

این مسئله، یک جمع‌بندی عملی از مباحث پیشین است و نشان می‌دهد که چگونه «بازخورد ارتباط» (Relevance Feedback) به صورت ریاضی در مدل استقلال باینری (BIM) اعمال می‌شود. در اینجا مشاهده می‌کنیم که وزن یک کلمه (RSV) یک مقدار ثابت نیست، بلکه با دریافت اطلاعات جدید از سمت کاربر، از طریق یک فرایند «به‌روزرسانی بیزی» تکامل یافته و اصلاح می‌شود.

بخش 2) تحلیل داده‌ها و نمادگذاری

ابتدا مقادیر داده شده در صورت مسئله را با نمادهای استاندارد مدل سازی می‌کنیم:

اندازه کل مجموعه اسناد پایگاه داده: $N = 1000$

فراوانی سند برای کلمه هدف: $df_1 = 100$

تعداد کل اسناد مرتبط شناخته‌شده پس از بازخورد: $S = 3$

تعداد اسناد مرتبط حاوی کلمه هدف: $s = 2$

(نکته: تعداد 5 سندی که در دور اول برگردانده شده‌اند، در فرمول‌های پایه BIM تاثیری ندارند، زیرا در این مدل تنها اسناد مرتبط (تایید شده توسط کاربر) و داده‌های سراسری مجموعه برای تخمین احتمالات اهمیت دارند).

بخش 3) حل گام‌به‌گام

الف) محاسبه وزن کلمه t_1 قبل از بازخورد (Iteration 0)

پیش از دریافت بازخورد کاربر، سیستم هیچ دانشی درباره اسناد مرتبط ندارد ($S = 0$ و $s = 0$). همان‌طور که در مسئله‌های پیشین اثبات شد، در این حالت از فرض عدم قطعیت ($p_1 = 0.5$) و تقریب سراسری ($u_1 = df_1 / N$) استفاده می‌شود.

گام اول: جایگذاری مقادیر پیش‌فرض در فرمول وزن‌دهی

فرمول پایه وزن کلمه به این شکل ساده می‌شود:

$$c_1 = \log \frac{1 - u_1}{u_1}$$

گام دوم: محاسبه مقدار u_1 و وزن نهایی

مقدار u_1 برابر است با $0.1 = 100 / 1000$. این مقدار را در فرمول قرار می‌دهیم:

$$c_1 = \log \frac{1 - 0.1}{0.1} = \log \frac{0.9}{0.1} = \log(9)$$

وزن کلمه **t₁** قبل از بازخورد برابر با $\log(9)$ است.

ب) محاسبه وزن کلمه **t₁** پس از دریافت بازخورد

اکنون کاربر بازخورد داده است و مقادیر $S = 3$ و $s = 2$ در دسترس هستند. برای محاسبه دقیق و جلوگیری از خطای احتمالات 0، از تکنیک «هموارسازی نیم‌واحدی» (**Robertson-Sparck Jones**) استفاده می‌کنیم.

گام اول: به‌روزرسانی احتمال p_1 (حضور در اسناد مرتبط)

$$p_1 = \frac{s + 0.5}{S + 1} = \frac{2 + 0.5}{3 + 1} = \frac{2.5}{4} = 0.625$$

(احتمال عدم حضور در اسناد مرتبط: $1 - p_1 = 0.375$)

گام دوم: به‌روزرسانی احتمال u_1 (حضور در اسناد غیرمرتبط)

$$u_1 = \frac{df_1 - s + 0.5}{N - S + 1} = \frac{100 - 2 + 0.5}{1000 - 3 + 1} = \frac{98.5}{998} \approx 0.0987$$

(احتمال عدم حضور در اسناد غیرمرتبط: $1 - u_1 \approx 0.9013$)

گام سوم: محاسبه وزن جدید (RSV جدید)

اکنون احتمالات به‌روزرسانی‌شده را در فرمول اصلی قرار می‌دهیم:

$$c_1 = \log \frac{p_1(1 - u_1)}{u_1(1 - p_1)} = \log \frac{0.625 \times 0.9013}{0.0987 \times 0.375} = \log \frac{0.5633}{0.0370} \approx \log(15.22)$$

وزن کلمه **t₁** پس از بازخورد برابر با $\log(15.22)$ است.

نتیجه‌گیری نهایی و تحلیل تغییرات

قبل از بازخورد: نسبت شانس (آرگومان درون لگاریتم) برابر با 9 بود.

پس از بازخورد: نسبت شانس به 15.22 افزایش یافت.

نکته آموزشی:

افزایش وزن کلمه از $\log(9)$ به $\log(15.22)$ از نظر آماری کاملاً معنادار است. پیش از جستجو، سیستم تنها می‌دانست که این کلمه در 10 درصد از کل اسناد پایگاه داده (100 از 1000) وجود دارد. اما پس از بازخورد، سیستم متوجه شد که این کلمه در حدود 66 درصد از اسناد مرتبط کاربر (2 از 3) حضور دارد. چون تمرکز این کلمه در اسناد مرتبط بسیار بیشتر از توزیع عمومی آن در کل پایگاه

داده است، سیستم نتیجه می‌گیرد که کلمه t_1 یک «کلمه متمایزکننده» (**Discriminative Term**) بسیار عالی برای نیاز این کاربر است؛ بنابراین وزن آن را در رتبه‌بندی افزایش می‌دهد تا در جستجوی دور دوم، اسنادی که حاوی این کلمه هستند شانس بیشتری برای نمایش در رتبه‌های بالا داشته باشند.

پاسخنامه مسئله 11.19: مدل‌سازی وابستگی درختی عبارات

(Modeling Tree-Structured Dependencies)

بخش 1) مفاهیم کلیدی

همان‌طور که در مسائل پیشین بررسی شد، «فرض استقلال کلمات» در مدل استقلال باینری (**BIM**) یک نقطه ضعف بنیادین است. برای غلبه بر این مشکل، پژوهشگران (از جمله ون رایزبرگن) «مدل وابستگی درختی» (**Tree-Structured Dependencies**) را پیشنهاد دادند. در این مدل، به جای استقلال کامل کلمات یا وابستگی کامل تمام کلمات به یکدیگر (که از نظر محاسباتی غیرممکن است)، یک حد وسط در نظر گرفته می‌شود: هر کلمه می‌تواند حداکثر به 1 کلمه‌ی دیگر وابسته باشد. این ساختار، 1 گراف بدون دور یا «درخت» (**Tree**) تشکیل می‌دهد که در آن گره‌ها همان کلمات و یال‌ها نشان‌دهنده وابستگی آماری هستند.

بخش 2) تحلیل داده‌ها و نمادگذاری

پرسمان کاربر دارای 3 کلمه است. آن‌ها را با نمادهای متغیر تصادفی نام‌گذاری می‌کنیم:

t_1 : نمایانگر کلمه "Machine"

t_2 : نمایانگر کلمه "Learning"

t_3 : نمایانگر کلمه "Python"

طبق صورت مسئله، روابط وابستگی به این شکل تعریف شده‌اند:

1. وابستگی شدید میان t_1 و t_2 (**Machine Learning** یک عبارت متصل است).

2. وابستگی میان t_2 و t_3 (**Python** معمولاً در کنار **Learning** و در بستر یادگیری ماشین معنا پیدا می‌کند).

این ساختار به شکل یک زنجیره یا درخت خطی ($t_1 \rightarrow t_2 \rightarrow t_3$) مدل‌سازی می‌شود. هدف، محاسبه احتمال توأم (**Joint Probability**) حضور این 3 کلمه در یک سند مرتبط، یعنی $P(t_1, t_2, t_3 | R=1)$ است.

بخش 3) حل گام‌به‌گام

گام اول: بیان رابطه توأم بر اساس قانون زنجیره‌ای (بدون فرض درختی)

در تئوری احتمالات، قانون زنجیره‌ای (**Chain Rule**) برای هر 3 رویداد به صورت دقیق و بدون هیچ فرضی به شکل زیر بسط داده می‌شود (با فرض شرطی بودن روی $R=1$):

$$P(t_1, t_2, t_3 | R = 1) = P(t_1 | R = 1) \times P(t_2 | t_1, R = 1) \times P(t_3 | t_1, t_2, R = 1)$$

این فرمول کامل است، اما محاسبه جمله آخر (که کلمه سوم به هر 2 کلمه قبلی وابسته است) در پایگاه داده‌های بزرگ بسیار پیچیده و پرهزینه است.

گام دوم: اعمال فرض مدل‌سازی درختی (ویژگی مارکوف)

مدل وابستگی درختی بیان می‌کند که هر گره تنها به گره «پدر» (والد) خود در درخت متصل است و از سایر گره‌های پیشین مستقل می‌باشد. با در نظر گرفتن ساختار خطی مسئله $(t_1 \rightarrow t_2 \rightarrow t_3)$ ، کلمه t_3 تنها به کلمه t_2 وابسته است و حضور کلمه t_1 اطلاعات اضافه‌ای درباره t_3 به ما نمی‌دهد. به بیان ریاضی:

$$P(t_3|t_1, t_2, R=1) \approx P(t_3|t_2, R=1)$$

گام سوم: جایگذاری در معادله و فرمول‌بندی نهایی

اکنون تقریب به دست آمده در گام دوم را در قانون زنجیره‌ای اصلی (گام اول) جایگذاری می‌کنیم تا فرمول نهایی مدل درختی به دست آید:

$$P(t_1, t_2, t_3|R=1) \approx P(t_1|R=1) \times P(t_2|t_1, R=1) \times P(t_3|t_2, R=1)$$

(نکته: در گراف‌های درختی بدون جهت، انتخاب ریشه اختیاری است. اگر t_2 را به عنوان ریشه درخت انتخاب می‌کردیم، فرمول به شکل کاملاً معادل $P(t_2|R=1) \times P(t_1|t_2, R=1) \times P(t_3|t_2, R=1)$ نوشته می‌شد).

نتیجه‌گیری نهایی

با استفاده از تقریب درختی، احتمال مشترک حضور این 3 کلمه در یک سند مرتبط به جای حاصل ضرب 3 احتمال مستقل، به صورت حاصل ضرب احتمالات شرطی جفتی مدل‌سازی می‌شود:

$$P(t_1, t_2, t_3|R=1) \approx P(t_1|R=1) \times P(t_2|t_1, R=1) \times P(t_3|t_2, R=1)$$

نکته آموزشی:

این روش از نظر ریاضی بسیار ظریف است. در این مدل، سیستم متوجه می‌شود که اگر کلمه "Machine" در سند وجود داشته باشد، احتمال ظاهر شدن کلمه "Learning" بسیار بیشتر از حالت عادی است ($P(t_2|t_1)$ بزرگ است). در پیاده‌سازی‌های واقعی بازیابی اطلاعات، برای استخراج خودکار این درخت‌های وابستگی از داخل پایگاه داده، از ابزارهایی مانند «الگوریتم چو-لیو» (Chow-Liu Algorithm) استفاده می‌شود که با محاسبه «اطلاعات متقابل» (Mutual Information) بین تمام جفت‌کلمات، گراف وابستگی را به شکل 1 درخت پوشای کمینه (MST) رسم می‌کند.

پاسخنامه مسئله 11.20: الگوریتم ساخت درخت چو-لیو (Chow-Liu Tree Algorithm)

بخش 1) مفاهیم کلیدی

در مدل‌سازی وابستگی کلمات، سیستم نمی‌تواند به صورت تصادفی یا دستی مشخص کند که کدام کلمات به یکدیگر وابسته‌اند. برای کشف خودکار این روابط از درون داده‌ها، به یک ابزار ریاضی نیاز است که میزان هم‌بستگی آماری جفت‌کلمات را اندازه‌گیری کند. الگوریتم «چو-لیو» (**Chow-Liu Algorithm**) یک روش بنیادین در نظریه اطلاعات (**Information Theory**) است که بهترین و دقیق‌ترین تقریب درختی (**Tree Approximation**) را برای یک توزیع احتمال توأم پیچیده و چندمتغیره پیدا می‌کند. این الگوریتم تضمین می‌کند که درخت حاصل، کمترین میزان خطا (پراکندگی کولبک-لیبلر) را نسبت به توزیع واقعی کلمات در اسناد داشته باشد.

بخش 2) تحلیل مسئله و مدل‌سازی گراف

برای حل این مسئله، سیستم مجموعه کلمات پرسمان را به عنوان گره‌ها (**Vertices**) در یک گراف در نظر می‌گیرد. هدف نهایی این است که از میان تمام حالت‌های ممکن برای اتصال این گره‌ها، درختی استخراج شود که یال‌های آن (**Edges**)، قوی‌ترین وابستگی‌های آماری را نمایندگی کنند.

بخش 3) حل گام‌به‌گام

گام اول: تعریف معیار وزن‌دهی یال‌ها (Mutual Information)

سیستم برای اینکه بفهمد 2 کلمه (مثلاً t_i و t_j) چقدر به هم وابسته‌اند، از معیار «اطلاعات متقابل» (**MI یا Mutual Information**) استفاده می‌کند. این معیار نشان می‌دهد که با دانستن وضعیت حضور یک کلمه، چقدر از عدم قطعیت ما درباره کلمه دیگر کاسته می‌شود. برای هر جفت کلمه، سیستم مقدار **MI** را بر اساس توزیع احتمالات آن‌ها در مجموعه اسناد محاسبه کرده و آن را به عنوان «وزن یال» (W_{ij}) در نظر می‌گیرد:

$$W_{ij} = I(t_i, t_j) = \sum_{x_i \in \{0,1\}} \sum_{x_j \in \{0,1\}} P(x_i, x_j) \log \frac{P(x_i, x_j)}{P(x_i)P(x_j)}$$

دقت کنید در این فرمول، x_i و x_j نشان‌دهنده حضور (1) یا عدم حضور (0) کلمات در اسناد هستند. اگر 2 کلمه کاملاً مستقل باشند، صورت و مخرج کسر درون لگاریتم برابر شده و وزن یال 0 می‌شود.

گام دوم: تشکیل گراف کامل (Complete Graph)

سیستم وزن اطلاعات متقابل را برای تمام جفت‌کلمات ممکن در پرسمان محاسبه می‌کند. اگر پرسمان دارای n کلمه باشد، یک «گراف کامل بدون جهت» با n گره و $n(n-1)/2$ یال تشکیل می‌شود که روی هر یال، مقدار محاسبه‌شده در گام اول به عنوان وزن قرار گرفته است.

گام سوم: استخراج درخت پوشای بیشینه (Maximum Spanning Tree)

گراف کامل شامل حلقه‌ها (**Cycles**) است و یک ساختار درختی نیست. برای تبدیل آن به یک مدل تقریب درختی (طبق فرمول‌بندی‌های مسئله پیشین)، سیستم باید زیرگرافی را انتخاب کند که:

1. تمام گره‌ها را شامل شود (**Spanning**).

2. هیچ حلقه‌ای نداشته باشد (**Tree**).

3. مجموع وزن یال‌های آن بیشترین مقدار ممکن باشد (**Maximum**).

برای این کار، سیستم از الگوریتم‌های استاندارد گراف مانند الگوریتم کراسکال (**Kruskal**) یا پریم (**Prim**) استفاده می‌کند. الگوریتم یال‌ها را به ترتیب نزولی وزن (**MI**) مرتب کرده و قوی‌ترین یال‌ها را یکی پس از دیگری به درخت اضافه می‌کند، به شرطی که حلقه‌ای ایجاد نشود. این کار تا زمانی ادامه می‌یابد که دقیقاً **n-1** یال انتخاب شود.

گام چهارم: استخراج توزیع احتمالی نهایی

پس از تشکیل درخت پوشای بیشینه (**MST**)، سیستم یک گره را به دلخواه به عنوان ریشه (**Root**) انتخاب کرده و جهت یال‌ها را از ریشه به سمت برگ‌ها هدایت می‌کند. اکنون سیستم می‌داند که دقیقاً کدام کلمات قوی‌ترین وابستگی‌ها را دارند و می‌تواند فرمول قانون زنجیره‌ای را دقیقاً بر روی یال‌های همین درخت (همانند مسئله 11.19) اعمال کند.

نتیجه‌گیری نهایی

الگوریتم چو-لیو به سیستم بازایی اجازه می‌دهد تا با محاسبه اطلاعات متقابل (**Mutual Information**) به عنوان وزن یال‌ها و سپس یافتن درخت پوشای بیشینه (**MST**)، قوی‌ترین وابستگی‌های دوه‌دو میان کلمات را استخراج کند. درخت حاصل، بهترین ساختار ممکن برای مدل‌سازی وابستگی عبارات است که هم از خطای «شمارش مضاعف» جلوگیری می‌کند و هم از نظر محاسباتی بسیار بهینه‌تر از محاسبه وابستگی کامل تمام کلمات است.

نکته آموزشی:

دلیل استفاده از درخت پوشای «بیشینه» (و نه کمینه) این است که ما می‌خواهیم یال‌هایی در مدل نهایی باقی بمانند که دارای بیشترین اطلاعات متقابل هستند (یعنی کلماتی که به شدت با هم در اسناد هم‌رخدادی دارند، مانند **Machine** و **Learning**). اثبات ریاضی چو-لیو نشان می‌دهد که مجموع وزن یال‌ها در این درخت، مستقیماً با به حداقل رساندن خطای تقریب (**KL Divergence**) بین مدل درختی و توزیع احتمال واقعی کلمات در زبان طبیعی، در ارتباط است.

پاسخنامه مسئله 11.21: کالیبراسیون BM25 برای متون حقوقی

(BM25 Calibration for Legal Texts)

بخش 1) مفاهیم کلیدی

تابع رتبه‌بندی **Okapi BM25** یکی از موفق‌ترین و پرکاربردترین مدل‌های بازایی اطلاعات است که محدودیت‌های مدل‌های پایه احتمالی (مانند **BIM**) را با معرفی 2 پارامتر تنظیم‌پذیر برطرف می‌کند. این 2 پارامتر عبارتند از:

1. پارامتر **k₁**: کنترل‌کننده سرعت اشباع بسامد کلمه (**Term Frequency Saturation**).

2. پارامتر **b**: کنترل‌کننده میزان نرمال‌سازی طول سند (**Document Length Normalization**).

در سیستم‌های واقعی، مقادیر پیش‌فرض این پارامترها (معمولاً **k₁ = 1.2** و **b = 0.75**) برای متون عمومی وب تنظیم شده‌اند. با این حال، برای دامنه‌های تخصصی مانند متون حقوقی که رفتار زبانی متفاوتی دارند، این پارامترها نیازمند کالیبراسیون و تحلیل مجدد هستند.

بخش 2) تحلیل داده‌ها و مسئله

بر اساس صورت مسئله، با 2 ویژگی متمایز در مجموعه اسناد حقوقی مواجه هستیم:

ویژگی اول: تکرار زیاد یک کلمه کلیدی، ارزش اطلاعاتی آن را به همان نسبت بالا نمی‌برد و اشباع به سرعت رخ می‌دهد.

ویژگی دوم: اسناد طولانی به دلیل ماهیت جامع و چندموضوعی بودن پرونده‌های حقوقی قطور هستند، نه به دلیل استفاده از کلمات زائد (**Verbosity**)؛ پس نباید جریمه شوند.

بخش 3) حل گام به گام

گام اول: تحلیل و تنظیم پارامتر k_1 (اشباع بسامد کلمه)

پارامتر $k_1 \geq 0$ تعیین می‌کند که افزایش تکرار یک کلمه (**TF**)، تا چه حد و با چه سرعتی می‌تواند نمره سند را افزایش دهد.

تحلیل رفتار ریاضی: اگر k_1 عدد بزرگی باشد، نمره سند با تکرار کلمه به صورت خطی افزایش می‌یابد (اشباع دیررس). اما اگر k_1 به سمت 0 میل کند، رفتار تابع به مدل بولین و **BIM** نزدیک می‌شود؛ یعنی صرفاً «حضور» کلمه مهم است و تکرار آن تاثیری در افزایش نمره نخواهد داشت (اشباع سریع).

تصمیم‌گیری برای متون حقوقی: از آنجا که در متون حقوقی بیان شده است تکرار کلمات لزوماً به معنای مرتبط بودن بیشتر نیست (اشباع سریع فرکانس)، باید مقدار k_1 را کاهش داده و به سمت 0 متمایل کنیم (مثلاً در بازه 0.1 تا 0.5). با این کار، سندی که کلمه را 1 بار آورده با سندی که آن را 20 بار تکرار کرده است، تفاوت امتیاز چشمگیری نخواهند داشت.

گام دوم: تحلیل و تنظیم پارامتر b (نرمال سازی طول)

پارامتر $b \in [0, 1]$ تعیین می‌کند که طول یک سند (L_d) نسبت به میانگین طول اسناد مجموعه (L_{avg})، چقدر باید نمره نهایی آن سند را جریمه کند.

تحلیل رفتار ریاضی: اگر $b = 1$ باشد، سیستم فرض می‌کند طولانی بودن سند صرفاً به دلیل پرگویی است و اسناد طولانی را به شدت جریمه می‌کند. در مقابل، اگر $b = 0$ باشد، عامل طول سند به طور کامل از معادله حذف شده و هیچ سندی به خاطر طولانی بودن جریمه نمی‌شود.

تصمیم‌گیری برای متون حقوقی: چون پرونده‌های حقوقی طبیعتاً طولانی هستند و این طولانی بودن نشان‌دهنده پوشش جامع‌تر اطلاعات قانونی است، نباید سیستم این اسناد ارزشمند را سرکوب کند. بنابراین باید مقدار b را به شدت کاهش داده و به سمت 0 نزدیک کنیم (مثلاً در بازه 0.1 تا 0.2).

نتیجه‌گیری نهایی

برای بهینه‌سازی موتور جستجوی حقوقی، مهندس جستجو باید هر 2 پارامتر **BM25** را برخلاف مقادیر پیش‌فرض وب، به سمت 0 میل دهد. کاهش k_1 اثر تکرار مکرر کلمات را خنثی می‌کند و کاهش b مانع از جریمه شدن غیرعادلانه پرونده‌های قطور و طولانی می‌شود.

نکته آموزشی:

تاثیر مستقیم این تنظیمات را می‌توان در کسر مربوط به وزن **TF** در فرمول **BM25** مشاهده کرد:

$$\frac{TF \times (k_1 + 1)}{TF + k_1 \times (1 - b + b \times \frac{L_d}{L_{avg}})}$$

اگر $b = 0$ قرار دهیم، عبارت $\frac{L_d}{L_{avg}}$ کاملاً خنثی شده و طول سند بی‌اثر می‌شود. اگر $k_1 = 0$ قرار دهیم، کل این کسر مستقل از مقدار متغیر TF ، همواره برابر با عدد 1 می‌شود که دقیقاً همان رفتار استقلال باینری (BIM) را شبیه‌سازی می‌کند. در دامنه حقوقی، ما دقیقاً به دنبال همین رفتار شبه-باینری اما با مقداری انعطاف‌پذیری کنترل‌شده هستیم.

پاسخنامه مسئله 11.22: تحلیل ریاضی پارامتر b در $BM25$

بخش 1) مفاهیم کلیدی

در تابع رتبه‌بندی $Okapi\ BM25$ ، پارامتر b نقش «نرمال‌ساز طول سند» ($Document\ Length\ Normalization$) را ایفا می‌کند. هدف این پارامتر، تعدیل نمره اسناد بر اساس طول آن‌ها نسبت به میانگین طول اسناد مجموعه است تا از سوگیری سیستم به سمت اسناد بسیار طولانی (که به صورت تصادفی کلمات بیشتری دارند) جلوگیری شود.

بخش 2) تحلیل فرمول

بخشی از فرمول $BM25$ که مسئول وزن‌دهی به بسامد کلمه (TF) است، به صورت زیر تعریف می‌شود:

$$\frac{TF \times (k_1 + 1)}{TF + k_1 \times \left(1 - b + b \frac{dl}{avdl}\right)}$$

در اینجا:

dl : طول سند مورد نظر

$avdl$: میانگین طول اسناد در کل مجموعه

$b \in [0, 1]$: پارامتر کنترل‌کننده نرمال‌سازی

بخش 3) حل گام‌به‌گام (تحلیل رفتار ریاضی)

حالت اول: تحلیل رفتار سیستم در صورت $b = 1$

اگر مقدار پارامتر b را برابر با 1 قرار دهیم، عبارت داخل پرانتز به صورت زیر ساده می‌شود:

$$\left(1 - 1 + 1 \times \frac{dl}{avdl}\right) = \frac{dl}{avdl}$$

با جایگذاری در فرمول وزن‌دهی:

$$\frac{TF \times (k_1 + 1)}{TF + k_1 \times \left(\frac{dl}{avdl}\right)}$$

نتیجه تحلیل: در این حالت، مخرج کسر به شدت وابسته به نسبت $\frac{dl}{avdl}$ است. اگر طول سند (**dl**) بسیار بزرگتر از میانگین (**avdl**) باشد، مخرج کسر بزرگتر شده و در نتیجه، امتیاز نهایی سند (وزن کلمه) به شدت کاهش می‌یابد.

رفتار سیستم: این تنظیم باعث می‌شود اسناد طولانی به طور سیستماتیک "جریمه" شوند. سیستم در این حالت فرض می‌کند که تکرار یک کلمه در یک سند طولانی، ارزش کمتری نسبت به تکرار همان کلمه در یک سند کوتاه دارد.

حالت دوم: تحلیل رفتار سیستم در صورت $b = 0$

اگر مقدار پارامتر **b** را برابر با 0 قرار دهیم، عبارت داخل پرانتز به صورت زیر ساده می‌شود:

$$\left(1 - 0 + 0 \times \frac{dl}{avdl}\right) = 1$$

با جایگذاری در فرمول وزن‌دهی:

$$\frac{TF \times (k_1 + 1)}{TF + k_1}$$

نتیجه تحلیل: در این حالت، عبارت $\frac{dl}{avdl}$ به طور کامل از معادله حذف شده و عملاً طول سند (**dl**) هیچ تأثیری بر مخرج کسر ندارد.

رفتار سیستم: در این وضعیت، سیستم نسبت به طول سند کاملاً "کور" (**Blind**) است. تفاوتی نمی‌کند سند 100 کلمه باشد یا 10,000 کلمه؛ اگر میزان تکرار کلمه (**TF**) یکسان باشد، وزن نهایی آن کلمه در هر 2 سند دقیقاً برابر خواهد بود. در اینجا تأثیر طول سند به طور کامل خنثی شده است.

نتیجه‌گیری نهایی

از نظر ریاضی، پارامتر **b** یک «اهرم تنظیم‌کننده» است:

با $b = 1$ ، سیستم به سمت **سخت‌گیری حداکثری** و جریمه کردن اسناد طولانی حرکت می‌کند.

با $b = 0$ ، سیستم **نرمال‌سازی را غیرفعال می‌کند** و طول سند را کاملاً نادیده می‌گیرد.

انتخاب مقدار بهینه برای **b** (معمولاً بین 0.5 تا 0.8)، تعادلی میان این 2 دیدگاه ایجاد می‌کند تا اسناد مفید و جامع، بدون جریمه شدن یا بدون سوگیری ناعادلانه به سمت طول زیاد، رتبه‌بندی شوند.

پاسخنامه مسئله 11.23: شبکه بیزی 3 لایه برای بازیابی اطلاعات

(Three-Layer Bayesian Network for IR)

بخش 1) مفاهیم کلیدی

استفاده از «شبکه‌های استنتاجی» (**Inference Networks**) که بر پایه شبکه‌های بیزی (**Bayesian Networks**) بنا شده‌اند، یکی از پیشرفته‌ترین چارچوب‌ها در تئوری بازیابی اطلاعات است (که برای اولین بار توسط **Turtle** و **Croft** معرفی شد). این مدل گرافیکی، فرآیند بازیابی را به عنوان یک مسئله «استنتاج احتمالی» مدل‌سازی می‌کند. در این ساختار، متغیرهای تصادفی به عنوان گره‌ها (**Nodes**) و وابستگی‌های علی یا منطقی میان آن‌ها به عنوان یال‌های جهت‌دار (**Directed Edges**) در 3 لایه مجزا تعریف می‌شوند:

1. لایه اسناد (**D**): گره‌های ریشه (**Roots**) که نشان‌دهنده تک‌تک اسناد در پایگاه داده هستند.

2. لایه کلمات (**T**): گره‌های میانی که نشان‌دهنده مفاهیم یا کلمات نمایه (**Index Terms**) هستند.

3. لایه پرسمان (**Q**): گره‌های برگ (**Leaves**) که نشان‌دهنده نیاز اطلاعاتی یا پرسمان کاربر هستند.

بخش 2) تحلیل ساختار و رسم مسیر جریان

در این مدل، جهت فلش‌ها (یال‌ها) از سمت اسناد به کلمات (**D → T**) و از سمت کلمات به پرسمان (**T → Q**) است. این جهت‌گیری نشان‌دهنده یک «جریان تولیدی» است: یک سند مفاهیم را فعال می‌کند، و آن مفاهیم به نوبه خود پرسمان را ارضا می‌کنند. وابستگی‌های روی این یال‌ها توسط «جداول احتمالات شرطی» (**CPT**) مدیریت می‌شوند. بر اساس صورت مسئله، ما روی مسیر اختصاصی $D_1 \rightarrow T_1 \rightarrow Q$ متمرکز می‌شویم.

بخش 3) حل گام‌به‌گام (تحلیل جریان استنتاج و محاسبه احتمال پسین)

گام اول: مقداردهی اولیه شبکه (تعریف جداول شرطی و پیشین)

پیش از ورود پرسمان، شبکه در وضعیت خنثی قرار دارد:

احتمال پیشین اسناد: هر سند در ابتدا دارای یک احتمال پیشین بسیار کوچک و مساوی است (مثلاً

$$P(D_1 = 1) = 1/N$$

وابستگی سند-کلمه (**D → T**): صورت مسئله بیان می‌کند که «سند **D₁** دارای وزن بالایی برای کلمه **T₁** است».

در زبان شبکه بیزی، این بدان معناست که احتمال شرطی فعال شدن کلمه **T₁** به شرط فعال بودن سند **D₁**

بسیار بالاست. یعنی متغیر $P(T_1 = 1 | D_1 = 1)$ نزدیک به 1 است.

وابستگی کلمه-پرسمان (**T → Q**): پرسمان کاربر مستقیماً به کلمه **T₁** متصل است؛ بنابراین

$$P(Q = 1 | T_1 = 1)$$
 نیز مقدار بالایی دارد.

گام دوم: دریافت شواهد (Observing Evidence)

هنگامی که کاربر پرسمان خود را در سیستم ثبت می‌کند، این رویداد در شبکه به عنوان یک «شاهد قطعی»

(**Evidence**) ثبت می‌شود. به بیان ریاضی، گره **Q** در شبکه مقدار «درست» یا 1 به خود می‌گیرد (**Q = 1**). هدف سیستم

اکنون این است که متوجه شود کدام سند به احتمال بیشتری باعث به وجود آمدن این شاهد شده است.

گام سوم: انتشار باور (Belief Propagation) و جریان استقرایی

با فعال شدن گره **Q**، اطلاعات برخلاف جهت فلش‌ها در شبکه به سمت بالا جریان می‌یابد (استنتاج تشخیصی یا

Diagnostic Reasoning). سیستم می‌خواهد احتمال پسین (**Posterior Probability**) سند **D₁** را با توجه به شواهد

$$P(D_1 = 1 | Q = 1)$$
 جدید محاسبه کند، یعنی

طبق قضیه بیز، این احتمال برابر است با:

$$P(D_1 = 1|Q = 1) = \frac{P(Q = 1|D_1 = 1) \times P(D_1 = 1)}{P(Q = 1)}$$

گام چهارم: محاسبه تأثیر لایه میانی (حاشیه‌سازی روی لایه T)

برای حل صورت کسر بالا، سیستم باید احتمال $P(Q = 1 | D_1 = 1)$ را از طریق لایه میانی (کلمات) محاسبه کند. بر اساس قانون احتمالات کل و فرض استقلال شرطی در شبکه بیزی، این مقدار با جمع زدن روی تمام حالات ممکن گره T_1 به دست می‌آید:

$$P(Q = 1|D_1 = 1) = \sum_{t \in \{0,1\}} P(Q = 1|T_1 = t) \times P(T_1 = t|D_1 = 1)$$

از آنجا که می‌دانیم مقدار $P(T_1 = 1 | D_1 = 1)$ به دلیل «وزن بالای کلمه در سند» بسیار بزرگ است، و همچنین T_1 مستقیماً پرسمان را فعال می‌کند ($P(Q=1|T_1=1)$ بزرگ است)، حاصل ضرب این 2 مقدار در معادله بالا عدد بزرگی خواهد شد.

نتیجه‌گیری نهایی

ورود پرسمان به عنوان شواهد (Evidence)، گره Q را فعال می‌کند. این فعال‌سازی از طریق کلمه کلیدی T_1 به عنوان یک پل ارتباطی، به سمت بالا منتقل شده و باور شبکه را نسبت به گره D_1 به‌روزرسانی می‌کند. به دلیل وزن بالای ارتباط $D_1 \rightarrow T_1 \rightarrow Q$ ، احتمال پسین $P(D_1 = 1 | Q = 1)$ جهش چشمگیری نسبت به احتمال پیشین خود پیدا می‌کند. سیستم در نهایت تمام اسناد لایه ریشه را بر اساس همین احتمال پسین از بزرگ به کوچک مرتب (Rank) کرده و D_1 را در صدر نتایج به کاربر نمایش می‌دهد.

نکته آموزشی:

قدرت خارق‌العاده شبکه‌های بیزی در بازیابی اطلاعات، در انعطاف‌پذیری «جداول احتمالات شرطی» (CPT) نهفته است. اگر این جداول را با 0 و 1های مطلق مقداردهی کنیم، شبکه بیزی دقیقاً رفتار «مدل منطق بولین» (Boolean Model) را شبیه‌سازی می‌کند. اما اگر آن‌ها را با احتمالات برگرفته از مقادیر TF-IDF یا هموارسازی‌های BM25 مقداردهی کنیم، شبکه به یک سیستم رتبه‌بندی احتمالی بسیار قدرتمند تبدیل می‌شود. این معماری، مدلی جامع (Unified Model) است که تئوری‌های مختلف بازیابی را زیر یک چتر ریاضی واحد گرد هم می‌آورد.

پاسخنامه مسئله 11.24: تفاوت فلسفی مدل زبان و مدل احتمالی

(Philosophical Difference of Language vs. Probabilistic Models)

بخش 1) مفاهیم کلیدی

در اواخر دهه 90 میلادی، معرفی «مدل‌های زبانی» (Language Models) یک تغییر پارادایم (Paradigm Shift) بنیادین در تئوری بازیابی اطلاعات ایجاد کرد. تا پیش از آن، تمامی مدل‌های احتمالی کلاسیک (مانند BIM) حول محور پیش‌بینی یک متغیر تصادفی پنهان به نام «ارتباط» (Relevance) می‌چرخیدند. اما مدل‌های زبانی با اتخاذ یک رویکرد کاملاً متفاوت و معکوس،

مفهوم صریح «ارتباط» را از معادلات ریاضی خود حذف کردند و به جای آن، از مفهوم «تولید متن» (Text Generation) بهره بردند.

بخش 2) تحلیل و مدل سازی

برای درک این تفاوت فلسفی، باید نگاه سیستم به کاربر و نحوه شکل گیری پرسمان را در 2 مدل مقایسه کنیم. این تفاوت در تابع هدف و جهت شرطی شدن متغیرها خود را نشان می دهد.

بخش 3) حل گام به گام

گام اول: رویکرد احتمالی کلاسیک (نگاه تشخیصی در BIM)

در مدل استقلال باینری (BIM)، سیستم یک نگاه «تشخیصی» (Diagnostic) دارد. سیستم سند (d) و پرسمان (q) را به عنوان داده های ورودی دریافت می کند و تلاش می کند احتمال مرتبط بودن آن ها را محاسبه کند. تابع هدف در این حالت به صورت زیر تعریف می شود:

$$P(R = 1 | d, q)$$

فلسفه: «با توجه به شواهدی که از سند و پرسمان می بینم، چقدر احتمال دارد که این سند برای کاربر مفید (مرتبط) باشد؟» در این نگاه، ارتباط (R) یک ویژگی عینی است که سیستم باید آن را کشف کند و برای این کار، شدیداً به داده های آموزشی یا بازخورد (Relevance Feedback) نیاز دارد.

گام دوم: رویکرد مدل زبانی (نگاه تولیدی یا Generative)

مدل زبانی، متغیر ارتباط (R) را به طور کامل کنار می گذارد. در این فلسفه، هر سند در پایگاه داده به عنوان یک «مدل زبانی مجزا» (که با نماد M_d نشان داده می شود) در نظر گرفته می شود؛ گویی هر سند، یک جهان کوچک با دایره واژگان و توزیع احتمالات خاص خود است. تابع هدف در این مدل به صورت معکوس و به شکل زیر تعریف می شود:

$$P(q | M_d)$$

فلسفه: سیستم از خود می پرسد: «اگر کاربری در جهان کلمات این سند خاص زندگی می کرد و می خواست به صورت تصادفی کلماتی را از داخل این سند بیرون بکشد، چقدر احتمال داشت که دقیقاً کلمات پرسمان q را تولید کند؟»

گام سوم: تحلیل معکوس بودن مدل سازی (The Inverse Relationship)

تفاوت اصلی در «جهت پیکان شرطی سازی» است:

BIM در پیکان از سمت ویژگی های سند و پرسمان، به سمت متغیر ارتباط می رود ($d, q \rightarrow R$). سیستم می پرسد: آیا این سند برای پرسمان، یک پاسخ مرتبط است؟

(LM) در مدل زبانی پیکان از سمت سند به سمت پرسمان می رود ($d \rightarrow q$). سیستم می پرسد: آیا این سند،

نویسنده (تولیدکننده) این پرسمان است؟

در مدل زبانی، کاربر به عنوان شخصی در نظر گرفته می‌شود که یک سند ایده‌آل را در ذهن خود دارد و پرسمان را با انتخاب کلماتی از آن سند ذهنی تولید کرده است. کار موتور جستجو این است که بگردد و ببیند کدام سند واقعی در پایگاه داده، بیشترین شباهت را به آن مدل ذهنی دارد (بیشترین احتمال برای تولید آن پرسمان را دارد).

نتیجه‌گیری نهایی

تفاوت فلسفی این 2 مدل در این است که **BIM** یک مسئله‌ی «دسته‌بندی دودویی» (طبقه‌بندی اسناد به مرتبط و غیرمرتبط) است، در حالی که مدل زبانی یک مسئله‌ی «تخمین توزیع احتمال» است. در مدل زبانی، سندی مرتبط‌تر ارزیابی می‌شود که زبان و ادبیات پرسمان، هم‌خوانی و هم‌پوشانی بیشتری با توزیع زبانی آن سند داشته باشد.

نکته آموزشی:

بزرگترین مزیت ریاضی این «مدل‌سازی معکوس» این است که مدل‌های زبانی به هیچ‌گونه بازخورد اولیه یا داده آموزشی (برای تخمین p_i و u_i) نیاز ندارند. آن‌ها می‌توانند احتمالات را مستقیماً از روی بسامد کلمات درون خود سند (**TF**) محاسبه کنند و از طریق تکنیک‌های هموارسازی (مانند **Jelinek-Mercer** یا **Dirichlet Prior**)، به صورت کاملاً طبیعی متغیرهایی مانند «طول سند» و «اهمیت کلمه در کل مجموعه (**IDF**)» را در دل فرمول $P(q|M_d)$ جای دهند، بدون اینکه نیازی به پارامترهای ابتکاری و دستی (**Heuristics**) باشد.

پاسخنامه مسئله 11.25: معماری سیستم و فشرده‌سازی در BM25

(System Architecture and Compression in BM25)

بخش 1) مفاهیم کلیدی

در معماری موتورهای جستجو با مقیاس وب (**Web-scale IR Architecture**)، محاسبه تابع **BM25** برای رتبه‌بندی اسناد نیازمند دسترسی بی‌درنگ (**Real-time**) به طول تک‌تک اسناد (**dl**) است. اگر پایگاه داده‌ای شامل میلیاردها سند باشد، ذخیره دقیق طول هر سند در قالب اعداد صحیح استاندارد (**32** بیتی)، به ده‌ها گیگابایت حافظه اصلی (**RAM**) تنها برای نگهداری آرایه طول‌ها نیاز دارد. از آنجا که دسترسی به دیسک در زمان پردازش پرسمان باعث افت شدید سرعت می‌شود، مهندسان سیستم از ترکیب 2 تکنیک «فشرده‌سازی با اتلاف» (کوانتایزیشن/دسته‌بندی) و «فشرده‌سازی بدون اتلاف» (کدگذاری طول متغیر) استفاده می‌کنند تا این ساختار داده را در سریع‌ترین لایه‌های حافظه جا دهند.

بخش 2) تحلیل داده‌ها و مسئله

فرض کنید موتور جستجویی دارای $N = 10^{10}$ (**10** میلیارد) سند است.

حالت سنتی: ذخیره طول دقیق هر سند به عنوان یک عدد **32** بیتی (**4** بایت)، نیازمند **40** گیگابایت حافظه **RAM** است

که از نظر معماری گلوگاه (**Bottleneck**) محسوب می‌شود.

رفتار ریاضی BM25: فرمول نرمال‌سازی طول در **BM25** در برابر تغییرات بسیار جزئی طول سند، مقاوم است.

$$1 - b + b \cdot \frac{dl}{avdl}$$

به عنوان مثال، تفاوتی بین سندی با طول 1000 کلمه و سندی با طول 1005 کلمه در خروجی نهایی رتبه‌بندی احساس نمی‌شود. این ویژگی به ما اجازه می‌دهد به جای مقادیر دقیق، از مقادیر تقریبی استفاده کنیم.

بخش 3) طراحی استراتژی معماری

گام اول: تقریب مقادیر از طریق دسته‌بندی (Length Quantization & Bucketing)

به جای ذخیره طول دقیق، فضای طول اسناد را به تعدادی «دسته» (**Bucket**) تقسیم می‌کنیم. این دسته‌بندی به صورت غیرخطی (معمولاً لگاریتمی) انجام می‌شود؛ زیرا برای اسناد کوتاه، تفاوت 10 کلمه بسیار مهم است، اما برای اسناد بسیار طولانی (مثلاً ده‌ها هزار کلمه)، تفاوت 500 کلمه نیز تأثیری در نرمال‌سازی ندارد. سیستم تمام طول‌های ممکن را به تعداد محدودی دسته (مثلاً 256 دسته) نگاشت می‌کند. اکنون به جای ذخیره طول واقعی، تنها «شناسه دسته» (**Bucket ID**) را ذخیره می‌کنیم که عددی بین 0 تا 255 است.

گام دوم: اعمال کدگذاری طول متغیر (Variable-Length Encoding)

با کاهش مقادیر به شناسه‌های کوچک، اکنون از تکنیک‌های فشرده‌سازی بدون اتلاف مانند «کدگذاری بایت متغیر» (**Variable Byte Code**) یا کدهای بیتی (مانند **Elias Gamma**) برای ذخیره این شناسه‌ها در حافظه استفاده می‌کنیم. از آنجا که در وب، توزیع طول اسناد از «قانون زیف» (**Zipf's Law**) یا توزیع‌های توانی پیروی می‌کند، اکثریت قریب به اتفاق اسناد وب نسبتاً کوتاه هستند و در دسته‌های اولیه (با شناسه‌های بسیار کوچک مانند 1، 2 یا 5) قرار می‌گیرند. با استفاده از **Variable Byte Code**، این شناسه‌های کوچک تنها در 1 بایت (و گاهی در سطح بیت) ذخیره می‌شوند. این تکنیک، حجم آرایه را از 40 گیگابایت به حدود 10 گیگابایت یا حتی کمتر (بسته به الگوریتم بیتی) کاهش می‌دهد.

گام سوم: ادغام در معماری زمان جستجو (Query-Time Architecture)

برای استفاده از این داده‌های فشرده‌شده در زمان اجرای پرس‌مان، سیستم ساختاری به نام «نمایه پیش‌رو» (**Forward Index**) برای طول اسناد می‌سازد:

- هنگامی که یک کلمه جستجو می‌شود، سیستم «لیست ارسال» (**Posting List**) آن کلمه را از نمایه وارونه (**Inverted Index**) فراخوانی می‌کند.
- با خواندن شناسه سند (**DocID**) از لیست ارسال، سیستم مستقیماً به آرایه فشرده‌شده‌ی طول‌ها در **RAM** مراجعه می‌کند.
- مقدار فشرده‌شده (کد بایت متغیر) خوانده شده و در کسری از نانو ثانیه توسط پردازنده (با دستورات **SIMD**) از حالت فشرده خارج (**Decode**) می‌شود.
- شناسه دسته به دست می‌آید و مقدار تقریبی طول سند (از یک جدول مراجعه کوچک یا **Lookup Table**) استخراج شده و به تابع **BM25** تزریق می‌گردد.

نتیجه‌گیری نهایی

استراتژی ترکیبی «تقریب لگاریتمی طول‌ها» و متعاقب آن «فشرده‌سازی با کدهای طول متغیر»، یک راهکار معماری بهینه است که با قربانی کردن بخش بسیار ناچیزی از دقت غیرضروری در طول سند، حجم حافظه مورد نیاز را به صورت نمایی کاهش

می‌دهد. این معماری تضمین می‌کند که آرایه طول اسناد همواره در سریع‌ترین لایه‌های حافظه پنهان (**Cache**) باقی بماند و سرعت محاسبه تابع **BM25** در مقیاس‌های میلیارد سندی حفظ شود.

نکته آموزشی:

این دقیقاً همان استراتژی مهندسی درخشانی است که موتورهای جستجوی صنعتی و کتابخانه‌های متن‌باز نظیر **Apache Lucene** (هسته مرکزی **Elasticsearch** و **Solr**) از آن استفاده می‌کنند. در **Lucene** کلاسیک، فاکتورهای نرمال‌سازی طول سند (**Norms**) با استفاده از یک سیستم دسته‌بندی دقیقاً در 1 بایت (8 بیت) کدگذاری می‌شوند (ترکیب 3 بیت برای مانیتیس و 5 بیت برای نما). درک این ترفندها به دانشجویان نشان می‌دهد که در سیستم‌های توزیع‌شده، تئوری خالص بازیابی اطلاعات همواره باید با محدودیت‌های سخت‌افزاری و معماری کامپیوتر هم‌سو و کالیبره شود.

پاسخنامه مسئله 11.26: مسئله مقداردهی اولیه در سیستم‌های فروشگاه‌ای (Initialization Problem in E-commerce)

بخش 1) مفاهیم کلیدی

در موتورهای جستجوی تجارت الکترونیک (**E-commerce**)، یکی از چالش‌های بنیادین، پدیده «شروع سرد» (**Cold Start**) یا مسئله مقداردهی اولیه است. هنگامی که از مدل‌های بازخورد ارتباط احتمالی (**Probabilistic Relevance Feedback**) استفاده می‌شود، سیستم برای ارتقای رتبه‌بندی نیازمند سیگنال‌های رفتاری کاربران (مانند کلیک، افزودن به سبد خرید و ثبت سفارش) است. محصولات جدیدی که به تازگی وارد کاتالوگ فروشگاه شده‌اند، فاقد این تاریخچه رفتاری هستند. در نتیجه، الگوریتم‌های احتمالی در ارزیابی ارزش واقعی این محصولات دچار نقص اطلاعاتی شده و آن‌ها را به صورت ناعادلانه‌ای جریمه می‌کنند.

بخش 2) تحلیل داده‌ها و صورت مسئله

در مدل‌های احتمالی کلاسیک، وزن یک کلمه برای یک سند خاص، پس از دریافت بازخورد کاربر به‌روزرسانی می‌شود (متغیرهای **S** و **s**). برای محصولات قدیمی، کلیک‌های کاربران باعث می‌شود سیستم با اطمینان بالایی احتمال مرتبط بودن آن‌ها را تخمین بزند. اما برای یک محصول جدید (**d_{new}**):

تعداد کلیک‌ها برابر با 0 است.

احتمال مرتبط بودن بر اساس رفتار، معادل شانس تصادفی یا 0 در نظر گرفته می‌شود.

در این رقابت نابرابر، محصول جدید تنها بر اساس تطابق پایه کلمات کلیدی (مانند **IDF**) امتیاز می‌گیرد و همواره زیر سایه محصولات قدیمی‌تر و پرکلیک (حتی اگر آن محصولات قدیمی از رده خارج شده باشند) دفن می‌شود.

بخش 3) حل گام‌به‌گام با طراحی سناریو

گام اول: تعریف سناریوی فروشگاه‌ای

فرض کنید کاربری عبارت پرسمان «گوشی هوشمند سامسونگ با دوربین عالی» را در فروشگاه جستجو می‌کند.

سند الف (محصول قدیمی): گوشی **Samsung Galaxy S23** که 100,000 بار کلیک و خرید داشته است.

سیستم از طریق بازخورد کاربران آموخته است که این محصول دقیقاً با کلمات «دوربین عالی» مرتبط است (**p_i**

بسیار بالا).

سند ب (محصول جدید): گوشی Samsung Galaxy S24 که امروز صبح به سایت اضافه شده است. هیچ

کلیکی ندارد.

با وجود اینکه سند «ب» مدل جدیدتر و مرتبطتری است، مدل پایه احتمالی به دلیل نبود داده‌های بازخورد، سند «الف» را در رتبه بسیار بالاتری قرار می‌دهد.

گام دوم: راهکار حل مسئله (تغییر پارادایم از رفتار به محتوا)

برای حل این مشکل، باید متغیر پنهان «ارتباط» (R) را به جای اینکه منحصرأً به شناسه سند ($Document ID$) و کلیک‌های آن گره بزنیم، به «ویژگی‌های محتوایی و متادیتا» ($Content Features$) متصل کنیم. هر محصول جدید، اگرچه کلیک ندارد، اما دارای مجموعه‌ای از ویژگی‌های ساختاریافته $F(d)$ است:

$$F(d_{new}) = \{f_1, f_2, f_3, \dots, f_n\}$$

گام سوم: فرمول‌بندی احتمال پیشین مبتنی بر ویژگی‌ها (Feature-based Prior Probability)

در مدل BIM گسترش‌یافته، تابع امتیازدهی (RSV) شامل بخشی برای «احتمال پیشین» ($Prior Probability$) است که معمولاً برای همه اسناد ثابت فرض شده و حذف می‌گردد. اکنون ما این مقدار ثابت را بازگردانده و آن را به عنوان یک تابع پویا از ویژگی‌های محصول جدید تعریف می‌کنیم:

$$RSV_d = \log \frac{P(R=1|F(d))}{P(R=0|F(d))} + \sum_{x_i=1} \log \frac{p_i(1-u_i)}{u_i(1-p_i)}$$

در این معادله، عبارت اول (کسر پیشین)، ارزش محتوایی محصول را قبل از هرگونه بررسی کلمات کلیدی تخمین می‌زند.

گام چهارم: محاسبه شانس محتوایی از طریق تعمیم داده‌ها (Generalization)

سیستم برای محاسبه $P(R=1 | F(d_{new}))$ به پایگاه داده تاریخی خود مراجعه می‌کند. سیستم مشاهده می‌کند که برای جستجوی «گوشی هوشمند با دوربین عالی»، اسنادی که دارای ویژگی‌های $f_1 = Samsung$ و $f_2 = Flagship$ (پرچمدار) بوده‌اند، در گذشته نرخ کلیک (CTR) بسیار بالایی (مثلاً 40 درصد) داشته‌اند. بنابراین، سیستم این نرخ موفقیت تاریخی را به عنوان احتمال پایه به محصول جدید ($S24$) که دقیقاً همین ویژگی‌ها را دارد، تزریق می‌کند (به ارث می‌رساند).

نتیجه‌گیری نهایی

مسئله مقداردهی اولیه ($Cold Start$) در سیستم‌های فروشگاهی با ادغام «ویژگی‌های محتوایی» در دل «مدل احتمالی» حل می‌شود. با این معماری، محصول جدیدی که هیچ کلیک ندارد، اعتبار اولیه ($Prior Belief$) خود را از موفقیت تاریخی محصولات مشابه به آن بوده‌اند (هم‌خانواده، هم‌برند، یا هم‌رده) وام می‌گیرد.

نکته آموزشی:

این روش در ادبیات بازیابی اطلاعات تحت عنوان «هموارسازی مبتنی بر ویژگی» ($Feature-based Smoothing$) یا ترکیب مدل‌های محتوا-محور با فیلتر مشارکتی شناخته می‌شود. در موتورهای جستجوی مدرن تجارت الکترونیک (مانند آمازون)، بخش لگاریتمی اول

فرمول ما، توسط مدل‌های یادگیری ماشین (مانند درخت‌های تصمیم تقویتی یا **GBDT**) تخمین زده می‌شود تا به محض ورود یک کالای جدید، یک امتیاز پایه یا **Boost** منطقی به آن اختصاص یابد و کالا شانس دیده شدن و جمع‌آوری کلیک‌های اولیه را پیدا کند.

پاسخنامه مسئله 11.27: مقایسه Okapi BM25 و BIM (Comparing Okapi BM25 and BIM)

بخش 1) مفاهیم کلیدی

مدل استقلال باینری (**BIM**) به عنوان نقطه شروع مدل‌های احتمالاتی، یک چارچوب ریاضی مستحکم برای بازیابی اطلاعات ارائه داد. اما صفت «باینری» در این مدل، پاشنه آشیل آن در مواجهه با متون واقعی بود؛ زیرا این مدل جهان متن را تنها به صورت «وجود» (1) یا «عدم وجود» (0) کلمات می‌دید و از سیگنال‌های حیاتی دیگر چشم‌پوشی می‌کرد.

مدل **Okapi BM25 (Best Matching 25)** با حفظ هسته‌ی تئوریک **BIM** (یعنی منطق رتبه‌بندی احتمالی و نسبت شانس)، از قید و بند باینری بودن رها شد. این مدل با معرفی 3 مولفه «غیر باینری» و پیوسته که ریشه در ویژگی‌های ساختاری متن دارند، ضعف‌های بنیادین **BIM** را به طور کامل پوشش داد.

بخش 2) تحلیل داده‌ها و مسئله

برای درک تحلیلی این گذار، باید بررسی کنیم که فرمول **BM25** چگونه بسامد کلمه، طول سند و ساختار پرسمان را از حالت 0 و 1 خارج کرده و به توابعی پویا (**Dynamic**) تبدیل کرده است. در ادامه، این 3 تفاوت ساختاری را به صورت گام‌به‌گام کالبدشکافی می‌کنیم.

بخش 3) حل گام‌به‌گام

گام اول: گذار از حضور باینری به اشباع بسامد کلمه (Term Frequency Saturation)

ضعف BIM: در مدل باینری، ارزش سندی که کلمه هدف را 1 بار آورده با سندی که آن را 100 بار تکرار کرده برابر است ($x_i \in \{0, 1\}$). این یعنی از دست دادن سیگنال «تمرکز موضوعی» سند.

راهکار BM25: این مدل متغیر دودویی x_i را حذف کرده و مفهوم **TF (بسامد کلمه)** را وارد معادله می‌کند. اما برای جلوگیری از غلبه‌ی اسنادی که یک کلمه را به صورت افراطی تکرار می‌کنند، از یک «تابع اشباع غیرخطی» با پارامتر کنترل‌کننده k_1 استفاده می‌کند:

$$\frac{TF \times (k_1 + 1)}{TF + k_1}$$

از نظر تحلیلی، این فرمول یک رفتار هوشمندانه دارد: در تکرارهای اولیه، نمره به سرعت افزایش می‌یابد (پاداش برای تمرکز روی موضوع)، اما با افزایش بیشتر **TF**، نمودار به جانب $k_1 + 1$ میل کرده و اشباع می‌شود تا اثر تکرارهای بی‌مورد مهار گردد.

گام دوم: گذار از کوری ساختاری به نرمال‌سازی طول سند (Document Length Normalization)

ضعف BIM: مدل باینری طول سند را نادیده می‌گیرد. یک پاراگراف و یک کتاب برای این مدل تفاوتی ندارند، که این امر موجب «سوگیری طول» (**Length Bias**) به نفع اسناد طولانی می‌شود، زیرا اسناد طولانی به صورت تصادفی کلمات بیشتری را در خود جای می‌دهند.

راهکار BM25: مدل **BM25** مفهوم طول سند (**dl**) و میانگین طول اسناد مجموعه (**avdl**) را از طریق پارامتر تنظیمی **b** مستقیماً به مخرج کسر بسامد کلمه تزریق می‌کند:

$$\frac{TF \times (k_1 + 1)}{TF + k_1 \times (1 - b + b \frac{dl}{avdl})}$$

این رویکرد کاملاً غیر باینری، جریمه‌ای متناسب و پیوسته اعمال می‌کند. سندی که طول آن دقیقاً برابر میانگین باشد ($\frac{dl}{avdl} = 1$)، هیچ تغییری در نمره احساس نمی‌کند؛ اما اسناد بیش از حد طولانی، با بزرگ شدن مخرج کسر، با کاهش هوشمندانه‌ی وزن کلماتشان مواجه می‌شوند.

گام سوم: گذار از پرسمان باینری به وزن‌دهی بسامد پرسمان (Query Term Frequency)

ضعف BIM: در **BIM**، پرسمان کاربر نیز به عنوان یک بردار باینری در نظر گرفته می‌شود ($q_i \in \{0, 1\}$). اگر کاربری در یک پرسمان طولانی (مانند جستجوی مفهومی یا توصیفی) یک کلمه کلیدی را چند بار تکرار کند تا اهمیت آن را نشان دهد، سیستم آن را نادیده می‌گیرد.

راهکار BM25: مدل **BM25** به جای رفتار باینری با پرسمان، متغیر **qtf** (بسامد کلمه در پرسمان) را معرفی کرده و با استفاده از پارامتر **k₃**، وزن کلمات را در سمت پرسمان نیز تنظیم می‌کند:

$$\frac{qtf \times (k_3 + 1)}{qtf + k_3}$$

این ویژگی در سیستم‌های مدرن که پرسمان‌ها به شکل متون بلند (مثل پاراگرافی از یک مقاله برای پیدا کردن مقالات مشابه) وارد سیستم می‌شوند، به شدت حیاتی است و به مدل اجازه می‌دهد هسته معنایی پرسمان را از طریق تکرار کلمات در آن شناسایی کند.

نتیجه‌گیری نهایی

مدل **Okapi BM25** یک شاهکار در مهندسی بازیابی اطلاعات است، زیرا توانست **تئوری احتمالات مستحکم BIM** (از طریق استفاده از منطق **IDF** پایه رابرتسون-اسپارک جونز) را با **ویژگی‌های وزن‌دهی پیوسته‌ی مدل فضای برداری (TF)** و نرمال‌سازی طول ترکیب کند. با کنار گذاشتن محدودیت‌های **0** و **1**، **BM25** به مدلی منعطف و قابل کالیبره (پارامتریک) تبدیل شد که تا به امروز به عنوان خط پایه (**Baseline**) و استاندارد صنعتی در موتورهای جستجو شناخته می‌شود.

نکته آموزشی:

اگر پارامترهای مدل **BM25** را به صورت حدی دستکاری کنیم، می‌توانیم آن را به مدل‌های دیگر تقلیل دهیم. به عنوان مثال، اگر در فرمول **BM25** مقدار $k_1 = 0$ قرار داده شود، اثر بسامد کلمه و طول سند کاملاً از بین می‌رود و کل کسر مربوط به **TF** برابر با عدد **1**

می‌شود. در این حالت خاص، خروجی **BM25** دقیقاً به فرمول پایه و باینری **BIM** (فقط جمع اوزان **IDF**) تبدیل خواهد شد. این امر نشان می‌دهد که **BIM** صرفاً یک حالت خاص و محدودشده از مدل جامع **BM25** است.

پاسخنامه مسئله 11.28: سناریوی شبکه‌های بیزی با کلمات مترادف

(Bayesian Networks Scenario with Synonyms)

بخش 1) مفاهیم کلیدی

یکی از چالش‌های کلاسیک در مدل‌های بازیابی اطلاعات (به‌ویژه مدل‌های مبتنی بر تطابق دقیق کلمات مانند مدل فضای برداری پایه یا مدل بولین)، مسئله «اختلاف واژگانی» (**Vocabulary Mismatch**) است. کاربران غالباً برای بیان یک نیاز اطلاعاتی از کلماتی استفاده می‌کنند که نویسندگان سند از آن‌ها استفاده نکرده است (مانند جستجوی "**Car**" برای سندی که از کلمه "**Automobile**" استفاده کرده است).

در معماری «شبکه‌های استنتاجی» (**Inference Networks**) که مبتنی بر شبکه‌های بیزی هستند، این مشکل به ظریف‌ترین شکل ممکن و از طریق افزودن یک لایه جدید به نام «لایه مفاهیم» (**Concept Layer**) حل می‌شود. این لایه به عنوان یک پل معنایی میان کلمات فیزیکی موجود در متن و کلمات پرسمان کاربر عمل می‌کند.

بخش 2) تحلیل ساختار

برای مدل‌سازی مترادف‌ها، معماری 3 لایه‌ای استاندارد (سند → کلمه → پرسمان) به یک معماری 4 لایه ارتقا می‌یابد. در این معماری، چندین کلمه فیزیکی مترادف به یک «گره مفهومی» واحد متصل می‌شوند. نمای شماتیک گراف برای سناریوی مورد بحث به شکل زیر است:

لایه اسناد (**Document Node**): گره D_1 (سند شامل کلمه **Automobile**)

لایه کلمات (**Term Nodes**): گره T_a (کلمه **Automobile**) و گره T_c (کلمه **Car**)

لایه مفاهیم (**Concept Node**): گره میانی C_1 (مفهوم انتزاعی «خودرو / وسیله نقلیه»)

لایه پرسمان (**Query Node**): گره Q (پرسمان ثبت‌شده توسط کاربر: "**Car**")

مسیر اتصال در گراف (**Topology**):

$$D_1 \longrightarrow T_a \longrightarrow C_1 \longrightarrow Q \longleftarrow T_c$$

بخش 3) تحلیل جریان انتشار احتمالات

گام اول: مقداردهی جداول احتمالات شرطی (CPT) برای یال‌های شبکه

سیستم برای برقراری این اتصالات معنایی، جداول احتمالات را در لایه‌های مختلف مقداردهی می‌کند:

1. یال $D_1 \rightarrow T_a$: چون کلمه "**Automobile**" صراحتاً در متن سند D_1 وجود دارد، احتمال فعال شدن این کلمه

توسط سند بسیار بالاست ($P(T_a = 1 | D_1 = 1) \approx 1$).

2 یال‌های $T_a \rightarrow C_1$ و $T_c \rightarrow C_1$: سیستم (از طریق یک هستان‌شناسی (Ontology) یا گنج‌واژه (Thesaurus))

مانند (WordNet) می‌داند که هر 2 کلمه، نمایانگر مفهوم C_1 هستند. بنابراین، فعال شدن هر یک از این

کلمات، گره مفهومی را با احتمال بالایی فعال می‌کند ($P(C_1=1 | T_a=1)$ بزرگ است).

3. یال $C_1 \rightarrow Q$: پرسمان کاربر با استفاده از مفهوم خودرو شکل گرفته است، بنابراین وابستگی شدیدی میان

این 2 گره وجود دارد.

گام دوم: مشاهده شواهد (Observing the Evidence)

کاربر پرسمان "Car" را وارد می‌کند. در شبکه بیزی، گره Q به عنوان یک شاهد قطعی مقدار «درست» به خود می‌گیرد (

$Q = 1$). اکنون سیستم باید استنتاج تشخیصی (Diagnostic Inference) را آغاز کرده و احتمالات را برخلاف جهت

یال‌ها به سمت بالا (لایه اسناد) منتشر کند.

گام سوم: انتشار باور از طریق گره میانی (Belief Propagation)

گره Q فعال شده است و به دنبال مفاهیم تولیدکننده‌اش می‌گردد. این امر باعث می‌شود گره مفهومی C_1 در

لایه بالاتر فعال شود.

گره مفهومی C_1 به 2 کلمه T_a و T_c متصل است. سیستم بررسی می‌کند که کدام اسناد این کلمات را فعال

کرده‌اند.

سند D_1 کلمه T_a ("Automobile") را فعال کرده است. این فعال‌سازی از طریق مسیر $D_1 \rightarrow T_a \rightarrow C_1$ به گره

مفهومی منتقل می‌شود.

گام چهارم: محاسبه احتمال پسین (Posterior Probability)

با وجود اینکه سند D_1 هیچ اتصال مستقیمی به کلمه "Car" (T_c) ندارد، اما به دلیل وجود مسیر جایگزین از طریق گره

مفهومی C_1 ، احتمال پسین مرتبط بودن سند با پرسمان، یعنی $P(D_1=1 | Q=1)$ ، مقدار قابل‌توجهی دریافت می‌کند.

طبق قوانین استقلال شرطی در گراف، این مقدار از طریق جمع زدن روی حالات مختلف گره‌های میانی (حاشیه‌سازی)

محاسبه می‌شود:

$$P(Q = 1 | D_1 = 1) = \sum_{c \in \{0,1\}} \sum_{t \in \{0,1\}} P(Q = 1 | C_1 = c) P(C_1 = c | T_a = t) P(T_a = t | I)$$

نتیجه‌گیری نهایی

در شبکه‌های بیزی، مفهوم «کلمات مترادف» با اضافه کردن یک «لایه مفاهیم» در میانه شبکه و متصل کردن کلمات هم‌معنی به

یک گره مفهومی واحد، مدل‌سازی می‌شود. این معماری به سیستم اجازه می‌دهد تا پرسمان "Car" را ثبت کرده و از طریق گره

میانی (مفهوم خودرو)، جریانی از احتمالات را به سمت کلمه "Automobile" و در نهایت به سمت سند D_1 هدایت کند؛ در نتیجه،

سند مرتبط با وجود عدم تطابق لغوی، با موفقیت بازیابی و رتبه‌بندی می‌شود.

نکته آموزشی:

یکی از مزیت‌های قدرتمند این معماری نسبت به روش‌های سنتی «بسط پرسمان» (Query Expansion) این است که سیستم

درگیر اضافه‌بار کلمات زائد نمی‌شود. در روش‌های سنتی، کلمه "Automobile" به صورت دستی به پرسمان کاربر اضافه می‌شود که

ممکن است تعادل وزنی فرمول رتبه‌بندی را به هم بزند. اما در شبکه بیزی، این ارتباط معنایی به صورت یک «احتمال شرطی

کنترل شده» در پس زمینه گراف لحاظ می شود و سیستم می تواند وزن مترادف ها را به دقت تنظیم کند (مثلاً مترادف های دورتر، احتمال شرطی کمتری دریافت کنند).

پاسخنامه مسئله 11.29: تحلیل حساسیت پارامتر k_1 در عمل

(Sensitivity Analysis of k_1 in Practice)

بخش 1) مفاهیم کلیدی

در تابع رتبه بندی **Okapi BM25**، تأثیر تکرار یک کلمه بر نمره نهایی به صورت خطی نیست، بلکه از یک تابع «اشباع غیرخطی» پیروی می کند. پارامتر k_1 (که معمولاً مقدار پیش فرض آن بین 1.2 تا 2.0 تنظیم می شود)، به عنوان «کنترل کننده سرعت اشباع» (**Saturation Rate**) عمل می کند. تحلیل حساسیت این پارامتر به ما نشان می دهد که چگونه می توان با تغییر آن، رفتار مدل احتمالی **BM25** را بین 2 طیف کاملاً متفاوت «مدل بولی/باینری» و «مدل فضای برداری خام» جابه جا کرد.

بخش 2) تحلیل داده ها و فرمول پایه

برای تحلیل اثر بسامد کلمه (**TF**) و پارامتر k_1 ، فرض می کنیم طول سند برابر با میانگین طول اسناد است تا اثر پارامتر b خنثی شود. در این حالت، بخش مربوط به بسامد کلمه در فرمول **BM25** به شکل زیر ساده می شود:

$$W(TF) = \frac{TF \times (k_1 + 1)}{TF + k_1}$$

بخش 3) حل گام به گام (تحلیل نموداری و ریاضی)

گام اول: تحلیل رفتار نمودار در مقادیر استاندارد ($k_1 \approx 1.2$)

اگر نمودار تابع $W(TF)$ را رسم کنیم (محور افقی **TF** و محور عمودی نمره **W**)، برای مقادیر استاندارد مانند $k_1 = 1.2$ ، نمودار در ابتدا با شیب تندى بالا می رود. این یعنی بار 1 و 2 که کلمه در متن تکرار می شود، ارزش بسیار زیادی دارد. اما به سرعت شیب نمودار کم شده و به یک خط افقی (مجاذب) نزدیک می شود. از نظر ریاضی، حد تابع در بی نهایت برابر است با:

$$\lim_{TF \rightarrow \infty} \frac{TF \times (k_1 + 1)}{TF + k_1} = k_1 + 1$$

یعنی اگر $k_1 = 1.2$ باشد، نمره تکرار کلمه هرگز از عدد 2.2 فراتر نخواهد رفت؛ حتی اگر کلمه 1000 بار تکرار شده باشد.

گام دوم: افزایش پارامتر k_1 و تغییر شکل نمودار

هرچه مقدار k_1 را افزایش دهیم (مثلاً $k_1 = 5$ ، سپس $k_1 = 20$ و ...):

1. تأخیر در اشباع: مجانب نمودار ($k_1 + 1$) بالاتر می رود.

2. کاهش تحدب نمودار: نمودار دیرتر خم می‌شود و بخش اولیه آن که شیب ثابتی دارد، طولانی‌تر می‌گردد. به

بیان ساده‌تر، سیستم دیرتر از تکرار کلمات «خسته» می‌شود و برای تکرارهای **10** و **20** نیز همچنان پاداش قابل‌توجهی در نظر می‌گیرد.

گام سوم: اثبات همگرایی به مدل فضای برداری خام (حالت $k_1 \rightarrow \infty$)

مدل فضای برداری (**VSM**) در حالت خام خود (بدون لگاریتم‌گیری از **TF**)، فرض می‌کند که ارزش تکرار کلمه به صورت کاملاً خطی افزایش می‌یابد (یعنی سندی که کلمه را **10** بار آورده، دقیقاً **10** برابر ارزشمندتر از سندی است که آن را **1** بار آورده است).

اکنون بررسی می‌کنیم که اگر در **BM25**، مقدار **k₁** را به سمت بی‌نهایت میل دهیم، چه اتفاقی می‌افتد:

$$\lim_{k_1 \rightarrow \infty} \frac{TF \times (k_1 + 1)}{TF + k_1}$$

هنگامی که **k₁** بسیار بزرگتر از محدوده‌ی طبیعی **TF** باشد ($k_1 \gg TF$)، می‌توانیم از **TF** در مخرج کسر و همچنین از عدد **1** در صورت کسر (در برابر بزرگی **k₁**) صرف‌نظر کنیم:

$$\approx \frac{TF \times k_1}{k_1}$$

بنابراین معادله به شکل زیر ساده می‌شود:

$$W(TF) \approx TF$$

نتیجه‌گیری نهایی

با افزایش پارامتر **k₁**، سرعت اشباع کاهش یافته و نمودار از حالت یک منحنی تند، به یک خط راست مورب تبدیل می‌شود. از نظر ریاضی اثبات می‌گردد که وقتی $k_1 \rightarrow \infty$ ، مخرج کسر عملاً اثر خود را از دست می‌دهد و وزن اختصاص‌یافته به کلمه، مستقیماً و به صورت خطی برابر با خود فرکانس کلمه (**TF**) می‌شود. در این حالت حدی، مدل احتمالی **BM25** رفتار پیچیده خود را از دست داده و دقیقاً به مدل خطی و خام فضای برداری (**Linear Vector Space Model**) تقلیل می‌یابد.

نکته آموزشی:

این تحلیل، زیبایی و جامعیت معماری **BM25** را ثابت می‌کند. پارامتر **k₁** در واقع یک «پل ارتباطی» میان مدل‌های مختلف بازیابی اطلاعات است:

اگر **k₁ = 0** تنظیم شود: سیستم دقیقاً مدل استقلال باینری (**BIM**) را شبیه‌سازی می‌کند (نمره ثابت **1** برای هر $TF > 0$).

اگر **k₁ ≈ 1.2** تنظیم شود: سیستم مدل اشباع غیرخطی (**BM25** استاندارد) را اجرا می‌کند.

اگر $k_1 \rightarrow \infty$ تنظیم شود: سیستم مدل فضای برداری خام خطی (**Linear TF**) را شبیه‌سازی می‌کند.

در نتیجه، سیستم با یک فرمول واحد، می‌تواند رفتار **3** خانواده‌ی بزرگ از الگوریتم‌های بازیابی را بازتولید کند.

پاسخنامه مسئله 11.30: خلاصه و ترکیب مفاهیم (Summary and Concept Integration)

بخش 1) مفاهیم کلیدی

در سیر تکامل علم بازیابی اطلاعات (IR)، 3 مدل «فضای برداری» (VSM)، «استقلال باینری» (BIM) و «اوکاپی» (Okapi BM25) به عنوان ستون‌های اصلی تئوری و پیاده‌سازی سیستم‌های رتبه‌بندی شناخته می‌شوند. هر یک از این مدل‌ها با فلسفه ریاضی متفاوتی به مسئله نگاه می‌کنند: VSM بر پایه هندسه و جبر خطی، BIM بر پایه تئوری احتمالات کلاسیک و باینری، و BM25 بر پایه تلفیق احتمالات با رویکردهای تجربی و غیرخطی بنا شده است. خلاصه کردن ویژگی‌های ساختاری این 3 مدل، درک عمیقی از نحوه تکامل الگوریتم‌های جستجو به ما می‌دهد.

جدول مقایسه‌ای مدل‌های بنیادین بازیابی اطلاعات

معیار مقایسه	مدل فضای برداری (VSM)	مدل استقلال باینری (BIM)	مدل اوکاپی (BM25)
1. بسامد کلمه (TF)	پیوسته و غیرخطی: از فرکانس کلمه برای نشان دادن وزن و اهمیت محتوایی استفاده می‌کند (مانند $1 + \log TF$).	مطلقاً باینری: بسامد کلمه را کاملاً نادیده می‌گیرد و صرفاً «حضور» (1) یا «عدم حضور» (0) کلمه در متن ملاک است.	اشباع غیرخطی: از TF به صورت پیوسته استفاده می‌کند، اما با پارامتر k_1 سرعت رشد آن را کنترل و اشباع می‌کند.
2. نرمال‌سازی طول سند	نرمال‌سازی هندسی: طول سند را از طریق محاسباتی مانند نرمال‌سازی کسینوسی در مخرج کسر خنثی می‌کند.	نادیده گرفتن کامل: طول سند جایگاهی در فرمول‌بندی ندارد؛ اسناد طولانی شانس بازیابی بیشتری دارند.	نرمال‌سازی پویا: با استفاده از نسبت $dl/avdl$ و اعمال پارامتر b ، جریمه طول را به صورت منعطف اعمال می‌کند.
3. بازخورد ارتباط	انتقال هندسی: از طریق الگوریتم Rocchio، بردار پرسمان را در فضای چندبعدی به سمت اسناد مرتبط حرکت می‌دهد.	به‌روزرسانی بیزی: با شمارش کلمات در اسناد مرتبط، پارامترهای احتمالی p_i و u_i را مجدداً محاسبه می‌کند.	به‌روزرسانی فرمول: هسته BIM را حفظ کرده و با دریافت بازخورد، احتمالات هموارشده را جایگزین IDF اولیه می‌کند.

بخش 2) تحلیل و ترکیب تئوریک مفاهیم

برای درک عمیق‌تر این جدول، سیر تکاملی این 3 مدل را در 3 گام کالبدشکافی می‌کنیم:

گام اول: ایده هندسی VSM

مدل فضای برداری با معرفی مفاهیم TF و نرمال‌سازی طول سند، انقلابی در کیفیت رتبه‌بندی ایجاد کرد. این مدل به خوبی درک کرد که متون، ساختاری فراتر از 0 و 1 دارند. با این حال، بزرگترین ضعف VSM، نبود یک پشتوانه تئوریک ساختاریافته در نظریه احتمالات بود؛ فرمول‌های وزن‌دهی آن (مانند $TF \times IDF$) بیشتر بر اساس شَم مهندسی و آزمایش و خطا شکل گرفته بودند.

گام دوم: ساختار آماری BIM

مدل استقلال باینری برعکس VSM، از یک منطق ریاضی و احتمالی فوق‌العاده مستحکم (قضیه بیز و تئوری تصمیم‌گیری) بهره می‌برد. این مدل به ما آموخت که چگونه بازخورد کاربر را به صورت علمی پذیرا باشیم. اما این مدل برای دستیابی به این ریاضیات روان، مجبور شد چشمان خود را بر روی واقعیت‌های فیزیکی متن (مانند تکرار کلمات و طول اسناد) ببندد و همه‌چیز را باینری کند، که این امر کارایی آن را در عمل کاهش داد.

مدل **Okapi BM25** به عنوان نقطه اوج این تکامل شناخته می‌شود. طراحان این مدل متوجه شدند که می‌توان «روح احتمالی BIM» را با «جسم محتوایی VSM» ترکیب کرد. **BM25** فرمول احتمالی نسبت شانس در **BIM** را به عنوان هسته اصلی (بخش **IDF**) حفظ کرد، اما محدودیت باینری بودن را شکست و توابع پیوسته و اشباع‌شونده‌ی **TF** و نرمال‌سازی طول سند را به آن تزریق کرد.

نتیجه‌گیری نهایی

این مقایسه به دانشجویان نشان می‌دهد که در علم بازیابی اطلاعات، هیچ مدلی به صورت ناگهانی خلق نشده است؛ بلکه **BM25** به عنوان استاندارد طلایی امروز، بر روی شانه‌های تئوری احتمالات **BIM** و بینش‌های ساختاری **VSM** بنا شده است تا بتواند توازن کاملی میان «دقت ریاضی» و «واقعیت‌های زبان طبیعی» ایجاد کند.

پاسخنامه مسئله 11.31: پیاده‌سازی تابع محاسبه IDF در BM25 با پایتون

(Coding: Implementing BM25 IDF in Python)

بخش 1) مفاهیم کلیدی

همان‌طور که در مباحث نظری بررسی شد، فرمول **IDF** در مدل **Okapi BM25** با فرمول کلاسیک $\log(N/df_i)$ تفاوت دارد. این فرمول بر پایه احتمالات هموارشده‌ی رابرتسون-اسپارک جونز (**Robertson-Sparck Jones**) بنا شده است. در پیاده‌سازی‌های عملی و استاندارد (مانند آنچه در موتورهای جستجوی واقعی نظیر **Lucene** استفاده می‌شود)، برای جلوگیری از تولید مقادیر منفی برای کلماتی که در بیش از نیمی از اسناد ظاهر شده‌اند (**Stop-words**)، عدد 1 به آرگومان لگاریتم اضافه می‌شود. فرمول استاندارد و ایمن برای پیاده‌سازی به شکل زیر است:

$$IDF_i = \log\left(1 + \frac{N - df_i + 0.5}{df_i + 0.5}\right)$$

بخش 2) پیاده‌سازی در پایتون

این قطعه کد به گونه‌ای طراحی شده است که علاوه بر خوانایی بالا، قابلیت استفاده مستقیم در اسکریپت‌های ارزیابی خودکار پروژه‌ها (**Autograders**) را داشته باشد. مطابق با استانداردهای مهندسی نرم‌افزار، تمامی مستندات و توضیحات داخل قطعه کد برای جلوگیری از تداخل کاراکترها صرفاً با کاراکترهای لاتین نوشته شده‌اند.

bm25_idf.py

```
import math

def calculate_bm25_idf(df_dict, total_documents):
    """
    Calculates the Okapi BM25 IDF values for a given vocabulary.

    Args:
        df_dict (dict): A dictionary where keys are terms (strings)
            and values are Document Frequencies (integers).
        total_documents (int): The total number of documents in the collection (N).
    Returns:
        dict: A dictionary mapping each term to its computed BM25 IDF score.
    """
    bm25_idf_results = {}
```

```
for term, df in df_dict.items():
    # Applying the smoothed Robertson-Sparck Jones formula.
    # Adding 1.0 to the argument prevents negative IDF values for highly
    # frequent terms, which can disrupt the final ranking score.
    numerator = total_documents - df + 0.5
    denominator = df + 0.5
    idf_value = math.log(1.0 + (numerator / denominator))
    bm25_idf_results[term] = idf_value
return bm25_idf_results

# Execution Example for Testing and Validation
if __name__ == "__main__":
    # N: Total number of documents
    N = 1000
    # Sample dictionary representing df_i for various terms
    sample_dfs = {
        "algorithm": 15,      # Rare technical term
        "search": 250,        # Medium frequency term
        "the": 980             # Stop-word (appears in almost all documents)
    }
    computed_idfs = calculate_bm25_idf(sample_dfs, N)
    print("BM25 IDF Results:")
    for term, score in computed_idfs.items():
        # Rounding to 4 decimal places for cleaner output
        print(f"Term: '{term:10}' | DF: {sample_dfs[term]:<4} | BM25 IDF: {score:.4f}")
```

پاسخنامه مسئله 11.32: پیاده‌سازی کامل سیستم رتبه‌بندی BM25

(Coding: Full BM25 Ranking System)

بخش 1) مفاهیم کلیدی

پیاده‌سازی یک موتور رتبه‌بندی **BM25** نیازمند ترکیب 2 مرحله اصلی است: **پیش‌پردازش داده‌های سراسری** (مانند محاسبه میانگین طول اسناد و فراوانی کلمات) و **امتیازدهی محلی** (محاسبه نمره هر سند نسبت به پرسمان). هسته محاسباتی این پیاده‌سازی بر اساس معادله استاندارد زیر بنا شده است:

$$Score(Q, d) = \sum_{q_i \in Q} IDF(q_i) \times \frac{tf(q_i, d) \times (k_1 + 1)}{tf(q_i, d) + k_1 \times (1 - b + b \frac{dl}{avdl})}$$

بخش 2) پیاده‌سازی در پایتون

قطعه کد زیر با استفاده از کتابخانه‌های استاندارد و با رعایت اصول بهینه‌سازی محاسباتی طراحی شده است. به منظور سازگاری با محیط‌های توسعه و اتوگریدرها (**Autograders**)، تمامی متغیرها، مستندات و کامنت‌های درون کد منحصراً به زبان انگلیسی (**Latin characters**) نوشته شده‌اند.

```
bm25_ranking.py

import math
from collections import Counter

def rank_documents_bm25(query, documents, k1=1.5, b=0.75):
    """
    Ranks a list of documents against a given query using the Okapi BM25 algorithm.
    Args:
        query (list of str): The search query tokenized into a list of words.
        documents (list of list of str): A list where each element is a document
            represented as a list of words.
        k1 (float): Term frequency saturation parameter. Default is 1.5.
        b (float): Document length normalization parameter. Default is 0.75.
    Returns:
        list of tuples: A list of (document_index, bm25_score) sorted in
            descending order of relevance.
    """
    N = len(documents)
    if N == 0:
```

```

        return []
# Step 1: Calculate document lengths and average document length (avdl)
doc_lengths = [len(doc) for doc in documents]
avdl = sum(doc_lengths) / N if N > 0 else 0.0
# Step 2: Calculate Document Frequency (DF) for each unique query term
unique_query_terms = set(query)
df = Counter()
for doc in documents:
    unique_doc_terms = set(doc)
    for term in unique_query_terms:
        if term in unique_doc_terms:
            df[term] += 1
# Step 3: Pre-compute IDF values for the query terms
idf = {}
for term in unique_query_terms:
    n_q = df[term]
    # Using the smoothed Robertson-Sparck Jones IDF formula
    idf_value = math.log(1.0 + (N - n_q + 0.5) / (n_q + 0.5))
    idf[term] = idf_value
# Step 4: Calculate the total BM25 score for each document
scores = []
for doc_id, doc in enumerate(documents):
    score = 0.0
    doc_len = doc_lengths[doc_id]
    term_frequencies = Counter(doc)
    for term in unique_query_terms:
        # Skip computation if the query term does not exist in the document
        if term not in term_frequencies:
            continue
        tf = term_frequencies[term]
        # Calculate the TF normalization factor
        numerator = tf * (k1 + 1.0)
        denominator = tf + k1 * (1.0 - b + b * (doc_len / avdl))
        # Add to the document's total score
        score += idf[term] * (numerator / denominator)
    scores.append((doc_id, score))
# Step 5: Sort the final results in descending order based on the score
scores.sort(key=lambda x: x[1], reverse=True)
return scores

# Execution Example for Testing and Validation

if __name__ == "__main__":
    # Sample corpus
    sample_docs = [
        ["machine", "learning", "is", "a", "subset", "of", "artificial", "intelligence"],
        ["deep", "learning", "models", "require", "large", "datasets", "for", "training"],
        ["python", "is", "a", "great", "language", "for", "machine", "learning", "and", "data", "science"],
        ["reinforcement", "learning", "is", "used", "in", "robotics"]
    ]
    sample_query = ["machine", "learning"]
    # Run the ranking algorithm
    ranked_results = rank_documents_bm25(query=sample_query, documents=sample_docs, k1=1.2, b=0.75)
    print("Query:", sample_query)
    print("-" * 40)
    for doc_idx, score in ranked_results:
        snippet = " ".join(sample_docs[doc_idx])
        print(f"Doc {doc_idx:2d} | Score: {score:.4f} | Snippet: {snippet}")

```